BetterCP/M 1.0
User’s Guide

Document v. 1.0

20 October 2026

Contents

Part I - Using BetterCP/M **4**

Chapter 1 - Introduction . . . 4

Chapter 2 - An Overview of BetterCP/M . . . 7

Chapter 3 - Getting Started . . . 12

Chapter 4 - Working with Drives and Files . . . 17

Chapter 5 - The Command Environment . . . 25

Chapter 6 - Batch Processing . . . 30

Chapter 7 - Configuring BetterCP/M . . . 37

Chapter 8 - Resident Extensions . . . 44

Chapter 9 - Terminal Support . . . 50

Chapter 10 - Working with Disks . . . 58

Chapter 11 - Date and Time . . . 66

Chapter 12 - Troubleshooting . . . 71

Part II - Command and Utility Reference **80**

Chapter 13 - Introduction to the Command Reference . . . 80

13.1 Command Syntax Conventions . . . 80

13.2 Command Options . . . 82

13.3 Resident and Transient Versions . . . 84

13.4 Getting Help and Version Information . . . 86

Chapter 14 - File and Directory Commands . . . 88

14.1 DIR - Display Directory . . . 88

14.2 ERA - Erase Files . . . 92

14.3 REN - Rename Files . . . 96

14.4 TYPE - Display Text Files . . . 99

14.5 COPY - Copy Files . . . 101

14.6 MOVE - Move Files . . . 107

Chapter 15 - System and Environment Commands . . . 110

15.1 USER - Select User Area . . . 110

15.2 CLS - Clear Screen . . . 111

15.3 VER - Display Version Information . . . 112

15.4 SAVE - Save Memory to a File . . . 113

15.5 GET - Load a File into Memory . . . 114

15.6 GO - Execute at 0100H . . . 115

15.7 JUMP - Execute at an Address . . . 116

15.8 PEEK / P - Display Memory . . . 117

15.9 POKE - Modify Memory . . . 120
Chapter 16 - System Utilities . . . 123
16.1 CONFIG - System Configuration . . . 123
16.2 STAT - System and File Statistics . . . 128
16.3 TIME - Date and Time . . . 131
16.4 DUP - Disk Utilities . . . 133
16.5 SYSGEN - System Generation . . . 139
Chapter 17 - Batch and Extension Commands . . . 147
17.1 SUBMIT - Batch Command Processing . . . 147
17.2 XSUB - Extended Batch Input . . . 150
17.3 CPX - Command Processor Extension Manager . . . 152
17.4 RSX - Resident System Extension Management . . . 155

Appendices **161**

Appendix A - Command Summary . . . 161
Appendix B - Command-Line Editing Keys . . . 164
Appendix C - Control Characters and Keyboard Conventions . . . 168
Appendix D - File Attributes and File Specifications . . . 176
Appendix E - BetterCP/M and CP/M 2.2 Compatibility Notes . . . 188
Appendix F - System Messages and Error Reference . . . 197
Appendix G - Glossary . . . 210
Appendix H - Further Reading and Technical Documentation . . . 216

Part I - Using BetterCP/M

Chapter 1 - Introduction

1.1 What Is BetterCP/M?

BetterCP/M is an enhanced implementation of the CP/M operating system for Z80-based computers. It preserves the familiar CP/M 2.2 operating environment while adding new capabilities that make the system more convenient, flexible, and enjoyable to use.

If you've used CP/M before, you'll feel at home with BetterCP/M. You'll find the familiar command prompt, drive letters, user areas, and `.COM` programs. The commands you already know are still there, and ordinary CP/M applications continue to use the same programming interfaces they've always used.

But BetterCP/M offers considerably more than a conventional CP/M 2.2 system.

The command processor has been enhanced with command-line editing, command history, and more flexible ways of working with files and directories. Many familiar commands have gained additional capabilities, while new commands and utilities make routine operations easier.

BetterCP/M also introduces a system of resident extensions. These allow additional commands and services to be loaded into memory and made available throughout a working session, without requiring modifications to the operating system itself.

Despite these enhancements, BetterCP/M remains recognizably CP/M. Its purpose is not to replace the traditional CP/M environment with something entirely different, but to build upon it.

1.2 BetterCP/M and Digital Research CP/M 2.2

BetterCP/M is designed to maintain compatibility with Digital Research's CP/M 2.2, one of the most widely used operating systems of the early microcomputer era.

This compatibility extends beyond the appearance of the command prompt.

BetterCP/M preserves the standard CP/M programming environment, including the BDOS interface, file system conventions, program loading, and other facilities upon which existing CP/M software depends.

For most applications, the operating system should be transparent. A program written for CP/M 2.2 should ordinarily run under BetterCP/M without modification.

There are, however, some important distinctions between BetterCP/M and the original operating system.

BetterCP/M is a new implementation, not simply a patched copy of Digital Research's CP/M. Its internal organization differs in several respects, particularly where additional facilities or greater flexibility are required.

These differences are generally invisible to conventional CP/M applications. BetterCP/M aims to preserve the established programming environment while allowing the operating system itself to evolve.

Compatibility is a fundamental design objective, but it should not be interpreted as a guarantee that every program written for every CP/M computer will work unchanged. Some programs depend on particular hardware, undocumented operating-system behavior, or features supplied by individual manufacturers.

For additional information about compatibility, see Appendix E, *BetterCP/M and CP/M 2.2 Compatibility Notes*.

1.3 What's New in BetterCP/M?

BetterCP/M introduces a number of enhancements to the traditional CP/M environment. Some are immediately apparent when using the command prompt; others provide additional capabilities for system configuration, software development, and everyday operation.

An improved command environment

The BetterCP/M command processor provides command-line editing and a history of previously entered commands. Instead of retyping a long command after making a mistake, you can recall it, make the necessary correction, and execute it again.

The command environment also supports convenient drive and user-area selection, along with additional ways of locating and executing programs.

Enhanced commands and utilities

Familiar commands such as DIR, ERA, REN, and TYPE retain their traditional purposes while offering additional options.

BetterCP/M also supplies more capable file-management utilities, including COPY and MOVE, together with facilities for examining disks, configuring the system, and managing the operating environment.

Resident extensions

One of BetterCP/M's most significant additions is its extensible resident environment.

Command Processor Extensions (CPXs) can add commands to the command processor, while Resident System Extensions (RSXs) provide additional services that remain available while other programs execute.

Extensions can be loaded and unloaded as needed, allowing the system to be tailored to different tasks without permanently incorporating every facility into the operating system.

Improved terminal support

BetterCP/M provides loadable terminal-monitor facilities supporting several familiar terminal environments, including ADM-3A, VT52, and a supported subset of ANSI/VT100 operation.

These facilities help programs written for different terminal conventions operate on supported BetterCP/M systems.

System configuration and disk management

BetterCP/M provides additional facilities for configuring the operating environment, preparing disks, and installing or maintaining system disks.

The available facilities depend in part on the hardware or emulator being used, but the general command environment remains consistent across supported platforms.

Compatibility with existing CP/M software

Perhaps the most important feature is what BetterCP/M does *not* require you to change.

You can continue using conventional CP/M applications, programming tools, and file formats while taking advantage of BetterCP/M's additional facilities.

1.4 About This Manual

This manual explains how to install, configure, and use BetterCP/M 1.0.

It assumes that you have some familiarity with CP/M or similar command-line operating systems. You need not be an assembly-language programmer or understand the internal workings of the operating system.

The manual is divided into two principal parts.

Part I - Using BetterCP/M introduces the operating system and explains its facilities through descriptions, examples, and practical procedures.

You'll learn how to start BetterCP/M, work with drives and files, use the command environment, configure the system, manage resident extensions, and perform common maintenance tasks.

Part II - Command and Utility Reference provides detailed descriptions of BetterCP/M's commands and utilities. It is intended primarily for consultation when you need to look up command syntax, available options, or particular operating details.

The two parts complement one another. The introductory chapters demonstrate common operations and refer you to the command reference when more detailed information is needed.

Additional technical documentation is available for readers who want to understand BetterCP/M's architecture, develop software that uses its extended facilities, or adapt the operating system to new hardware.

You don't need those documents to use BetterCP/M. This User's Guide should provide the information necessary for ordinary operation.

Chapter 2 - An Overview of BetterCP/M

BetterCP/M builds upon the familiar organization of Digital Research's CP/M 2.2, but introduces several important changes to the way the operating system is organized and used.

You don't need to understand its internal workings to use BetterCP/M. Nevertheless, a general understanding of how its major components fit together will help you appreciate what the system offers and how its additional facilities work.

This chapter provides that introduction. Readers interested in a more detailed technical discussion should consult the Overall System Architecture.

2.1 The BetterCP/M Operating Environment

Like conventional CP/M, BetterCP/M provides an environment in which programs can be loaded from disk, executed, and given access to files and peripheral devices.

When the system is ready, it displays a command prompt:

```
A0>
```

From this prompt, you can enter commands, run programs, manage files, and configure the operating environment.

BetterCP/M retains the familiar CP/M conventions of lettered disk drives, numbered user areas, and executable programs with the `.COM` file type.

A program written for CP/M 2.2 ordinarily sees the same operating environment under BetterCP/M. It can access files, communicate with the console, and use the standard CP/M programming services without needing to know which operating system is running.

BetterCP/M extends this environment with additional commands, improved command-line facilities, and resident extensions that remain available while other programs execute.

These additions are designed to work alongside conventional CP/M software rather than require applications to be rewritten.

2.2 How BetterCP/M Is Organized

CP/M has traditionally been organized around three principal operating-system components: the Console Command Processor (CCP), the Basic Disk Operating System (BDOS), and the Basic Input/Output System (BIOS).

BetterCP/M retains this basic organization.

The Console Command Processor (CCP) provides the command-line environment. It accepts commands from the keyboard, interprets them, and either performs the requested operation or starts a program from disk.

The Basic Disk Operating System (BDOS) provides the standard CP/M services used by application programs. These include file operations, console input and output, disk selection, and other operating-system functions.

The Basic Input/Output System (BIOS) provides the interface between the operating system and the particular computer on which it runs. It handles the machine-dependent operations needed to communicate with disks, the console, and other devices.

The distinction between BDOS and BIOS is especially important to CP/M's portability. Much of the operating system can remain unchanged when it is adapted to another computer; the BIOS supplies the machine-specific operations.

BetterCP/M also provides two important extension mechanisms:

Command Processor Extensions (CPXs) add commands to the command environment.

Resident System Extensions (RSXs) provide additional resident services that can remain available while ordinary programs execute.

Together, these components provide the familiar CP/M environment while allowing BetterCP/M to offer capabilities beyond those of the original operating system.

2.3 The Command Processor

The command processor is the part of BetterCP/M with which users interact most frequently.

When you enter a command such as:

```
A0>DIR
```

the command processor determines how to carry it out.

Some commands are provided directly by the command environment. Others are supplied by installed CPXs. Still others are ordinary programs loaded from disk.

For example:

```
A0>DIR
A0>TYPE README.TXT
A0>MYPROG
```

The first two commands perform familiar file operations. The third starts a program named `MYPROG.COM`, assuming it can be found.

BetterCP/M's command processor offers several improvements over the traditional CP/M command environment.

You can edit a command before executing it, recall previously entered commands, and use convenient drive/user specifications.

You can also extend the command processor by loading CPXs. These make additional commands available without requiring the command processor itself to be permanently enlarged.

The distinction between resident and transient commands is normally invisible during everyday use. You enter the command name and its arguments in the usual way.

There are occasions when the distinction matters, however. BetterCP/M allows you to explicitly request a transient program, even when a resident command has the same name.

Chapter 5 explains the command environment in greater detail.

2.4 Programs and Memory

Like CP/M 2.2, BetterCP/M provides a region of memory called the **Transient Program Area**, or **TPA**.

This is where ordinary CP/M programs execute.

When you enter the name of a program, BetterCP/M locates its `.COM` file, loads it into memory, and transfers control to it.

For example:

```
A0>ED LETTER.TXT
```

starts the ED editor, provided the program is available.

When the program finishes, control returns to the operating system and the command prompt reappears.

BetterCP/M preserves the conventional CP/M program-loading environment. This allows existing CP/M applications to execute without requiring changes to their normal use of memory or operating-system services.

Resident extensions introduce an additional consideration.

CPXs supply commands while the command environment is active. Their command code, like the command processor itself, is reclaimable: a transient program can overwrite it. BetterCP/M reconstructs the command environment when control returns through a warm boot. RSXs remain in protected memory while programs execute, so their services stay available to applications.

Loading an RSX therefore involves a tradeoff: additional services in exchange for some reduction in available program memory.

BetterCP/M allows supported extensions to be unloaded when they are no longer needed, releasing their memory for other uses.

For most users, the important point is simple: **loaded RSXs can reduce program memory; reclaimable CPX command code does not permanently reduce the maximum transient program size.**

Chapter 8 explains how to manage extensions and examine their effect on available memory.

2.5 Disks and Files

BetterCP/M retains the familiar CP/M disk and file organization.

Disk drives are identified by letters, beginning with `A:`. Files are identified by names of up to eight characters and optional file types of up to three characters.

For example:

```
README.TXT
MYPROG.COM
REPORT.DAT
```

As in conventional CP/M, each drive also supports numbered user areas.

BetterCP/M extends the command environment by allowing drive and user-area combinations to be written together.

For example:

```
B3:
```

identifies drive B, user area 3.

This notation is particularly useful when working with files distributed among several user areas.

BetterCP/M also provides enhanced file-management commands, including facilities for locating, copying, moving, renaming, and examining files.

These commands retain familiar CP/M conventions while offering additional options.

The underlying filesystem remains compatible with conventional CP/M disk organization. BetterCP/M's enhancements do not require ordinary CP/M applications to adopt a new file format.

Chapter 4 explains how to work with drives and files.

2.6 Resident Extensions

One of BetterCP/M's distinguishing features is its ability to load additional operating-system facilities into memory.

These facilities are called **resident extensions** because they remain available after the command or program that loaded them has finished.

BetterCP/M provides two principal types.

Command Processor Extensions

A CPX adds commands to the command processor.

For example, BetterCP/M's standard resident command package provides familiar operations such as DIR, ERA, TYPE, REN, COPY, and MOVE.

Because these commands are resident, they can be invoked without loading a separate transient program each time.

Additional CPXs can be loaded when their commands are needed and unloaded when they are no longer required.

Resident System Extensions

An RSX provides services that remain available to the operating environment and other programs.

Terminal support is one example.

A terminal-monitor extension can interpret terminal control sequences used by an application and provide the corresponding display operations on the computer's screen.

BetterCP/M includes terminal facilities supporting several familiar terminal conventions, including ADM-3A, VT52, and a subset of ANSI/VT100.

These facilities allow supported applications to use the terminal environment they expect, even when the underlying display hardware differs.

Managing Extensions

BetterCP/M provides commands for examining, loading, and unloading resident extensions.

Extensions can also be loaded automatically during system startup, allowing you to establish a preferred working environment.

Ordinary warm boots preserve supported resident extensions, so they need not be reloaded every time a program terminates.

Chapter 8 explains how to use and manage extensions. For complete command syntax and options, consult the CPX and RSX entries in the Command Reference.

2.7 Hardware Independence

CP/M's separation of the operating system from the hardware-specific BIOS made it possible to run the same operating environment on many different computers.

BetterCP/M retains this approach.

The standard operating-system services and programming environment remain substantially the same across supported machines, while the BIOS supplies the operations needed for each particular platform.

Some facilities necessarily depend on the capabilities of the underlying computer. Disk formats, display behavior, peripheral devices, and certain configuration options may differ between platforms.

BetterCP/M's terminal extensions help bridge some of these differences by providing familiar terminal environments through machine-specific display support.

For users, this means that most commands and programs work in the same general way regardless of the supported platform.

For information about the machines and emulators currently supported, consult the *BetterCP/M Supported Platforms* documentation.

Developers interested in adapting BetterCP/M to additional computers should consult the *BetterCP/M Porting Guide*.

Chapter 3 - Getting Started

Getting started with BetterCP/M is much like getting started with conventional CP/M. Once the operating system has been prepared for your computer, you boot it, wait for the command prompt, and begin working.

The exact steps required to prepare and start BetterCP/M depend on the computer or emulator you're using. Rather than duplicate those instructions here, this chapter describes the general process and introduces the facilities you'll encounter when the system starts.

For current platform preparation and launch instructions, see Default high-capacity test disks and the z80pack / cpmsim adapter. For the selected System Disk build procedure, see Building the current BetterCP/M System Disks. These are working build/platform instructions; a consolidated Supported Platforms user manual is not yet available.

3.1 Obtaining BetterCP/M

BetterCP/M is distributed as a collection of operating-system files, utilities, and supporting documentation.

The distribution includes the software needed to run BetterCP/M on supported platforms, together with the programs used to configure and maintain the system.

Depending on the platform, the operating system may be supplied as a prepared disk image or as files from which a bootable system can be constructed.

A disk image is a file containing the contents of a disk. Emulators can generally use such images directly, while users of physical Z80 computers may need to transfer the image to suitable media.

BetterCP/M's distribution is organized around a system disk containing the operating system and the programs needed for ordinary operation. Additional utilities may be supplied separately.

The precise contents of the distribution may vary between releases and platforms.

Before preparing a system, make sure you have obtained the files appropriate to your computer. A disk image intended for one machine may not boot on another, even though both machines use the Z80 processor.

Use the distribution manifest to identify selected contents and Building the current BetterCP/M System Disks for preparation. Check the release report accompanying the images you obtain.

3.2 Preparing Your System

Before booting BetterCP/M, you'll need a supported computer or emulator and a suitable system disk.

For an emulator, preparation generally involves selecting the appropriate machine configuration and attaching the supplied disk image.

For a physical computer, you may need to prepare a floppy disk, configure a storage device, or transfer the system image using equipment appropriate to that machine.

BetterCP/M depends on the BIOS to communicate with the underlying hardware. Consequently, the system image must be appropriate to the computer on which it will run.

You should also consider how you intend to store your own files.

The system disk may have sufficient free space for experimentation, but a separate working disk is often more convenient. Keeping personal files separate from the operating-system distribution also makes it easier to replace or update the system disk.

If you're using physical removable media, it's a good idea to prepare a backup of the original system disk before making configuration changes.

The Model 4 test-disk instructions and z80pack / cpmsim adapter instructions describe the current emulator setup. These documents include development history; use instructions that match your build and platform.

What Is on Each Disk

The System Disk contains the operating system and the following user-0 files. The two platform versions use different boot and BIOS implementations.

Platform	Selected System Disk files
Both platforms: file commands	`DIR.COM`, `ERA.COM`, `TYPE.COM`, `REN.COM`, `VER.COM`, `COPY.COM`, `PIP.COM`, `ED.COM`
Both platforms: system utilities	`CONFIG.COM`, `DUP.COM`, `TIME.COM`, `SYSGEN.COM`, `STAT.COM`, `SUBMIT.COM`, `CPX.COM`, `RSX.COM`
Both platforms: support files	`README.TXT`, `STARTUP.SUB`, `DISK.FDB`, `RCP.CPX`, `R3PACK.RSX`
Both platforms: portable monitors and tests	`ADM3A.RSX`, `ADMTST.COM`, `VT52.RSX`, `VT52TST.COM`, `ANSI.RSX`, `ANSITST.COM`
TRS-80 Model 4 / trs80gp only	`M4SCRN.RSX`, `FREHDCLK.RSX` (requires FreHD hardware or emulation)
z80pack / cpmsim only	`ANSISCRN.RSX`, `ZPRTC.RSX`, `BYE.COM`

A supplied clock provider is available for explicit loading; neither platform loads one by default. Model 4 startup does load its screen backend and ADM-3A personality, as shown in Section 7.3.

The separate **Utilities Disk** is selected for `XSUB.COM`, `DUMP.COM`, ZSM4 and LINK, along with additional tools whose exact selections remain pending. Utilities Disk assembly and the remaining selections are separate from the current System Disk. The **CP/M Tools Disk** is another separate collection, selected for SYSINFO, BIOSINFO, DISKINFO, MEMINFO and related portable diagnostics; it is not the System Disk or Utilities Disk.

An implemented or buildable component is not automatically a selected distribution file. In particular, `CLS.COM`, `P2DOS.RSX`, the 104/105 clock-compatibility adapters, and `FDF.RSX` are not selected System Disk contents. CLS is available through `RCP.CPX`. Optional adapters and FDF support require separately prepared, compatible files; a maintainer can build these from the matching BetterCP/M source, then install extension files in A0 as described in Section 8.2. Diagnostic and conformance packages are also outside the selected distribution.

Historical ASM, LOAD and DDT examples assume separately obtained compatible tools; these programs are not selected BetterCP/M disk contents. ED is supplied on the System Disk. Refer to the distribution manifest and the disk's release report for the approved inventory and any outstanding packaging decisions, rather than treating a development disk or build directory as a release inventory.

3.3 Booting BetterCP/M

Once your system has been prepared, start BetterCP/M using the normal boot procedure for your computer.

During startup, BetterCP/M initializes the operating environment and establishes the configured command processor and resident facilities.

The system can also execute a startup command automatically.

The standard BetterCP/M distribution uses a command equivalent to:

```
SUBMIT STARTUP
```

This executes the commands contained in `STARTUP.SUB`, allowing the system to establish a preferred working environment.

Startup differs by platform: cpmsim displays the banner; the Model 4 loads M4SCRN and ADM3A, clears the display, then shows the banner. Section 7.3 lists both supplied scripts and explains customization.

When startup is complete, BetterCP/M displays its command prompt:

```
A0>
```

The system is now ready to accept commands.

Cold Boots and Warm Boots

CP/M distinguishes between two kinds of system startup.

A **cold boot** starts the operating system from its initial state. It establishes the operating environment and performs the configured startup operations.

A **warm boot** returns control to the operating system without repeating the entire cold-start procedure.

In BetterCP/M, an ordinary warm boot preserves supported resident extensions and other system state intended to survive program termination.

The startup command is executed during a cold boot, not during an ordinary warm boot.

This distinction becomes useful when you begin customizing BetterCP/M. For example, an extension loaded during startup can remain available throughout the working session without needing to be loaded again after every program.

Chapter 7 explains how to configure startup commands. Chapter 8 discusses the behavior of resident extensions during warm boots.

3.4 The Command Prompt

When BetterCP/M is ready for a command, it displays a prompt identifying the current drive and user area.

The prompt always includes both the drive and decimal user number, including user zero. For example:

```
A0>
```

This indicates that drive A is selected and the current user area is zero.

If you select another drive, the prompt changes accordingly:

```
A0>B:
B0>
```

BetterCP/M also allows you to select a user area directly:

```
A0>5:
A5>
```

The prompt now indicates drive A, user area 5.

You can select a drive and user area together:

```
A0>B3:
B3>
```

This selects drive B, user area 3.

Throughout this manual, examples include the command prompt to show how commands would appear on the screen. You type only the text following the prompt.

For example:

```
A0>DIR
```

means that you type `DIR` and press `RETURN`.

Chapter 4 explains drives and user areas in greater detail.

3.5 Your First Commands

Once BetterCP/M displays its prompt, you can begin entering commands.

If you've used CP/M before, many of these will already be familiar.

Displaying a Directory

Enter:

```
A0>DIR
```

BetterCP/M displays the files in the current drive and user area.

You can also examine another drive without selecting it:

```
A0>DIR B:
```

Or examine a particular drive and user area:

```
A0>DIR B3:
```

Displaying a Text File

To display the contents of a text file, use TYPE:

```
A0>TYPE README.TXT
```

The file's contents are displayed on the console.

TYPE is useful for examining documentation, program listings, command files, and other ordinary text files.

Copying a File

BetterCP/M provides a COPY command for copying files between drives and user areas.

For example:

```
A0>COPY README.TXT B:README.TXT
```

copies `README.TXT` from the current drive and user area to drive B.

The original file remains unchanged.

Running a Program

To execute a transient program, enter its name without the `.COM` extension.

For example:

```
A0>MYPROG
```

starts `MYPROG.COM`, provided BetterCP/M can locate it.

If the program accepts command-line arguments, enter them after its name:

```
A0>MYPROG INPUT.DAT
```

The arguments are passed to the program using the conventional CP/M command-line environment.

Displaying System Information

BetterCP/M provides the VER command for displaying version information.

```
A0>VER
```

This identifies the running BetterCP/M system.

These examples are enough to begin exploring the command environment. Chapter 5 introduces its additional capabilities, while Part II provides the complete command reference.

3.6 Restarting and Shutting Down

BetterCP/M follows the traditional CP/M model of operating on a computer that is started and stopped using the facilities provided by its hardware.

There is no general requirement to perform an operating-system shutdown before turning off a conventional CP/M computer.

However, **you should never interrupt a disk operation by switching off the computer or removing its storage media while data is being written**.

Before stopping BetterCP/M, allow any active program to finish and return to the command prompt.

If you've been editing a file, make sure it has been saved and the editor has terminated normally.

If a disk utility is formatting, copying, or modifying a disk, wait until the operation has completed.

On an emulator, stop the emulation using the emulator's normal controls. On physical hardware, follow the shutdown procedure appropriate to the machine.

Restarting the computer normally performs a cold boot.

An ordinary CP/M warm boot returns control to the operating environment without necessarily reinitializing the entire machine.

The distinction matters when using resident extensions or saved startup configuration, but for everyday operation the familiar CP/M rule remains a useful one: finish your work, make sure your files are saved, and return to the command prompt before stopping the system.

Chapter 4 - Working with Drives and Files

BetterCP/M retains the familiar CP/M system of disk drives, user areas, and file names. If you've used CP/M before, you'll recognize most of the conventions described in this chapter.

BetterCP/M also introduces some useful extensions, particularly in the way drives, user areas, and files can be specified in commands.

This chapter explains how to work with files and disks during ordinary operation. The complete syntax and options for individual commands are provided in Part II, *Command and Utility Reference*.

4.1 Drives and User Areas

Disk Drives

Like conventional CP/M, BetterCP/M identifies disk drives by letters.

Drives are named `A:` through `P:`, although the number actually available depends on your computer and its configuration.

The current drive is shown in the command prompt:

```
A0>
```

To select another drive, enter its letter followed by a colon:

```
A0>B:
B0>
```

The prompt changes to indicate that drive B is now selected.

You can also refer to files on another drive without changing the current drive.

For example:

```
A0>DIR B:
```

displays the directory of drive B while leaving drive A selected.

Similarly:

```
A0>TYPE B:README.TXT
```

displays `README.TXT` from drive B.

User Areas

CP/M divides each disk into numbered user areas.

BetterCP/M retains this convention, allowing files to be organized into separate logical areas on the same disk.

The standard CP/M user areas are numbered 0 through 15. BetterCP/M also supports extended user-area facilities where provided by its commands and configuration.

User areas are useful for organizing files by project, application, or purpose.

For example, you might keep programming tools in one user area, source files in another, and application data in a third.

Files in different user areas are ordinarily treated as belonging to separate collections, even when they reside on the same disk.

To select user area 5, enter:

```
A0>5:
A5>
```

The prompt now indicates drive A, user area 5.

To return to user area 0:

```
A5>0:
A0>
```

Drive/User Specifications

BetterCP/M provides a convenient way to specify a drive and user area together.

A **drive/user specification**, often abbreviated **DU**, combines the drive letter and user number.

For example:

```
A0:
B3:
C12:
```

These identify drive A/user 0, drive B/user 3, and drive C/user 12, respectively.

To select drive B, user area 3, enter:

```
A0>B3:
B3>
```

You can also use a DU specification when referring to a file:

```
A0>TYPE B3:README.TXT
```

This displays `README.TXT` from drive B, user area 3, without changing the current drive or user area.

The ability to specify a drive and user area directly is particularly useful when working with files distributed across several user areas.

Not every conventional CP/M program understands BetterCP/M's extended DU notation. It is a feature of the BetterCP/M command environment and of utilities that explicitly support it.

Programs using the standard CP/M programming interface continue to operate according to conventional CP/M drive and user-area rules.

4.2 File Names and File Specifications

BetterCP/M uses the standard CP/M file-naming convention.

A file name consists of a name of up to eight characters and an optional file type of up to three characters, separated by a period.

For example:

```
README.TXT
STAT.COM
MYPROG.ASM
DATA
```

The file type often indicates the purpose of the file.

Some common examples are:

File type	Common purpose
`.COM`	Executable program
`.ASM`	Assembly-language source
`.TXT`	Text file
`.SUB`	SUBMIT command file

File type	Common purpose
.BAK	Backup file
.HEX	Intel hexadecimal object file
.PRN	Assembly listing

These are conventions rather than restrictions imposed by the filesystem.

A file named REPORT.TXT, for example, is not required to contain text merely because its file type is .TXT.

File Specifications

A **file specification**, or **filespec**, identifies a file and may include a drive and user area.

For example:

```
README.TXT
B:README.TXT
B3:README.TXT
```

The first refers to README.TXT in the current drive and user area.

The second refers to the file on drive B, using the applicable user area.

The third explicitly identifies the file in drive B, user area 3.

If the current prompt is:

```
A2>
```

then:

```
TYPE README.TXT
```

displays README.TXT from drive A, user area 2.

By contrast:

```
TYPE B3:README.TXT
```

displays the file from drive B, user area 3.

BetterCP/M commands use file specifications extensively, and many accept the extended DU notation.

The individual command descriptions identify which forms are supported.

4.3 Wildcards

Wildcards allow a single file specification to refer to several files.

BetterCP/M retains the standard CP/M wildcard characters:

- ? matches a character in a particular position.
- * is a convenient abbreviation for matching the remaining characters in a file name or file type.

For example:

```
A0>DIR *.COM
```

lists files with the .COM file type.

Similarly:

```
A0>DIR LETTER.*
```

lists files named LETTER, regardless of their file type.

To list all applicable files:

```
A0>DIR *.*
```

Using Question Marks

A question mark matches any character in the corresponding position.

For example:

```
A0>DIR TEST?.COM
```

can match:

```
TEST1.COM
TEST2.COM
TESTA.COM
```

Wildcards are especially useful when performing operations on groups of related files.

For example:

```
A0>DIR B3:*.ASM
```

lists assembly-language source files in drive B, user area 3.

Wildcards and File Operations

Many BetterCP/M commands accept wildcards.

For example, you can use them to select files for copying, deleting, or examining.

However, **not every command permits wildcards in every operand**.

Some operations require an explicit destination filename, while others can operate on groups of files.

The command reference explains the rules for each command.

Be especially careful when using wildcards with commands that modify or delete files. A specification such as `*.*` can select every applicable file in a directory.

4.4 File Attributes

In addition to its name and file type, a CP/M file can have attributes that affect how it is treated.

BetterCP/M supports the familiar read-only, system, and archive attributes.

Read-Only Files

The **read-only (R/O)** attribute protects a file against ordinary operations that would modify or erase it.

This is useful for files that should not be changed accidentally, such as important programs or reference data.

BetterCP/M respects read-only protection.

Some utilities provide explicit options for overriding this protection, but such operations should be used carefully.

System Files

The **system (SYS)** attribute identifies a file as a system file.

Some directory commands may omit system files from their ordinary listings unless instructed to include them.

In BetterCP/M 1.0, the SYS attribute does not automatically make a file visible across different user areas. Normal drive and user-area rules continue to apply.

Archive Files

The **archive (ARC)** attribute can be used to identify files for backup or other file-management purposes.

BetterCP/M preserves this attribute where appropriate, but does not impose a universal automatic backup policy based on it.

Individual utilities may provide options that make use of archive attributes.

Examining and Changing Attributes

BetterCP/M provides utilities for displaying and changing file attributes.

For example, STAT can be used to examine file information and perform supported attribute operations.

The complete procedures are described in the STAT command reference.

4.5 Changing Disks

Many CP/M computers use removable storage media.

Whether you're working with physical floppy disks or disk images attached to an emulator, it's important to understand how BetterCP/M handles changes in media.

Never remove or replace a disk while an operation is reading from or writing to it.

Doing so can interrupt a file operation and may damage files or directory information.

CP/M maintains information about the disks it is using. If a disk is replaced without the operating system recognizing the change, information associated with the previous disk could otherwise be used incorrectly.

To help prevent this, BetterCP/M retains CP/M's protective behavior when a disk change is detected or suspected.

In some circumstances, a disk may be treated as read-only until the operating system has safely recognized the new medium.

If BetterCP/M reports that a disk has become read-only after a media change, follow the appropriate disk-reset or warm-boot procedure before attempting further writes.

Do not attempt to force a write merely to bypass the warning.

A Practical Precaution

When working with removable disks, it's good practice to finish the current operation and return to the command prompt before changing media.

If you're using an emulator, follow its procedure for replacing or remounting disk images.

The details vary between systems, so consult the appropriate platform documentation when necessary.

4.6 Disk Capacity and Free Space

Not all CP/M disks have the same capacity.

The amount of storage available depends on the disk format, which determines how the disk is organized and how much space is available for files.

BetterCP/M supports different disk formats through its disk and system configuration facilities.

File Storage

CP/M allocates file space in units called **allocation blocks**.

An allocation block contains a fixed amount of disk space, determined by the disk format.

A small file may occupy an entire allocation block, even if it contains only a few bytes of data.

Consequently, the amount of free space on a disk does not always decrease by exactly the size of each file written to it.

Directory Capacity

A CP/M disk also has a limited number of directory entries.

Directory entries contain the information used to identify and locate files.

This means that a disk can sometimes have unused storage space but insufficient directory capacity to create additional files.

If you encounter this situation, deleting unneeded files or using a disk format with a larger directory may resolve the problem.

Examining Disk Information

BetterCP/M provides utilities for examining disk capacity, available space, and related information.

STAT provides disk and file information, while other utilities can examine or configure supported disk formats.

Chapter 10 explains disk operations in more detail.

For complete command syntax, consult the appropriate entries in Part II.

4.7 Copying, Moving, and Managing Files

BetterCP/M provides familiar file-management commands together with additional facilities that make it easier to work across drives and user areas.

The most commonly used operations are listing files, copying them, moving them, renaming them, and deleting files that are no longer needed.

Listing Files

To display the files in the current drive and user area:

```
A0>DIR
```

To list files on another drive:

```
A0>DIR B:
```

To list files in a particular drive and user area:

```
A0>DIR B3:
```

BetterCP/M's DIR utility provides additional options for selecting, sorting, and displaying files.

Copying Files

COPY creates a copy of a file while leaving the original in place.

For example:

```
A0>COPY README.TXT B:README.TXT
```

copies `README.TXT` from the current drive and user area to drive B.

BetterCP/M also supports copying files between explicitly specified user areas.

For example:

```
A0>COPY B3:REPORT.TXT C2:REPORT.TXT
```

copies the file from drive B, user area 3, to drive C, user area 2.

COPY provides additional options for handling existing destination files, preserving attributes, and performing other file-management operations.

Moving Files

MOVE transfers a file from one location to another.

Unlike COPY, MOVE is intended to remove the file from its original location after the move has been completed successfully.

For example:

```
A0>MOVE REPORT.TXT B:REPORT.TXT
```

moves `REPORT.TXT` from the current drive and user area to drive B.

Moving files can be convenient when reorganizing disks or user areas.

However, remember that MOVE changes the location of the original file. If you want to retain the original, use COPY instead.

Renaming Files

REN changes a file's name.

For example:

```
A0>REN NEWNAME.TXT=OLDNAME.TXT
```

renames `OLDNAME.TXT` to `NEWNAME.TXT`.

Renaming is useful for organizing files, changing file types, or assigning more descriptive names.

BetterCP/M provides additional REN options beyond the conventional CP/M command.

Deleting Files

ERA deletes files.

For example:

```
A0>ERA OLD.DAT
```

deletes `OLD.DAT` from the current drive and user area.

Wildcards can be used to select multiple files:

```
A0>ERA *.BAK
```

This selects backup files matching the specification.

Use particular care with ERA. Deleting files may make their contents unrecoverable.

BetterCP/M provides additional ERA facilities, including options for confirmation and handling protected files.

Choosing the Right Command

COPY, MOVE, REN, and ERA perform different operations.

COPY duplicates files.

MOVE relocates files.

REN changes their names.

ERA removes them.

These distinctions are straightforward, but they become especially important when working with wildcard specifications or files on different drives.

For complete descriptions of these commands, including their options, confirmation behavior, and error messages, consult Part II, *Command and Utility Reference*.

Chapter 5 - The Command Environment

The command processor is the part of BetterCP/M you'll use most often. It accepts commands, starts programs, and provides the familiar CP/M prompt.

If you've used CP/M before, you already know the basics. BetterCP/M retains the traditional command-line environment while adding conveniences such as command editing, command history, and more flexible ways of finding and running programs.

This chapter introduces those facilities and explains how they work together.

5.1 Entering Commands

When BetterCP/M is ready to accept a command, it displays a prompt:

```
A0>
```

The prompt identifies the current drive and user area, including zero.

Enter a command by typing its name, followed by any required arguments, and pressing `RETURN`.

For example:

```
A0>DIR
```

displays the directory of the current drive and user area.

To display a text file:

```
A0>TYPE README.TXT
```

Commands are not case-sensitive, so:

```
A0>dir
```

has the same meaning as `DIR`.

Many commands accept additional arguments or options. For example:

```
A0>DIR B3:*.COM
```

lists executable files in drive B, user area 3.

BetterCP/M accepts commands provided directly by the command processor, commands supplied by resident extensions, and ordinary programs stored on disk.

For most purposes, you needn't worry about the distinction. You enter the command in the same general way regardless of where it comes from.

The complete syntax for individual commands is given in Part II, *Command and Utility Reference*.

5.2 Command-Line Editing

Anyone who has used a command-line operating system knows how frustrating it can be to type a long command, discover a mistake, and have to enter the entire line again.

BetterCP/M makes this easier by allowing you to edit a command before executing it.

You can move the cursor through the command line, correct characters, insert new text, and remove unwanted text.

For example, suppose you enter:

```
A0>COPY B3:REPROT.TXT C2:REPORT.TXT
```

but notice that you've mistyped `REPORT` as `REPROT`.

Rather than erase the entire command and start over, you can move the cursor back to the mistake and correct it.

The command isn't executed until you press `RETURN`, so you can make as many corrections as necessary before submitting it.

BetterCP/M uses editing conventions familiar to users of WordStar and other CP/M programs, although not every command-line function uses the same keys as WordStar.

Use Ctrl-S/D to move left/right, Ctrl-A/F to move by word, Ctrl-G/H to delete at/left of the cursor, and Ctrl-V to toggle insert/overwrite. Ctrl-U or Ctrl-X discards the line. Ctrl-E continues on a new physical screen line without executing the command.

The complete key assignments are listed in Appendix B, *Command-Line Editing Keys*.

5.3 Command History

BetterCP/M remembers previously entered commands, allowing you to recall, edit, and execute them again without retyping the entire command.

Press **Ctrl-K** to recall an earlier command, or **Ctrl-J** to move forward through the command history.

On the original Model 4 keyboard, Up and Down also navigate history. Host emulators and other consoles may translate arrow keys differently; Ctrl-K and Ctrl-J are the portable assignments.

For example, suppose you've just entered:

```
A0>COPY B3:REPORT.TXT C2:REPORT.TXT
```

and then realize that you also want to copy another file.

Instead of retyping the command, recall it from history, change the filename, and press `RETURN`.

The recalled command behaves just like one you've typed yourself. You can execute it unchanged or edit it first.

Command history remains available after ordinary programs finish and return to the prompt. It also survives a normal warm boot.

A cold boot begins with an empty history.

Because history storage is limited, older entries are eventually discarded as new commands are entered.

Command history is especially convenient when working with long file specifications or commands containing several options.

5.4 Selecting Drives and User Areas

BetterCP/M provides several convenient ways to move among drives and user areas.

To select drive B:

```
A0>B:
B0>
```

To select user area 5 on the current drive:

```
B0>5:
B5>
```

To select both a drive and user area:

```
B5>C3:
C3>
```

This selects drive C, user area 3.

BetterCP/M refers to a drive and user-area combination as a **drive/user**, often abbreviated **DU**.

You can also specify a drive and user area when entering a command without changing your current selection.

For example:

```
A2>DIR B3:
A2>
```

displays the directory of drive B, user area 3, and then returns to the unchanged prompt.

This is useful when working with files scattered across several drives or user areas.

Chapter 4 provides a fuller explanation of drive/user specifications.

5.5 Resident and Transient Commands

A resident command is supplied by the active command environment; a transient utility is loaded from a `.COM` file. Some commands have both forms. For example:

```
A0>DIR *.COM
A0>:DIR /WHEREIS ED.COM
```

The first uses resident DIR when available. The leading colon in the second explicitly loads `DIR.COM`, whose additional `/WHEREIS` mode locates files across drives and users.

Use this explicit form when you need a transient-only option. Section 13.3 gives the complete resident/transient selection rules; Section 5.6 explains where transient programs are found.

5.6 Command Search and Execution

When you enter a command, BetterCP/M determines how to execute it.

It first recognizes commands that select drives or user areas. It then checks commands built into the command processor, followed by those supplied by installed Command Processor Extensions.

If none of these recognizes the command, BetterCP/M looks for a transient program.

Finding Programs

When you enter an ordinary program name without specifying a drive or user area, BetterCP/M searches the current drive and user area first.

If the program isn't found there, it searches drive A, user area 0.

For example, suppose you're working in user area 3 on drive B:

```
B3>
```

You enter:

```
B3>MYPROG
```

BetterCP/M first looks for `MYPROG.COM` in B3:.

If it isn't found there, BetterCP/M looks in A0:.

This is convenient because commonly used programs can be kept in A0: while your working files remain in other locations.

If a program is found in A0:, it runs with your original drive and user area still selected.

You can therefore keep applications in A0: without having to move your working files there.

Specifying a Program's Location

You can also identify a program's location explicitly:

```
B3>C2:MYPROG
```

This tells BetterCP/M to look for the program in C2:.

When a location is explicitly specified, BetterCP/M uses that location rather than searching A0: as a fallback.

The same general lookup rules apply when you explicitly request a transient program using the `:` or `.` prefix.

Running Programs

When BetterCP/M finds a transient program, it loads the program and starts it.

For example:

```
A0>MYPROG INPUT.DAT
```

runs `MYPROG.COM` and supplies `INPUT.DAT` as a command-line argument.

The program receives its arguments through the conventional CP/M command-line environment.

When it finishes, BetterCP/M restores the command environment and displays the prompt again.

These facilities allow ordinary CP/M programs to run without needing to know how BetterCP/M's enhanced command processor works internally.

5.7 Command Processor Extensions

Command Processor Extensions, or **CPXs**, allow additional commands to become part of the resident command environment.

The standard BetterCP/M configuration includes `RCP.CPX`, which provides commonly used commands such as:

```
DIR  ERA  TYPE  REN  USER
CLS  VER  COPY  MOVE
```

Because these commands are resident, they're available without loading separate transient programs.

Additional CPXs can be installed to provide other commands.

More than one CPX can be active at a time, allowing the command environment to be customized for different purposes.

For example, a user might load an extension providing specialized commands for a particular application or working environment.

CPXs can be listed, loaded, and unloaded while the system is running.

Chapter 8 explains how to manage resident extensions. For complete CPX command syntax and options, consult the CPX entry in Part II.

5.8 Interrupting Commands and Programs

Most commands return to the BetterCP/M prompt when their work is complete.

Sometimes, however, you may need to interrupt a command before it finishes.

Many CP/M programs recognize **Ctrl-C** as a request to stop the current operation.

BetterCP/M commands also use Ctrl-C where appropriate to abandon an operation and return control to the command environment.

The exact effect depends on the program being executed.

Some programs may ignore Ctrl-C, while others recognize it only at particular points.

A warm boot also returns control to the operating system and restores the command environment.

Interrupting Disk Operations

Be particularly careful when interrupting commands that modify files or disks.

For example, if a disk-copying operation is writing data when it is interrupted, the destination disk may be incomplete.

Likewise, interrupting a file operation can leave a partially written file.

Whenever possible, allow disk operations to finish normally.

If an operation fails or is interrupted unexpectedly, examine the affected files or media before continuing.

BetterCP/M's utilities provide error messages and, where appropriate, recovery procedures to help deal with these situations.

Chapter 6 - Batch Processing

BetterCP/M can execute a series of commands automatically, saving you the trouble of entering the same instructions over and over again.

This facility is called **batch processing**. If you've used Digital Research CP/M before, you'll already be familiar with SUBMIT, the utility that reads commands from a file and executes them in sequence.

BetterCP/M retains the traditional CP/M batch-processing environment, including SUBMIT command files and the XSUB extension, while allowing batch files to take advantage of BetterCP/M's enhanced commands and utilities.

Batch processing is particularly useful for repetitive tasks such as assembling programs, copying files, preparing working disks, or performing a series of operations that you would otherwise enter individually.

This chapter introduces batch processing and shows how to create and use command files. Complete descriptions of SUBMIT and XSUB are provided in Part II, *Command and Utility Reference*.

6.1 Introduction to Batch Processing

Suppose you're working on an assembly-language program. Each time you make a change, you might need to assemble the source file, load the resulting program, and run a test.

The ASM and LOAD examples in this chapter assume separately obtained compatible tools. They are not selected System Disk contents; Section 3.2 lists the standard files and the separate Utilities Disk selection.

You could enter each command separately, but if you perform the same sequence repeatedly, it's more convenient to place the commands in a file.

For example, a command file might contain:

```
ASM MYPROG
LOAD MYPROG
MYPROG
```

Instead of typing these commands individually, you can instruct BetterCP/M to execute the entire sequence.

The commands are processed in order, just as though you had entered them at the command prompt.

A batch file can contain ordinary CP/M commands, BetterCP/M commands, and the names of transient programs. Commands supplied by resident extensions can also be used, provided the necessary extensions are loaded.

Batch files are especially convenient for operations that require several steps but little interaction with the user.

6.2 SUBMIT and SUB Files

BetterCP/M uses the familiar CP/M **SUBMIT** utility to execute command files.

A SUBMIT file is an ordinary text file containing commands, usually with the file type `.SUB`.

For example, you might create a file named:

```
BACKUP.SUB
```

containing:

```
DIR
:COPY *.ASM B:
DIR B:
```

The first command displays the current directory. The second copies the selected assembly-language source files to drive B. The third displays the directory of drive B so you can examine the results.

To execute the file, enter:

```
A0>SUBMIT BACKUP
```

SUBMIT reads the commands from `BACKUP.SUB` and arranges for them to be executed in sequence. The leading colon in `:COPY` forces the transient `COPY.COM` utility; make sure it is available on the command search path.

You don't need to type the `.SUB` extension when invoking SUBMIT.

Keep drive A user 0 writable: SUBMIT and XSUB share the queue `A0:$$$.SUB`, even when the source script and working files are elsewhere. Queue access restores your drive/user context, so commands may change location without moving the queue. See the SUBMIT reference for limits and cancellation behavior.

Creating a SUBMIT File

Because SUBMIT files contain ordinary text, you can create them with any CP/M text editor.

For example, you could use ED to create `BACKUP.SUB`, enter the commands you want executed, and save the file.

You can also prepare SUBMIT files on another computer and transfer them to your CP/M system, provided they are saved in a suitable text format.

Each command should appear on its own line.

When writing a SUBMIT file, it's generally best to use commands that you have already tested individually. This is particularly important for commands that modify or delete files.

Running a SUBMIT File

To execute a command file named `BUILD.SUB`, enter:

```
A0>SUBMIT BUILD
```

BetterCP/M processes the commands in the file in sequence.

The commands are executed in the normal command environment, so they can use the same drives, user areas, and resident facilities available at the command prompt.

A command file can also invoke ordinary transient programs.

If a program or command cannot be found, or if an operation fails, the result depends on the command and the batch-processing facilities involved. Don't assume that every failure will be ignored or that subsequent commands will always execute.

For important procedures, especially those that modify disks or files, test the complete sequence before relying on it for unattended operation.

6.3 Parameters and Substitution

SUBMIT files can be made more flexible by accepting parameters.

Rather than creating a separate command file for every program you want to assemble, you can create one file that accepts the program name as an argument.

For example, suppose `BUILD.SUB` contains:

```
ASM $1
LOAD $1
$1
```

Here, `$1` represents the first parameter supplied when SUBMIT is invoked.

To assemble and run a program named `TEST`, enter:

```
A0>SUBMIT BUILD TEST
```

SUBMIT substitutes `TEST` for `$1`, producing the equivalent of:

```
ASM TEST
LOAD TEST
TEST
```

This allows a single SUBMIT file to be reused with different programs.

Using Multiple Parameters

A SUBMIT file can accept more than one parameter.

For example, a file named `COPYFILE.SUB` might contain:

```
:COPY $1 $2
```

You could then enter:

```
A0>SUBMIT COPYFILE REPORT.TXT B:REPORT.TXT
```

The resulting command would be:

```
:COPY REPORT.TXT B:REPORT.TXT
```

This is a simple example, but parameter substitution becomes particularly useful in longer command files.

Literal Dollar Signs in Command Files

SUBMIT treats dollar signs as parameter markers. Write `$$` in a `.SUB` file wherever the executed command needs a literal `$`, including attribute selectors such as `$ARC`, `$RO`, and `$SYS`. An unescaped `$ARC` is an invalid parameter reference and causes `Parameter Error.`

For example, save these lines as `ARCBACK.SUB`:

```
:COPY /B /O /V /BACKUP B[-]:*.DOC[$$ARC] D0:
DIR D0:*.DOC
```

With the source and destination drives available, run:

```
A0>SUBMIT ARCBACK
```

SUBMIT expands `$$ARC` to `$ARC`, so the first command executed is:

```
:COPY /B /O /V /BACKUP B[-]:*.DOC[$ARC] D0:
```

This copies archived `.DOC` files from all user areas on B to D user 0, permits replacement of writable destination files, verifies each copy, and clears ARC after a successful backup. Read-only destinations remain protected. The leading colon selects the transient COPY utility. At the command prompt, type `$ARC` directly; the doubling is needed only in the `.SUB` source.

Parameter Substitution Is Textual

SUBMIT substitutes parameter text into the command file. It doesn't interpret the meaning of the resulting command.

For example, if a parameter contains a filename, SUBMIT places that filename into the command wherever the corresponding parameter reference appears.

The command that eventually executes is responsible for interpreting the filename, options, and other arguments.

This distinction is useful to remember when constructing more elaborate command files.

It also means that an incorrect parameter can produce an incorrect command. When a SUBMIT file performs destructive operations, take care to verify the parameters before executing it.

6.4 XSUB

Ordinary SUBMIT files execute commands entered at the command-processor level. Some programs, however, ask for additional input after they have started.

For example, a program might display a prompt asking for a filename, an option, or some other information.

Normally, the user must type a response at the keyboard.

XSUB extends batch processing by allowing a SUBMIT file to supply input to programs that read from the console.

This makes it possible to automate certain interactive programs that would otherwise require someone to sit at the keyboard.

How XSUB Works

Consider a program that prompts for two filenames:

```
Source file:
Destination file:
```

Without XSUB, you would start the program and type the filenames when requested.

With XSUB, a suitably prepared command file can supply those responses automatically.

XSUB works with programs that obtain their input through the supported CP/M buffered console-input interface.

It does not automatically provide input to every program. Software that reads the keyboard directly, uses its own device drivers, or employs other input methods may not be compatible with XSUB.

Using XSUB

XSUB is normally activated as part of a SUBMIT procedure.

Once active, it allows subsequent submitted input to be passed to compatible programs.

Put `XSUB` before the program, then put its buffered-input responses immediately after the program invocation. Section 17.2 gives a complete `XDEMO.SUB` procedure using ED and explains the Utilities Disk and BATCHIO requirements.

Returning to Keyboard Input

BetterCP/M's XSUB facility allows a program to continue receiving ordinary keyboard input after its submitted input has been exhausted.

For example, a program may receive its initial responses from the batch file and then continue interactively.

This is useful when only part of a program's operation needs to be automated.

XSUB uses resident support while supplying batch input. In some circumstances, that support may remain loaded after the batch has finished.

If necessary, it can be removed with:

```
A0>RSX UNLOAD BATCHIO
```

This releases the resident batch-input facility when it is no longer needed.

For additional information about managing resident extensions, see Chapter 8.

6.5 Batch Execution and Termination

A SUBMIT file is executed as a sequence of commands, but there are some important differences between entering commands manually and allowing them to execute automatically.

Interactive Commands

A command executed from a SUBMIT file does not necessarily become noninteractive.

For example, a file-copying command may ask whether an existing destination file should be replaced.

If the command normally requires confirmation, it may still require confirmation when executed from a SUBMIT file.

This is particularly important when preparing batch files that you intend to run unattended.

BetterCP/M provides options for some utilities that allow you to specify how such situations should be handled.

Interrupting a Batch

Batch execution can be interrupted by commands, programs, or operating-system conditions that terminate the current batch procedure.

The precise behavior depends on the program involved and the way it returns control to the operating system.

If a batch operation is interrupted, do not assume that all commands in the file have been executed.

For example, a batch that copies several files may have copied some files successfully before encountering an error.

Before restarting such a procedure, examine the files or disks involved to determine which operations have already completed.

Errors During Batch Processing

Some commands report errors and return control to the command processor. Others may terminate the active batch procedure.

The presence of an error message does not necessarily tell you whether subsequent commands will execute.

For this reason, a batch file should not be treated as an automatic error-recovery system.

When preparing an important batch procedure, test how its commands behave when a file is missing, a destination already exists, or a disk operation fails.

Warm Boots and Batch Processing

CP/M uses warm boots as part of its normal program-execution environment.

BetterCP/M preserves the mechanisms needed for conventional SUBMIT operation, allowing batch execution to continue across ordinary program execution where appropriate.

Resident extensions used by the batch environment are also subject to BetterCP/M's normal extension-management rules.

A cold boot, however, establishes a new operating session rather than continuing a previous batch procedure.

6.6 Practical Examples

The following examples illustrate some common uses of batch processing.

Example 1: Assembling a Program

Suppose you frequently assemble and run a program while developing it.

Create `BUILD.SUB` containing:

```
ASM $1
LOAD $1
$1
```

To build and run `TEST`, enter:

```
A0>SUBMIT BUILD TEST
```

This executes the equivalent of:

```
ASM TEST
LOAD TEST
TEST
```

You can use the same command file for another program simply by changing the parameter.

Example 2: Copying Source Files

Suppose you want to copy all assembly-language source files from one user area to another.

Create `SOURCES.SUB` containing:

```
:COPY B1:*.ASM C3:
DIR C3:*.ASM
```

Then enter:

```
A0>SUBMIT SOURCES
```

The first command copies the selected files. The second displays the destination directory.

If destination files already exist, COPY may request confirmation before replacing them.

For unattended execution, you can use COPY's batch option.

Example 3: Copying Files Without Prompts

BetterCP/M's transient COPY utility provides a `/B` option for batch-oriented operation. Use `:COPY` to force `COPY.COM`, even when resident COPY is installed.

For example:

```
:COPY B1:*.DAT C3: /B
```

This copies matching files without stopping to ask how existing destination files should be handled.

If a writable destination already exists, COPY reports `FILE EXISTS` for that file and continues with the remaining files.

It does not automatically overwrite the existing file.

You can specify a different policy explicitly.

To overwrite existing writable destination files:

```
:COPY B1:*.DAT C3: /B /O
```

To skip existing destination files:

```
:COPY B1:*.DAT C3: /B /S
```

In both cases, files without existing destinations are copied normally.

Read-only destination files remain protected.

The `/B` option applies to COPY itself. It does not make other commands in the SUBMIT file noninteractive.

For the complete description of COPY's options, consult the COPY command reference.

Example 4: Customizing System Startup

BetterCP/M uses a SUBMIT file as part of its normal startup configuration.

The standard distribution provides:

```
STARTUP.SUB
```

The configured cold-start command is:

```
SUBMIT STARTUP
```

The supplied startup file follows the platform-specific sequence listed in Section 7.3: banner only on cpmsim; screen backend, ADM-3A personality, screen clear, then banner on Model 4.

You can edit this file to perform additional operations automatically when the system starts.

For example, startup commands can load resident extensions needed for your preferred working environment.

Because the startup procedure runs automatically, it's best to test new commands individually before adding them to `STARTUP.SUB`.

A mistake in an ordinary batch file is inconvenient. A mistake in a startup file can make every cold boot inconvenient until the file is corrected.

Chapter 7 explains how to configure the startup command, while Chapter 8 describes loading and managing resident extensions.

Chapter 7 - Configuring BetterCP/M

BetterCP/M provides several facilities for customizing the operating environment to suit your computer and the way you prefer to work.

You can change the command executed when the system starts, configure disk drives and formats, select peripheral devices, and save settings for subsequent sessions.

Most of these operations are performed using **CONFIG**, BetterCP/M's system configuration utility.

If you've used conventional CP/M, you'll be familiar with the idea of configuring the BIOS and regenerating a system disk to accommodate different hardware. BetterCP/M makes many of these operations more convenient by providing configuration facilities that can be used from the command prompt.

Some settings can be changed while the system is running. Others must be saved to the system disk and take effect the next time BetterCP/M starts.

This chapter explains how to use these facilities. Complete descriptions of CONFIG and the related utilities appear in Part II, *Command and Utility Reference*.

7.1 Configuration Facilities

BetterCP/M distinguishes between settings currently in use and settings saved for future system startup.

This distinction is important because changing a setting does not necessarily make that change permanent.

For example, you might temporarily configure a drive to read a particular disk format. That configuration can remain active during your current session without becoming the default the next time you boot BetterCP/M.

Similarly, changing the system's startup command does not immediately execute the new command. Instead, it determines what BetterCP/M will execute during a subsequent cold boot.

Current and Saved Settings

There are two principal kinds of configuration changes.

Current settings affect the operating environment that is running now. These may include drive assignments, disk formats, and peripheral-device selections.

Saved settings are stored with the operating system so that they can be restored during a later cold boot.

BetterCP/M allows you to make temporary changes without necessarily replacing your saved configuration.

This is useful when experimenting with different disk formats or working with hardware that is only occasionally connected.

For example, you might normally use drive B with one disk format but temporarily configure it to read a disk prepared on another CP/M computer.

Once you've finished, you can restore your usual configuration without having permanently changed the system.

Platform-Dependent Settings

Not every configuration option is available on every computer.

Some settings depend on the capabilities of the BIOS, disk controller, or other hardware.

A computer with physical floppy drives may provide options for drive geometry and timing that have little meaning on a system using emulated storage.

Likewise, a peripheral device must actually be supported by the computer and its BIOS before BetterCP/M can use it.

The CONFIG utility provides access to supported configuration facilities, while the platform documentation describes any machine-specific requirements.

7.2 Using CONFIG

CONFIG is BetterCP/M's principal system configuration utility.

To start it, enter:

```
A0>CONFIG
```

CONFIG displays a menu of available configuration operations.

The configuration menu groups together the facilities used to control startup behavior, device assignments, disk drives, and disk formats.

A typical BetterCP/M 1.0 configuration menu includes:

```
[ A ] Startup command

[ B ] Cold-boot IOBYTE

[ C ] Disk drive definitions
[ D ] Disk format definitions
[ E ] Create or edit a disk format

[ S ] Save current configuration on disk (SYSGEN)
```

Select the letter corresponding to the operation you want to perform.

CONFIG does not require a full-screen terminal monitor. It can be used through the ordinary BetterCP/M console.

Examining the Current Configuration

Before making changes, it's often useful to examine the settings already in use.

Enter:

```
A0>CONFIG /SHOW
```

CONFIG displays information about the current system configuration, including the active drive settings and saved startup information.

Where applicable, it distinguishes between settings currently in use and those saved for the next cold boot.

This is particularly useful when you've made temporary changes and want to determine whether they will survive a restart.

The `/SHOW` option is informational. It does not modify the configuration.

Making Changes

Some CONFIG operations change the running system immediately. Others prepare changes that must be saved before they become permanent.

For example, changing a drive assignment can affect how BetterCP/M accesses that drive during the current session.

Changing the saved startup command, on the other hand, affects what happens during a later cold boot.

CONFIG identifies when a change requires saving.

Before making significant changes to your system configuration, it's good practice to keep a working backup of the system disk.

An incorrect disk configuration can make a drive inaccessible until its settings are corrected. Changes to the saved boot configuration can also affect the system's ability to start normally.

7.3 Startup Commands

BetterCP/M can execute a command automatically during a cold boot.

This provides a convenient way to establish your preferred working environment whenever the system starts.

The standard distribution uses:

```
SUBMIT STARTUP
```

as its saved startup command.

This causes BetterCP/M to execute the commands contained in `STARTUP.SUB`.

Supplied STARTUP.SUB Files

The default scripts are installed as `A0:STARTUP.SUB`. Each table cell lists the commands in execution order, one per source line:

z80pack / cpmsim	TRS-80 Model 4 / trs80gp

`:VER /BANNER`	`RSX LOAD M4SCRN`
	`RSX LOAD ADM3A`
	`CLS`
	`:VER /BANNER`

The Model 4 loads its screen backend before the ADM-3A terminal personality, clears the screen through resident CLS, and then displays the banner. cpmsim displays only the banner and does not select a terminal personality automatically. Neither script loads a clock provider.

Editing STARTUP.SUB

Because `STARTUP.SUB` is an ordinary SUBMIT file, you can edit it using a text editor. Test new commands individually before adding them. On Model 4, the backend and ADM3A are already loaded by the supplied script; do not append another copy of those commands just to obtain the default display environment.

Chapter 8 explains resident extension management, while Chapter 9 describes terminal support.

Changing the Startup Command

You can change the saved startup command using CONFIG.

For example:

```
A0>CONFIG /STARTUP="SUBMIT STARTUP"
```

sets the startup command to `SUBMIT STARTUP`.

To disable automatic startup commands:

```
A0>CONFIG /STARTUP=""
```

CONFIG requests confirmation before saving the change.

The new command takes effect during a subsequent cold boot. It is not executed immediately.

Restoring the Default Startup Command

CONFIG also provides a convenient way to prepare the standard startup command:

```
A0>CONFIG /DEFAULT
```

This prepares `SUBMIT STARTUP` as the startup command and opens the relevant configuration menu.

The change is not automatically saved. You can inspect or modify it before committing it to the system disk.

Cold Boots and Warm Boots

The startup command is executed during a cold boot.

It is not repeated during an ordinary warm boot.

This distinction is especially useful when loading resident extensions. An extension loaded during cold startup can remain available throughout the session, even after transient programs terminate.

There is ordinarily no need to reload the extension after each warm boot.

For a complete description of startup-command options, consult the CONFIG reference in Part II.

7.4 Drive Configuration

BetterCP/M can work with a variety of disk formats, including formats used by other CP/M systems.

Because different computers use different disk drives and controllers, BetterCP/M separates the characteristics of a physical drive from the format of the disk being used.

CONFIG provides facilities for examining and changing both.

Physical Drives and Logical Drives

A **physical drive** is the actual disk drive or storage device connected to the computer.

A **logical drive** is the drive letter, such as `A:` or `B:`, through which BetterCP/M accesses that device.

The distinction allows BetterCP/M to associate a logical drive with a particular physical device and disk format.

For example, you might configure logical drive B to use a physical drive containing a disk written in a format different from your usual system disks.

The available assignments depend on the computer and its BIOS.

Examining Drive Settings

To examine the current configuration, enter:

```
A0>CONFIG /SHOW
```

CONFIG displays the active drive settings and, where available, the corresponding saved defaults.

This is a useful first step before making changes.

You can also examine drive definitions through CONFIG's interactive menus.

Selecting a Disk Format

CONFIG provides a catalog of disk-format definitions.

When assigning a format to a drive, you can select an appropriate definition from this catalog rather than entering all the disk parameters individually.

For example:

```
A0>CONFIG /LOGICAL=C /PHYSICAL=1 /FORMAT="MM 80T DS DATA"
```

This assigns logical drive C to physical drive 1 using the named disk format.

The format name must match a definition available in the system's format catalog. The example is illustrative; the correct format depends on the disk and drive you're using.

CONFIG checks whether the requested format is compatible with the selected physical drive before applying the assignment.

You can also select formats through the interactive configuration menus.

Temporarily Changing a Drive

One of the advantages of BetterCP/M's configuration facilities is that drive assignments can be changed without necessarily changing the saved startup configuration.

Suppose you need to read a disk from another CP/M computer.

You can select an appropriate format, use the disk, and then restore your usual drive configuration.

If you don't save the temporary assignment, it won't replace your normal cold-boot defaults.

This makes it easier to work with different disk formats without maintaining a separate operating-system disk for each configuration.

Removing a Drive Assignment

A logical drive can also be disconnected from its current assignment.

For example:

```
A0>CONFIG /LOGICAL=C /UNBIND
```

removes the active assignment for logical drive C.

This does not erase or format the disk. It simply removes the logical drive's association with the physical device.

BetterCP/M protects drive A from this operation because of its importance to the running system.

Creating and Editing Disk Formats

CONFIG also provides facilities for creating and modifying disk-format definitions.

These are useful when working with disks whose characteristics are not already represented in the format catalog.

However, they require an accurate understanding of the disk's organization.

An incorrect format definition can cause BetterCP/M to interpret the disk incorrectly, potentially making files inaccessible or placing existing data at risk if the disk is written.

For ordinary operation, use an existing format definition whenever possible.

The CONFIG command reference describes the available format-editing facilities. More detailed information about disk organization and format definitions is provided in the technical documentation.

7.5 Console and Device Configuration

Like conventional CP/M, BetterCP/M provides logical device assignments for console, reader, punch, and list operations.

These assignments allow programs to use standard CP/M device interfaces without necessarily knowing which physical device provides them.

BetterCP/M retains the traditional **IOBYTE** mechanism for selecting these assignments.

The IOBYTE

The IOBYTE is a system value used to select logical device assignments.

For example, it can determine which supported device is used for list output or other conventional CP/M character-device operations.

The available choices depend on the computer and its BIOS.

An IOBYTE assignment does not create a device or provide a driver for hardware that the system doesn't support. It merely selects among the device facilities available through the BIOS.

Changing Device Assignments

BetterCP/M distinguishes between the IOBYTE currently in use and the value saved for the next disk cold boot.

The STAT utility provides facilities for changing the live IOBYTE assignment.

CONFIG allows you to save a separate assignment that will be established during a subsequent cold boot.

For example:

```
A0>CONFIG /IOBYTE=85
```

requests that decimal value 85 be saved as the cold-boot IOBYTE.

CONFIG asks for confirmation before saving the change.

The command does not change the current IOBYTE. To change the active assignment, use STAT.

You can examine the current and saved values with:

```
A0>CONFIG /SHOW
```

Choosing Device Assignments

Before changing the IOBYTE, make sure the requested assignment is supported by your computer.

Some traditional CP/M device names refer to hardware that may not exist on a particular system.

For example, selecting a printer interface does not make a printer available unless the BIOS provides the corresponding output facility.

BetterCP/M also requires the console assignment to remain compatible with its supported console environment.

For the available device assignments and their meanings, consult the STAT and CONFIG command references.

Platform-specific documentation describes any additional requirements or limitations.

7.6 Saving Configuration Changes

Many BetterCP/M configuration changes can be made temporarily.

To retain them after a cold boot, you must save the appropriate settings to the system disk.

CONFIG provides facilities for doing this, using BetterCP/M's system-generation services.

Temporary and Permanent Changes

Suppose you temporarily change a drive assignment to read a disk from another computer.

You may want to use that configuration for the remainder of the current session without making it your permanent default.

In that case, you can leave the saved configuration unchanged.

On the other hand, if you've installed a new disk drive or established a configuration you intend to use regularly, you'll probably want to save it.

BetterCP/M allows you to distinguish between these situations.

Saving from CONFIG

CONFIG's main menu includes:

```
[ S ] Save current configuration on disk (SYSGEN)
```

This option provides access to the system-configuration saving facilities.

CONFIG distinguishes between saving pending cold-boot changes and saving the current configuration as the new startup defaults.

These operations are not necessarily equivalent.

For example, you might have changed the startup command while also temporarily reassigning a disk drive.

Saving only the pending cold-boot changes allows you to retain the new startup command without making the temporary drive assignment permanent.

Saving the current configuration as the new defaults is appropriate when you want those active settings restored during future cold boots.

CONFIG presents the available choices and requests confirmation before writing the saved configuration.

Verifying the Configuration

After saving, it's good practice to examine the configuration again:

```
A0>CONFIG /SHOW
```

Check that the saved settings correspond to what you intended.

If you've changed startup settings, remember that they won't take effect until the next cold boot.

If you've changed drive assignments, verify that the appropriate drives and formats remain available.

Protecting Your System Disk

Saving configuration changes modifies information associated with the operating system on disk.

For this reason, you should keep a backup of a known working system configuration.

This is particularly important when changing disk-drive assignments or other settings needed to boot the computer.

If a saved configuration prevents the system from starting correctly, a backup system disk may be needed to restore a working environment.

CONFIG performs checks when saving settings, but no software can protect against every possible hardware failure or interruption.

Avoid resetting or switching off the computer while configuration information is being written.

Returning to a Known Configuration

If you're experimenting with unfamiliar disk formats or device assignments, consider retaining a separate system disk with your normal configuration.

This gives you a convenient way to return to a known working system if an experiment produces unexpected results.

For ordinary use, however, BetterCP/M's configuration facilities should allow you to make most supported changes without rebuilding the operating system manually.

Chapter 8 - Resident Extensions

One of BetterCP/M's most useful features is its ability to add commands and system services without rebuilding the operating system.

These additions are called **resident extensions**. Unlike ordinary transient programs, which are loaded when needed and normally disappear when they finish, resident extensions remain in memory and continue to provide their facilities while you work.

BetterCP/M supports two principal kinds of extensions: Command Processor Extensions (CPXs) and Resident System Extensions (RSXs).

This chapter explains how to use and manage them. You don't need to understand their internal workings to take advantage of the facilities they provide.

For complete command syntax and options, consult the CPX and RSX entries in Part II, *Command and Utility Reference*.

8.1 Introducing CPX and RSX Extensions

In conventional CP/M, the command processor provides a small collection of built-in commands. Most other commands are transient programs loaded from disk.

BetterCP/M extends this arrangement by allowing additional commands and services to remain resident.

For example, a resident extension might provide frequently used file-management commands, add a new operating-system service, or support a particular terminal.

Resident extensions offer several advantages. A resident command is immediately available without loading a separate program from disk each time it is used. A resident service can remain available while different applications execute. Extensions can also be loaded and unloaded to suit the work you're doing.

RSXs occupy protected memory that would otherwise be available to transient programs. CPX command code is reclaimable when a transient program starts.

BetterCP/M allows you to choose which optional extensions to keep loaded, making it possible to balance additional facilities against available program memory.

Command Processor Extensions

A **Command Processor Extension**, or **CPX**, adds commands to BetterCP/M's command processor.

The standard BetterCP/M environment includes a resident command package named `RCP.CPX`.

This package provides familiar commands such as:

```
DIR   ERA   TYPE   REN   USER
CLS   VER   COPY   MOVE
```

These commands can be executed without loading separate `.COM` programs.

For example:

```
A0>DIR
```

normally invokes the resident DIR command when the standard resident command package is installed.

Some commands are available in both resident and transient forms. The resident version generally provides commonly used operations, while the transient version may offer additional options.

BetterCP/M allows you to request a transient program explicitly by prefixing its name with a colon:

```
A0>:DIR /WHEREIS ED.COM
```

This bypasses the resident DIR command and executes `DIR.COM`.

A period can also be used as the prefix:

```
A0>.DIR /WHEREIS ED.COM
```

Both forms have the same meaning.

BetterCP/M 1.0 does not automatically switch to the transient version when you request an option supported only by that version. Instead, the resident command may tell you to invoke the transient utility explicitly.

For example:

```
A0>DIR /WHEREIS ED.COM
DIR: Use :DIR.
```

You can then enter:

```
A0>:DIR /WHEREIS ED.COM
```

For more information about resident and transient command execution, see Chapter 5.

Resident System Extensions

A **Resident System Extension**, or **RSX**, provides services that remain available while other programs execute.

Unlike CPXs, which add commands to the command processor, RSXs extend the operating environment itself.

Terminal support is one of the most useful examples.

A program written for a particular terminal may expect certain control sequences to move the cursor, clear the screen, or change the display.

BetterCP/M can provide these facilities through resident terminal-monitor extensions.

Other RSXs provide facilities such as support for batch input or disk-format services.

CPXs are available in the command environment and are reconstructed after transient execution. RSXs remain in protected memory while applications run. They serve different purposes and are managed separately.

8.2 Installing Extensions

Resident extensions are supplied as files that BetterCP/M can load when their facilities are needed.

CPX files use the `.CPX` file type, while RSX files use `.RSX`.

For example:

```
RCP.CPX
ADM3A.RSX
VT52.RSX
ANSI.RSX
```

Place normal `.CPX` and `.RSX` extension files on **drive A, user 0 (A0)** before loading them. The carrier loader uses A0 even when you run `CPX.COM` or `RSX.COM` from another drive/user. It does not use the transient-program current-location/A0 search sequence.

Keep `R3PACK.RSX` in A0 as well. It is the RSX manager's required runtime container and must match the system build. `RSX.COM` alone is not sufficient. Keep the supplied container with the system files; it is an internal dependency, **not** a module to pass to `RSX LOAD`.

Configured CPX files must remain in A0 after their commands become available. BetterCP/M rereads them when reconstructing the command environment after transient execution or warm boot. A missing carrier can leave a configured name listed without its commands being available.

Extensions must also be compatible with the running operating system. An extension prepared for a different version or system configuration may not load successfully.

Some extensions depend on other resident components.

For example, BetterCP/M's terminal personalities require a suitable screen backend. The backend provides the connection to the computer's display, while the terminal personality interprets the control sequences expected by applications.

These supporting components must be loaded before the extensions that depend on them.

The documentation supplied with an extension should identify any such requirements.

Installing an Extension for Regular Use

Install compatible carriers in A0 and test their load commands at the prompt before adding them to `STARTUP.SUB`. Keep dependencies in order: screen backend first, terminal personality second.

See Section 7.3 for the supplied platform scripts and startup editing, and Section 9.1 for terminal-loading procedures. Model 4 startup already loads M4SCRN and ADM3A; customize that sequence rather than adding duplicate loads.

8.3 Listing Installed Extensions

Before changing the resident environment, it's useful to see which extensions are currently installed.

BetterCP/M provides separate commands for examining CPXs and RSXs.

Listing CPXs

Enter:

```
A0>CPX LIST
```

BetterCP/M displays information about the configured command processor extensions.

The list can help you identify which resident command packages are available and determine whether an optional extension is present.

An extension's appearance in the configured list does not necessarily mean it loaded successfully. If an extension could not be loaded, BetterCP/M may retain its configured entry while reporting a warning.

This distinction is useful when troubleshooting missing resident commands.

Listing RSXs

To examine the installed resident system extensions, enter:

```
A0>RSX LIST
```

BetterCP/M displays the currently installed RSXs.

This is useful when checking which services are available or determining whether an extension is still occupying memory.

The two lists serve different purposes. `CPX LIST` describes the resident command environment, while `RSX LIST` describes the installed resident system services.

8.4 Loading and Unloading Extensions

BetterCP/M allows optional extensions to be added to or removed from the active operating environment.

This makes it possible to customize the system for different tasks without permanently incorporating every available facility.

Loading a CPX

To load a command processor extension, use the CPX management command.

For example, an extension named `TOOLS.CPX` would be requested with:

```
A0>CPX LOAD TOOLS
```

The extension must be available as `A0:TOOLS.CPX` and compatible with the running system. Keep it there for command-environment reconstruction; see Section 8.2.

Once successfully loaded, its commands become available through the normal command environment.

You can enter those commands just as you would any other BetterCP/M command.

The CPX reconstruction profile has four slots, including the standard RCP entry. New names are appended and their commands are tried in profile order after built-in CCP commands. The first package that handles a command wins. See Section 17.3 for capacity, ordering and duplicate/absent-operation details.

Unloading a CPX

To remove an optional CPX, use:

```
A0>CPX UNLOAD TOOLS
```

After the extension has been removed, its commands are no longer available as resident commands.

If corresponding transient programs are present, they may still be executed normally.

Unloading an optional CPX is useful when you no longer need its commands or want to change the resident command environment.

Loading an RSX

To load an RSX, use the RSX command.

For example:

```
A0>RSX LOAD ADM3A
```

requests the ADM-3A terminal personality.

Some RSXs depend on other extensions being loaded first.

Terminal personalities, for example, require a suitable screen backend. The backend provides the connection to the computer's display, while the terminal personality interprets the control sequences expected by applications.

The required loading order is described in Chapter 9.

Unloading an RSX

To remove an RSX, enter:

```
A0>RSX UNLOAD ADM3A
```

BetterCP/M removes the extension when it can safely do so.

An extension that is still needed by another resident component cannot necessarily be unloaded immediately.

For example, a screen backend should not be removed while a terminal personality that depends on it remains active.

In such cases, unload the dependent extension first.

The RSX manager checks these relationships and reports operations that cannot be performed safely.

Required Extensions

RCP supplies the standard resident commands, but the current CPX manager permits its removal. Doing so removes those commands; only built-in CCP commands and separately available transients remain. Section 17.3 explains restoring RCP and its place in the four-entry profile. RSX removals are separate operations and can be refused when dependencies make removal unsafe.

8.5 Extension Persistence

One of the distinguishing characteristics of BetterCP/M's resident extension system is that supported extensions remain available across ordinary warm boots.

In conventional CP/M, a warm boot returns control to the command environment after a program terminates.

BetterCP/M retains this behavior while preserving the resident extensions that are intended to survive.

For example, suppose you've loaded a terminal monitor and then started a full-screen editor.

When you leave the editor and return to the command prompt, the terminal monitor remains installed.

You can start another compatible application without loading the monitor again.

Command processor extensions are reconstructed with the command environment after transient programs finish through a warm boot. Their command code can be overwritten by the transient program.

Cold Boots and Startup Configuration

A cold boot establishes a new session. Extensions needed in every session must be loaded by the startup procedure. Section 7.3 contains the platform defaults and editing instructions; use Section 8.2 for carrier placement and dependencies.

8.6 Memory Considerations

RSXs remain in protected memory until unloaded or the operating environment is reinitialized. CPX command code occupies reclaimable command-environment memory and is reconstructed after transient execution.

Because CP/M systems have limited memory, the number and size of loaded extensions can matter.

Available Program Memory

Ordinary CP/M programs execute in the Transient Program Area, or TPA.

Loaded RSXs reduce the memory available to those programs. The CCP and CPX command code can be reclaimed for transient execution.

Use `STAT MEM` to examine the current memory map and maximum loadable COM size. That maximum can include memory currently occupied by the reclaimable CCP and CPXs; it excludes protected RSX memory.

For example, you might load a terminal monitor to run a full-screen editor. While the monitor is active, less memory is available to the editor.

If you later want to run a particularly large program that doesn't require terminal emulation, you can unload the monitor and recover the memory it occupied.

This flexibility is one of the advantages of BetterCP/M's extension system.

Choosing Which Extensions to Load

There's no particular advantage to loading every available extension simply because it exists.

Instead, consider which facilities you actually use.

Frequently used commands may be worth keeping resident.

A terminal monitor may be essential when running a particular application but unnecessary when using ordinary command-line utilities.

Specialized extensions can be loaded when needed and removed afterward.

Extension Dependencies

Some extensions depend on other resident components.

For example, a terminal personality may require a particular screen backend.

These dependencies affect the order in which extensions must be loaded and unloaded.

In general, load the supporting component first, followed by the extension that uses it.

When removing them, reverse the order.

Chapter 9 provides examples involving terminal monitors.

Recovering from Extension Problems

Resident extensions are normally straightforward to manage, but occasionally an extension may be missing, damaged, incompatible, or incorrectly configured.

BetterCP/M provides safeguards intended to prevent an unusable optional extension from making the entire command environment unavailable.

If an optional CPX cannot be loaded or fails validation, BetterCP/M may display:

```
CPX?
```

The system continues to the command prompt with the remaining usable extensions.

Commands supplied by the rejected CPX will not be available.

The extension may nevertheless remain in the configured list, so its appearance in `CPX LIST` does not necessarily indicate that it loaded successfully.

To remove an unusable optional extension, enter:

```
A0>CPX UNLOAD BAD
```

where `BAD` is the name of the affected extension.

Alternatively, replace the damaged extension file with a valid copy.

Until the problem is corrected, the warning may recur when BetterCP/M restores the command environment.

If BetterCP/M refuses to unload an RSX, another extension may still depend on it. Unload the dependent extension first, then remove the supporting component.

If a transient program reports insufficient memory, use `STAT MEM` and `RSX LIST` to examine memory use. Unloading unnecessary RSXs can restore memory to the TPA.

For additional troubleshooting information, see Chapter 12.

Chapter 9 - Terminal Support

BetterCP/M provides terminal-monitor facilities that allow programs written for different terminal types to operate on supported computers.

If you've used CP/M for any length of time, you've probably encountered programs that expect a particular terminal. A full-screen editor, for example, may need to know how to move the cursor, clear part of the screen, or display text in reverse video.

Different terminals use different control sequences to perform these operations. A program configured for one terminal may not display correctly on another.

BetterCP/M addresses this problem through **loadable terminal monitors**. These provide familiar terminal environments without requiring every application to understand the particular computer's display hardware.

BetterCP/M 1.0 includes support for Montezuma Micro-style ADM-3A, VT52, and a supported subset of ANSI/VT100 terminal operation.

This chapter explains how to select, load, and use these facilities. For complete RSX command syntax, consult Part II, *Command and Utility Reference*.

9.1 The BetterCP/M Terminal Monitor

In conventional CP/M, the BIOS provides the basic console input and output services used by application programs.

These services are sufficient for ordinary character input and output, but full-screen applications often require additional capabilities.

A text editor, for example, may need to move the cursor to a particular position, clear the remainder of a line, or scroll part of the screen.

Historically, CP/M applications often supported several terminal types. The user selected the appropriate terminal during installation or configuration.

BetterCP/M retains this approach while providing additional terminal support through resident extensions.

Terminal Personalities

A **terminal personality** interprets the control sequences associated with a particular terminal.

For example, an ADM-3A personality recognizes the screen-control conventions used by compatible ADM-3A applications.

A VT52 personality recognizes the supported control sequences used by programs configured for a VT52 terminal.

An ANSI personality provides a supported subset of ANSI/VT100 terminal behavior.

Only one terminal personality can be active at a time.

Screen Backends

A terminal personality does not communicate directly with every possible display device.

Instead, it uses a **screen backend** that provides the connection to the computer's actual display environment.

For example, BetterCP/M provides a screen backend for the TRS-80 Model 4 and another for systems using a compatible ANSI host terminal.

This arrangement allows the same terminal personality to be used with different display systems.

From the user's perspective, the important distinction is straightforward:

- The terminal personality determines which terminal control sequences an application can use.
- The screen backend determines how those operations appear on the computer's display.

Both must be suitable for the system you're using.

Loading a Terminal Monitor

Terminal monitors are loaded using BetterCP/M's RSX command.

The usual procedure is to load the appropriate screen backend first, followed by the desired terminal personality.

For example:

```
RSX LOAD M4SCRN
RSX LOAD ADM3A
CLS
```

This loads the Model 4 screen backend, activates the ADM-3A personality, and clears the screen.

On a system using the ANSI screen backend, the corresponding commands would be:

```
RSX LOAD ANSISCRN
RSX LOAD ADM3A
CLS
```

The correct backend depends on the computer or emulator.

The distribution manifest identifies the platform screen backends: M4SCRN on Model 4 and ANSISCRN on cpmsim. See Chapter 9's procedures for loading the backend and personality.

9.2 Selecting a Terminal Personality

Before running a full-screen application, determine which terminal types it supports.

Many CP/M programs provide an installation or configuration utility that allows you to select a terminal.

Others have terminal-specific versions or require a configuration file.

For example, an editor might offer several choices:

```
ADM-3A
VT52
VT100
```

Choose the terminal personality that matches the BetterCP/M monitor you intend to use.

If the program is configured for VT52, load BetterCP/M's VT52 personality before starting it.

If the program expects ADM-3A control sequences, use the appropriate ADM-3A personality.

The terminal monitor does not automatically detect which terminal type an application expects.

Changing Personalities

Only one terminal personality can be active at a time.

To change from one personality to another, unload the current personality before loading the replacement.

For example, to change from ADM-3A to VT52:

```
RSX UNLOAD ADM3A
RSX LOAD VT52
CLS
```

If the appropriate screen backend is already loaded, you can continue using it.

There is no need to unload and reload the backend merely to change terminal personalities, provided the backend supports the new personality.

Returning to the Basic Console

If you no longer need terminal emulation, unload the active personality.

For example:

```
RSX UNLOAD VT52
```

You can then unload the screen backend if no other extension requires it:

```
RSX UNLOAD M4SCRN
```

Removing the monitor releases the memory it occupied, making more memory available to ordinary programs.

This can be useful when running applications that do not require full-screen terminal support.

9.3 ADM-3A Support

BetterCP/M provides an ADM-3A terminal personality intended particularly for software using the Montezuma Micro-style ADM-3A conventions found on historical TRS-80 Model 4 CP/M systems.

The extension is supplied as:

```
ADM3A.RSX
```

It can be used with either the Model 4 screen backend or the ANSI screen backend, depending on the platform.

Using ADM-3A on the Model 4

On a TRS-80 Model 4 system, the screen backend is:

```
M4SCRN.RSX
```

To load the ADM-3A environment:

```
A0>RSX LOAD M4SCRN
A0>RSX LOAD ADM3A
A0>CLS
```

The Model 4 distribution normally loads the appropriate backend and ADM-3A personality during startup.

If they are already installed, there is no need to load them again.

Using ADM-3A with an ANSI Host Terminal

On a system using the ANSI screen backend, load:

```
A0>RSX LOAD ANSISCRN
A0>RSX LOAD ADM3A
A0>CLS
```

The ANSI backend translates the supported display operations into sequences understood by the host terminal.

For best results, use a compatible terminal emulator.

Testing ADM-3A Support

BetterCP/M supplies a test utility named:

```
ADMTST.COM
```

After loading the appropriate terminal monitor, enter:

```
A0>ADMTST
```

The utility displays a series of test screens and prompts designed to exercise the supported ADM-3A facilities.

These include cursor movement, reverse video, line operations, and other display functions.

The test also requests several keyboard characters to help confirm that the terminal environment is receiving the expected input.

Compatibility Considerations

The ADM-3A personality is intended to support the conventions used by compatible CP/M applications.

It does not make all programs written for a particular computer automatically portable.

A program that communicates directly with display hardware rather than using the supported console interface may still require its original hardware environment.

For ordinary terminal-oriented applications, however, the monitor provides a convenient way to reproduce the expected display behavior.

9.4 VT52 Support

The DEC VT52 was a widely supported video terminal, and many CP/M applications can be configured to use its control sequences.

BetterCP/M provides a loadable VT52 personality named:

```
VT52.RSX
```

The same VT52 extension can be used on different supported platforms by loading the appropriate screen backend.

Loading the VT52 Monitor

On a TRS-80 Model 4:

```
A0>RSX LOAD M4SCRN
A0>RSX LOAD VT52
A0>CLS
```

On a system using the ANSI screen backend:

```
A0>RSX LOAD ANSISCRN
A0>RSX LOAD VT52
A0>CLS
```

These examples assume that no other terminal personality is currently active.

If another personality is loaded, unload it first.

For example, if ADM-3A is active:

```
A0>RSX UNLOAD ADM3A
A0>RSX LOAD VT52
A0>CLS
```

You can retain the existing screen backend when changing personalities, provided it supports VT52.

Configuring Applications for VT52

When installing or configuring a full-screen application, select VT52 as its terminal type.

The application will then send VT52 control sequences for operations such as moving the cursor or clearing portions of the display.

BetterCP/M's VT52 personality interprets the supported sequences and passes the corresponding display operations to the screen backend.

For systems using an ANSI host terminal, the host terminal should provide a suitable display area. An 80-column, 24-row configuration is appropriate for the supplied VT52 support.

Testing VT52 Support

BetterCP/M supplies a test utility named:

```
VT52TST.COM
```

After loading the VT52 personality, enter:

```
A0>VT52TST
```

The utility exercises supported VT52 operations, including cursor positioning, tab stops, display erasure, and terminal identification.

It pauses during the test so you can examine the results.

If the display behaves as expected, the monitor is ready for compatible applications.

VT52 Compatibility

BetterCP/M's VT52 personality supports the terminal facilities needed by compatible applications, but it is not intended to reproduce every possible hardware characteristic of a physical VT52.

Graphics facilities, specialized keypad modes, and host keyboard translation are outside the supported subset.

In particular, the monitor does not guarantee that the host computer's arrow keys will generate the control characters expected by every application.

These details depend on the computer, terminal emulator, and application configuration.

For ordinary full-screen programs using supported VT52 control sequences, the monitor provides a convenient portable terminal environment.

9.5 ANSI/VT100 Support

BetterCP/M also provides an ANSI terminal personality supporting a subset of the control sequences commonly associated with ANSI and DEC VT100 terminals.

This is particularly useful for programs that expect cursor positioning, screen erasure, character attributes, and other ANSI-style display operations.

The extension is supplied as:

```
ANSI.RSX
```

As with the ADM-3A and VT52 personalities, ANSI support requires a suitable screen backend.

Loading the ANSI Monitor

On a TRS-80 Model 4:

```
A0>RSX LOAD M4SCRN
A0>RSX LOAD ANSI
A0>CLS
```

On a system using the ANSI screen backend:

```
A0>RSX LOAD ANSISCRN
A0>RSX LOAD ANSI
A0>CLS
```

If another terminal personality is active, unload it before loading ANSI.

For example, to change from ADM-3A to ANSI while retaining the Model 4 screen backend:

```
A0>RSX UNLOAD ADM3A
A0>RSX LOAD ANSI
A0>CLS
```

Supported Display Facilities

The ANSI personality provides a useful selection of common terminal operations.

These include cursor movement, screen and line erasure, character attributes, scrolling regions, cursor save and restore, and cursor visibility.

The supported character attributes include facilities such as reverse video.

The actual appearance of an attribute depends on the display hardware.

For example, a monochrome display cannot reproduce the full range of colors available on a modern ANSI terminal.

BetterCP/M provides the supported logical terminal behavior, while the screen backend determines how it appears on the physical display.

Testing ANSI Support

BetterCP/M supplies:

```
ANSITST.COM
```

After loading the ANSI personality, enter:

```
A0>ANSITST
```

The test utility demonstrates several supported ANSI display operations.

These include cursor positioning, reverse video, erasure, and related screen-control behavior.

The test pauses so you can inspect the display.

ANSI Is Not Complete VT100 Emulation

Although the ANSI personality supports many familiar ANSI/VT100 operations, it does not provide complete VT100 emulation.

Some specialized facilities are not supported, including terminal-status queries, character-set selection, keypad modes, and switching into VT52 mode.

Programs requiring these facilities may not operate correctly.

For this reason, selecting VT100 in an application's installation program does not automatically guarantee compatibility.

Where possible, consult the application's terminal requirements and use the supplied test utilities to verify the display environment.

Display Dimensions

Some full-screen programs assume a particular number of rows and columns.

For example, a program may expect an 80-column display with 24 or 25 rows.

BetterCP/M's terminal monitor cannot always compensate for an application configured for the wrong screen dimensions.

When configuring a program, select the screen size appropriate to the computer or terminal emulator you're using.

This is particularly important when moving applications between the Model 4's display and a modern host terminal.

9.6 Configuring Applications for Your Terminal

The terminal monitor provides the display environment, but applications must still be configured to use it.

Many CP/M applications include a terminal-installation program or configuration procedure.

Others provide several executable versions, each intended for a particular terminal.

Before running a full-screen application, determine which terminal types it supports.

Choosing a Terminal Type

If an application supports ADM-3A, VT52, or ANSI/VT100, select the type corresponding to the BetterCP/M personality you've loaded.

For example, if you intend to use BetterCP/M's VT52 monitor, configure the application for VT52.

If the application is already configured for ADM-3A, load the ADM-3A personality instead.

The terminal personality and application configuration must agree.

A program configured for VT52 will not necessarily display correctly when the ADM-3A personality is active.

A Typical Setup Procedure

Suppose you've obtained a full-screen editor that supports VT52.

A typical procedure would be:

1. Load the appropriate screen backend.
2. Load the VT52 personality.
3. Clear the screen.
4. Run `VT52TST` to check the terminal environment.
5. Configure the editor for VT52 operation.
6. Start the editor.

For example, on a system using the ANSI screen backend:

```
A0>RSX LOAD ANSISCRN
A0>RSX LOAD VT52
A0>CLS
A0>VT52TST
A0>EDITOR
```

Here, `EDITOR` represents the application you've installed.

The actual configuration procedure depends on the application.

When the Display Looks Wrong

If a full-screen program produces misplaced text, unexpected characters, or incorrect cursor movement, the first thing to check is the terminal configuration.

Make sure the program is configured for the personality currently loaded.

You can examine the installed RSXs with:

```
A0>RSX LIST
```

If the wrong personality is active, unload it and load the correct one.

If the correct personality is loaded but the display is still incorrect, run the corresponding terminal test utility.

If the test works but the application does not, examine the application's terminal settings and screen-size assumptions.

Some applications use terminal capabilities that are not supported by BetterCP/M's monitor. Others communicate directly with hardware rather than using the standard console interface.

Such programs may require their original hardware environment or additional configuration.

Loading Your Preferred Terminal at Startup

Follow Section 7.3 to customize `A0:STARTUP.SUB`, using the backend and personality already tested for your application. The supplied Model 4 script selects ADM3A; cpmsim leaves the basic console active. Ordinary warm boots preserve an installed monitor, and you can change personalities during the session.

Returning to the Basic Console

When you no longer need terminal emulation, unload the active personality and then its screen backend.

For example:

```
A0>RSX UNLOAD VT52
A0>RSX UNLOAD ANSISCRN
```

The screen backend can be removed only when no remaining extension requires it.

Unloading the monitor releases its resident memory, making that space available to transient programs.

If you subsequently need terminal emulation again, simply reload the appropriate backend and personality.

Chapter 10 - Working with Disks

BetterCP/M provides utilities for preparing, copying, checking, and installing system disks. These facilities are useful whether you're maintaining an existing BetterCP/M installation, preparing additional working disks, or experimenting with different CP/M disk formats.

If you've used conventional CP/M, you'll already be familiar with the distinction between copying files and copying an entire disk. BetterCP/M retains that distinction while providing additional facilities for working with different disk formats and system configurations.

Two utilities are particularly important:

DUP handles disk formatting, copying, and checking.

SYSGEN installs BetterCP/M onto a prepared disk or saves an installed operating system as a system-image file.

The two utilities complement one another. DUP prepares and copies disks, while SYSGEN installs and preserves the operating system itself.

This chapter explains how to perform common disk-management operations. Complete command syntax and options are provided in Part II, *Command and Utility Reference*.

Because disk-management operations can destroy existing data, read the instructions carefully before formatting or copying a disk.

10.1 Disk Formats

CP/M was used on a remarkable variety of computers, and those computers did not all use the same disk formats.

Even disks of the same physical size could differ in their number of tracks, sectors, recording density, and filesystem organization.

Consequently, a disk prepared on one CP/M computer might not be readable on another without suitable configuration.

BetterCP/M provides facilities for working with different disk formats, subject to the capabilities of the computer and its disk controller.

Physical and Logical Drives

As discussed in Chapter 7, BetterCP/M distinguishes between physical and logical drives.

A **physical drive** is the actual disk drive or storage device.

A **logical drive** is the drive letter, such as `A:` or `B:`, through which BetterCP/M accesses that device.

A logical drive is associated with a physical device and a disk-format definition.

For example, logical drive B might be configured to use a physical drive containing disks prepared in a particular CP/M format.

The available assignments depend on the computer and its BIOS.

Disk-Format Definitions

BetterCP/M uses disk-format definitions to describe how disks are organized.

These definitions tell the system how to access the sectors and filesystem structures of a particular format.

Rather than requiring users to enter all the necessary parameters each time, BetterCP/M provides a catalog of disk-format definitions.

The CONFIG utility allows you to select an appropriate definition and associate it with a logical drive.

For example:

```
A0>CONFIG /LOGICAL=C /PHYSICAL=1 /FORMAT="MM 80T DS DATA"
```

This assigns logical drive C to physical drive 1 using the named disk format.

The format name must match an available definition, and the requested format must be compatible with the selected physical drive.

The example illustrates the command's use; the correct format depends on the disk and hardware you're using.

Selecting the Correct Format

Before accessing a disk, make sure its format matches the definition assigned to the logical drive.

Using an incorrect definition may cause the disk to appear unreadable or produce errors when accessing files.

More seriously, writing to a disk using an incorrect format definition can damage existing data.

If you're uncertain about a disk's format, avoid writing to it until you've identified the correct configuration.

The CONFIG utility provides facilities for examining and changing disk assignments.

For more information, see Chapter 7, *Configuring BetterCP/M*.

10.2 Formatting Disks

Formatting prepares a disk for use by establishing its physical track and sector organization.

BetterCP/M provides the **DUP** utility for this purpose.

DUP can format disks, copy entire disks, and check disks for physical read errors.

To start DUP interactively, enter:

```
A0>DUP
```

The utility displays a menu similar to:

```
A  Format a disk
B  Copy a disk
C  Check a disk for errors
```

Select the operation you want to perform.

Formatting a Disk

To format a disk, select option A from the DUP menu.

DUP asks you to identify the destination drive.

The format used is the one currently assigned to that logical drive.

If the drive isn't configured for the format you want, exit DUP and use CONFIG to select the appropriate format first.

DUP checks whether the requested operation is supported by the drive and its disk-format definition.

It then displays a warning that formatting will destroy the existing contents of the destination disk.

Formatting erases the disk's previous contents. Make absolutely certain you've selected the correct drive before confirming.

DUP requires explicit confirmation before proceeding.

Only an affirmative response allows formatting to begin.

Formatting from the Command Line

DUP also provides a direct command-line form.

For example:

```
A0>DUP /FORMAT B:
```

requests formatting of drive B using its currently assigned disk format.

The command still requires confirmation. The command-line form does not bypass the normal safeguards.

Format Verification

DUP verifies its formatting operations by reading back the formatted tracks.

This helps identify disks or drives that cannot reliably perform the requested operation.

When formatting completes successfully, DUP reports:

```
Format complete.
```

If an error occurs, DUP reports the problem.

An interrupted or failed format should not be treated as a successfully prepared disk.

Protected System Drive

BetterCP/M protects the physical system drive from destructive DUP operations.

In the current implementation, physical drive 0 cannot be used as the destination for formatting or disk copying.

This protection follows the physical drive, even if logical drive letters have been reassigned.

Other drives may be used as destinations when their configurations support the requested operation.

A Note About Available Formats

Not every disk format listed in BetterCP/M's format catalog can necessarily be created by every supported computer.

Some formats may be readable but cannot be written or formatted by a particular disk controller or emulator.

DUP reports unsupported operations rather than silently substituting another format.

Consult the platform documentation for information about the formats supported by your hardware.

10.3 Copying Disks

Copying an entire disk is different from copying individual files.

When you use the COPY command, BetterCP/M copies the selected files from one location to another.

When you use DUP's disk-copying facility, BetterCP/M copies the disk's contents, including its system areas, directories, and file data.

This makes DUP particularly useful for preparing duplicate working disks or making backups of complete disks.

Starting a Disk Copy

Start DUP:

```
A0>DUP
```

Select option B, *Copy a disk*.

DUP asks you to select a source drive and a destination drive.

Alternatively, you can start the operation directly:

```
A0>DUP /COPY A: B:
```

This requests a copy from drive A to drive B.

The source disk is read, while the destination disk is written.

All existing contents of the destination disk will be replaced.

Make sure you've selected the correct drives before confirming.

Using Two Drives

BetterCP/M 1.0 requires two different physical drives for DUP's disk-copying operation.

It does not support copying a disk by repeatedly swapping source and destination media in a single drive.

The destination must also satisfy the system's drive restrictions. In particular, physical drive 0 is protected against destructive disk-copy operations.

Copying Between Different Formats

DUP uses the disk-format definitions currently assigned to the source and destination drives.

If the destination's format differs from the source, DUP can prepare the destination to match the source format, provided the operation is supported by the hardware.

DUP informs you when the destination will be reformatted.

This is an important warning: **the destination's previous contents will be destroyed, even if its original format was different from the source.**

After checking the displayed source and destination drives, confirm the operation to proceed.

Copy Verification

DUP verifies copied data by reading it back from the destination and comparing it with the source data.

When the operation completes successfully, DUP reports:

```
Copy complete; destination verified.
```

This indicates that the data copied by DUP has been verified.

If a read, write, or verification error occurs, DUP reports the problem.

An interrupted or failed copy may leave the destination only partly written. Do not rely on such a disk as a complete backup.

After Copying

If DUP temporarily changed the destination's disk-format assignment during copying, it restores the previous assignment when the operation finishes.

This does not restore the disk's former physical format.

The disk now contains a copy of the source, so you may need to use CONFIG to assign the copied format before accessing it normally.

If you intend to use that format regularly, you can save the new configuration as a startup default.

Chapter 7 explains drive configuration in more detail.

10.4 Checking Disks

DUP can also check a disk for physical read errors.

This is useful when examining an unfamiliar disk, investigating suspected media problems, or checking whether a disk can be read reliably.

Starting a Disk Check

Start DUP:

```
A0>DUP
```

Select option C, *Check a disk for errors*.

DUP asks you to select the drive to examine.

Alternatively, enter:

```
A0>DUP /CHECK B:
```

This checks the disk in drive B.

What DUP Checks

DUP reads the sectors described by the disk-format definition assigned to the selected drive.

It reports sectors that cannot be read successfully.

The operation does not modify the disk's contents.

Unlike formatting and copying, checking a disk does not require confirmation before proceeding.

Understanding the Results

As DUP checks the disk, it reports read errors and their locations.

When the check completes, it displays a summary similar to:

```
Check complete. Unreadable sectors: 0
```

A count of zero means that DUP was able to read the sectors examined during that operation.

A nonzero count indicates that one or more sectors could not be read successfully.

What a Disk Check Does Not Do

DUP's disk check is a test of physical readability.

It does not repair damaged sectors, reconstruct lost files, or examine the logical consistency of the CP/M filesystem.

It also does not compare the disk against another disk to determine whether their contents match.

A disk can therefore pass DUP's readability check while still containing damaged or incorrect files.

If you suspect that important data has been corrupted, compare it with a known good copy or restore it from a backup.

Interrupting a Check

You can interrupt a disk check with `Ctrl-C`.

Because the operation only reads the disk, interrupting it does not modify the disk's contents.

However, an interrupted check is incomplete and should not be treated as evidence that the entire disk is readable.

10.5 Installing the Operating System

Formatting a disk prepares its physical organization, but it does not necessarily make the disk bootable.

A CP/M system disk must also contain the operating system in the locations expected by the computer's bootstrap mechanism.

BetterCP/M provides **SYSGEN** for installing the operating system onto a suitably prepared disk.

What SYSGEN Does

SYSGEN writes the BetterCP/M operating-system image into the disk's reserved system area.

It does not format the disk.

It also does not copy ordinary `.COM` programs, utilities, or other files into the filesystem.

Consequently, preparing a complete system disk may involve several operations:

1. Configure the appropriate disk format.
2. Format the disk if necessary.
3. Install BetterCP/M using SYSGEN.
4. Copy the required utilities and supporting files onto the disk.

The exact procedure depends on the disk format and computer.

Installing from Another System Disk

SYSGEN can install BetterCP/M using an existing system disk as its source.

For example:

```
A0>SYSGEN A: B:
```

requests installation of the operating system from drive A onto drive B.

The source and destination must be different physical drives.

As with DUP, BetterCP/M protects physical drive 0 against destination writes.

Installing from a System-Image File

SYSGEN can also install BetterCP/M from a system-image package.

For example:

```
A0>SYSGEN SYSTEM.SYS B:
```

installs the operating system contained in `SYSTEM.SYS` onto drive B.

The package must be a valid BetterCP/M system image suitable for the destination.

A drive-qualified filename can also be used:

```
A0>SYSGEN C:SYSTEM.SYS B:
```

This reads the system-image package from drive C and installs it onto drive B.

Confirming Installation

Before writing the system area, SYSGEN checks the destination and requests confirmation.

It displays a warning similar to:

```
The destination system area will be replaced.
Files outside the reserved system tracks will be preserved.
Proceed (Y/N)?
```

Check the destination carefully.

Only an affirmative response allows installation to proceed.

SYSGEN verifies the system records it writes.

When installation completes successfully, it reports:

```
System installed and verified. The destination is bootable.
```

This confirms that the operating-system image has been installed and verified.

It does not mean that every utility or supporting file required for your preferred working environment has been copied to the disk.

Existing Files

Unlike formatting, SYSGEN is designed to preserve ordinary files outside the reserved system area.

This allows you to update or replace the installed operating system without necessarily erasing the disk's filesystem.

However, SYSGEN replaces the system-area contents.

If an installation is interrupted or fails, the destination may no longer contain a complete bootable operating system.

Keep a working system disk available when performing system installation or maintenance.

10.6 Backing Up and Restoring System Disks

A reliable backup is particularly important when working with system disks.

A damaged or incorrectly configured system disk may prevent your computer from booting, even if its ordinary files remain intact.

BetterCP/M provides several ways to preserve system disks and their contents.

Making a Complete Disk Copy

For a complete disk backup, use DUP.

For example:

```
A0>DUP /COPY A: B:
```

This copies the source disk to the destination, including the system area and ordinary file contents represented by the disk format.

The destination must be suitable for the operation, and its previous contents will be replaced.

DUP verifies the copied data before reporting successful completion.

A complete disk copy is often the simplest way to preserve a working system configuration.

Saving the Operating System to a File

SYSGEN provides another useful facility: saving the installed operating system as a system-image package.

For example:

```
A0>SYSGEN A:
```

saves the operating system installed on drive A as `SYSTEM.SYS` in the current drive and user area.

You can also specify a destination filename:

```
A0>SYSGEN A: B:BASE090.SYS
```

This saves the installed system from drive A into the named package on drive B.

The package contains the operating-system image and its saved configuration information.

It does not contain ordinary files from the source disk.

Maintaining Several System Configurations

System-image packages are particularly useful when maintaining several BetterCP/M configurations.

For example, you might preserve separate images for different drive arrangements or hardware configurations.

Each can be saved under a descriptive filename.

When you want to restore a particular configuration, install its package using SYSGEN.

For example:

```
A0>SYSGEN B:BASE090.SYS C:
```

This installs the saved system image onto drive C after the normal confirmation.

The destination must have a compatible system area.

Restoring a System Disk

If a system disk becomes damaged, the appropriate recovery procedure depends on the extent of the damage.

If only the installed operating system has been affected, you may be able to reinstall it using SYSGEN while preserving ordinary files.

If the disk's physical format or filesystem has been damaged, a complete restoration from a known good disk copy may be necessary.

If the disk itself is unreliable, replace the media before attempting restoration.

Protecting Your Backups

A backup is useful only if it can be read when needed.

For important system disks, consider maintaining more than one copy.

Keep at least one backup separate from the working system.

After preparing a backup, verify that it can be read successfully. When practical, test that a backup system disk can actually boot the computer.

Remember that a system-image package and a complete disk backup serve different purposes.

A system-image package preserves the installed operating system. A complete disk copy preserves the disk's contents, including ordinary files.

Both can be valuable when maintaining a BetterCP/M system.

Chapter 11 - Date and Time

BetterCP/M provides facilities for reading the system date and time through supported clock hardware or an emulator.

Unlike modern computers, many early CP/M systems did not have a built-in real-time clock. Some relied on external hardware, while others had no clock at all.

BetterCP/M accommodates these differences through loadable clock providers. A clock provider allows the operating system to obtain the current date and time without requiring applications to communicate directly with the clock hardware.

The **TIME** utility provides a convenient way to examine the system clock and identify the clock provider currently in use.

BetterCP/M 1.0 includes clock providers for FreHD and z80pack. Both support reading the clock, but neither supports changing it.

This chapter explains how to use these facilities. Complete command syntax and options are provided in Part II, *Command and Utility Reference*.

11.1 Clock Support

BetterCP/M provides a common clock interface that can be used with different hardware and emulated environments.

The interface separates the operating system's date and time services from the particular clock supplying the information.

For example, on a TRS-80 Model 4 equipped with FreHD, BetterCP/M can obtain the date and time through FreHD's clock interface.

Under z80pack, BetterCP/M can obtain the date and time from the clock facilities provided by the emulator.

Both arrangements use the same TIME utility.

Clock Providers

Clock support is supplied by Resident System Extensions, or RSXs.

BetterCP/M 1.0 includes two clock providers:

Provider	Environment	Extension
FreHD	TRS-80 Model 4 with FreHD	`FREHDCLK.RSX`
z80pack	z80pack/cpmsim	`ZPRTC.RSX`

Neither provider is loaded automatically merely because its file is present on the system disk.

You must load the appropriate provider before using TIME.

For example, on a Model 4 equipped with FreHD:

```
A0>RSX LOAD FREHDCLK
```

Under z80pack:

```
A0>RSX LOAD ZPRTC
```

Once the provider is installed, TIME can obtain the current date and time.

You can examine the installed RSXs with:

```
A0>RSX LIST
```

Read-Only Clock Support

Both BetterCP/M 1.0 clock providers are read-only.

They can obtain the date and time from their respective clock sources, but they do not change those clocks.

This does not prevent programs from using the current date and time.

It simply means that correcting an incorrect clock must be done through the facilities provided by the hardware, emulator, or host environment.

11.2 Displaying the Date and Time

To display the current date and time, enter:

```
A0>TIME
```

TIME obtains the date and time from the installed clock provider and displays the result.

The time shown is the time reported by the provider.

For example, under z80pack, the provider reads the emulator's clock interface.

On a Model 4 equipped with FreHD, the provider obtains the date and time maintained by FreHD.

Identifying the Clock Provider

To determine which clock provider is installed, enter:

```
A0>TIME /PROVIDER
```

TIME displays information about the provider, including its name, supported interface version, and capabilities.

For example, this can help you determine whether BetterCP/M is using the FreHD or z80pack clock service.

Both supplied providers identify themselves as supporting clock reads but not clock setting.

When No Provider Is Installed

If you enter TIME without first loading a clock provider, the utility cannot obtain the current date and time.

It reports that the required clock service is unavailable.

To correct this, load the appropriate provider for your system.

For example:

```
A0>RSX LOAD ZPRTC
A0>TIME
```

If a provider is already installed but TIME still cannot obtain the clock, the underlying hardware or emulator may not be supplying valid clock information.

The appropriate remedy depends on the platform.

11.3 Setting the Date and Time

BetterCP/M's TIME utility includes a command for requesting that a clock be set.

However, **the FreHD and z80pack providers supplied with BetterCP/M 1.0 are read-only**.

Consequently, attempts to set the clock through either provider are rejected.

The SET facility is nevertheless implemented in TIME.COM and is available for use with clock providers that support it.

The SET Command

The command has the following form:

```
TIME /SET LL HH HR MN SC
```

The five arguments represent the date and time in BetterCP/M's native clock-record format.

Unlike the date-setting commands found in some operating systems, TIME does not accept an ordinary calendar-date string.

Instead, the arguments are five two-digit hexadecimal values.

The first two bytes represent the day count, with the low byte first. The remaining three bytes represent the hour, minute, and second in packed-BCD notation.

For example:

```
A0>TIME /SET 01 00 12 34 56
```

requests day 1 of the CP/M clock epoch, January 1, 1978, at 12:34:56.

The example illustrates the syntax. It will not set the clock when either of the standard BetterCP/M 1.0 providers is installed.

Unsupported Clock Setting

When a clock provider does not support setting the clock, TIME reports that the operation is unsupported.

The existing clock value is not changed.

This is the expected result with both `FREHDCLK.RSX` and `ZPRTC.RSX`.

Invalid Date and Time Values

TIME checks the supplied hexadecimal values before requesting a clock-setting operation.

For example, it rejects malformed hexadecimal arguments, invalid packed-BCD values, and out-of-range hours, minutes, or seconds.

Invalid command arguments produce a usage message.

For complete details of the accepted format, consult the TIME command reference.

11.4 Clock Providers

Although both supplied clock providers offer the same basic date-and-time service, they obtain their information differently.

FreHD Clock Provider

The FreHD provider is supplied as:

```
FREHDCLK.RSX
```

It obtains the date and time through FreHD's extended hardware interface.

FreHD maintains its clock information using a DS1307-compatible clock facility.

To use it:

```
A0>RSX LOAD FREHDCLK
A0>TIME
A0>TIME /PROVIDER
```

The provider supports dates from 2000 through 2099.

It is read-only and does not modify the FreHD hardware clock.

FreHD's clock interface is also supported by suitable emulator configurations, allowing the same provider to be used without requiring a physical FreHD device.

The actual availability of the clock depends on the computer or emulator configuration.

z80pack Clock Provider

The z80pack provider is supplied as:

```
ZPRTC.RSX
```

It obtains the date and time through the clock interface provided by z80pack/cpmsim.

To use it:

```
A0>RSX LOAD ZPRTC
A0>TIME
A0>TIME /PROVIDER
```

Like the FreHD provider, ZPRTC is read-only.

It does not change the emulator's clock or its operating mode.

The provider supports dates through 2099.

Changing Providers

Clock providers are resident extensions.

You can examine and manage them using the RSX commands described in Chapter 8.

For example:

```
A0>RSX LIST
```

displays the installed extensions.

The corresponding RSX management commands can be used to unload a provider when it is no longer needed.

Applications using BetterCP/M's clock service obtain their information through the common clock interface, rather than communicating directly with either FreHD or z80pack.

11.5 Clock Availability and Reliability

The accuracy of BetterCP/M's date and time depends on the clock source.

If the underlying clock is incorrect, TIME will report an incorrect date or time.

If the source is unavailable or cannot provide a valid reading, TIME reports an error rather than inventing a time value.

Clock Errors

A clock-reading operation may fail because the provider cannot communicate with the hardware or because the returned information is invalid.

For example, a FreHD clock request may fail if the controller is unavailable or does not respond correctly.

Similarly, a clock provider may reject a reading whose date or time fields are invalid.

BetterCP/M's clock providers check the information they receive before returning it to TIME.

Reading the Clock During a Time Change

Clock values can change while they are being read.

For example, a program might begin reading the clock immediately before a minute changes.

BetterCP/M's providers account for this possibility.

The FreHD provider obtains a consistent clock sample through the FreHD interface.

The z80pack provider checks that its readings form a consistent timestamp and retries when necessary.

These measures help prevent applications from receiving inconsistent combinations of date and time fields.

They do not, however, correct an inaccurate underlying clock.

11.6 Compatibility with CP/M Clock Facilities

Digital Research CP/M 2.2 did not provide a standard general-purpose date-and-time service.

Consequently, different CP/M systems adopted different approaches to clock support.

One historically significant extension was the date-and-time interface provided by P2DOS.

BetterCP/M provides compatibility with this interface through a separate resident extension.

P2DOS Compatibility

The optional compatibility extension is implemented as:

```
P2DOS.RSX
```

It provides the P2DOS-compatible date-and-time interface for programs that use the corresponding BDOS functions.

P2DOS.RSX does not communicate directly with FreHD or z80pack.

Instead, it obtains clock services through BetterCP/M's installed clock provider.

This means that the same provider can serve both native BetterCP/M applications and programs using the P2DOS-compatible interface.

Loading P2DOS Support

`P2DOS.RSX` is not on the selected System Disk. To use it, obtain a compatible separately built carrier, install it in A0, then load an appropriate clock provider and the P2DOS extension. See Section 3.2, *What Is on Each Disk*, for distribution scope.

For example, under z80pack:

```
A0>RSX LOAD ZPRTC
A0>RSX LOAD P2DOS
```

Once installed, compatible programs can request clock information through the P2DOS interface.

As with the native TIME utility, the available operations depend on the provider.

Because the supplied providers are read-only, P2DOS-compatible requests to set the clock are not supported.

Conventional CP/M Applications

Ordinary CP/M 2.2 applications do not require these extensions.

They continue to use the standard CP/M programming interfaces.

Applications that require date-and-time services, however, must use a supported clock interface and have access to an appropriate provider.

For programming details, consult the BetterCP/M RSX and CPX Programmer's Guide and the relevant engineering specifications.

Chapter 12 - Troubleshooting

BetterCP/M is designed to provide a reliable CP/M operating environment, but sooner or later you're likely to encounter a command that doesn't behave as expected, a disk that can't be read, or a program that refuses to run.

Fortunately, many problems are straightforward to identify and correct.

Sometimes the cause is as simple as a mistyped filename or an incorrect drive selection. Other problems may involve disk formats, configuration settings, resident extensions, or the limitations of a particular computer.

This chapter describes common problems you may encounter when using BetterCP/M and suggests ways to resolve them.

It isn't intended to catalog every possible error message. The complete message reference appears in Appendix F, *System Messages and Error Reference*. Individual commands and utilities are documented in Part II, *Command and Utility Reference*.

When troubleshooting, it's generally best to begin with the simplest explanation before making changes to the system configuration.

12.1 Common Command Errors

Command Not Found

If BetterCP/M doesn't recognize a command, it may display a question mark or another command-related error.

For example:

```
A0>MYPROG
?
```

This usually means that BetterCP/M couldn't locate the requested program.

First, check the spelling of the command.

If you're trying to run a transient program, make sure its `.COM` file is available.

BetterCP/M normally searches the current drive and user area first, followed by drive A, user area 0.

For example, if you're working in `B3:`, BetterCP/M will look for an unqualified transient program in `B3:` and then `A0:`.

You can examine these locations with:

```
B3>DIR *.COM
B3>DIR A0:*.COM
```

If the program is elsewhere, you can specify its location explicitly.

For example:

```
B3>C2:MYPROG
```

This requests `MYPROG.COM` from drive C, user area 2.

Remember that an explicitly qualified program name restricts the search to the specified location.

A Command Doesn't Recognize an Option

Some BetterCP/M commands are available in both resident and transient versions.

The resident version generally provides the most frequently used operations, while the transient version may offer additional options.

For example, the resident DIR command doesn't provide every option available in `DIR.COM`.

If you enter:

```
A0>DIR /WHEREIS ED.COM
```

BetterCP/M may respond:

```
DIR: Use :DIR.
```

This means the requested operation is available through the transient version.

Enter:

```
A0>:DIR /WHEREIS ED.COM
```

The colon tells BetterCP/M to execute the transient program rather than the resident command.

You can also use a period as the prefix.

If the transient program isn't available, BetterCP/M reports the normal command-lookup failure.

For more information, see Chapter 5, *The Command Environment*.

Incorrect Command Syntax

If a command reports invalid syntax, check the order and spelling of its arguments.

Some BetterCP/M commands accept several forms, while others require arguments in a particular order.

For example, COPY and REN use different conventions for specifying source and destination files.

A command may also require an option to appear separately from its filename arguments.

Consult the command's reference entry in Part II for its accepted syntax.

Unexpected Interactive Prompts

Some commands request confirmation before modifying or replacing files.

This can happen even when the command is executed from a SUBMIT file.

For example, transient `COPY.COM` may ask how to handle an existing writable destination file. Resident COPY refuses that collision unless `/O` is supplied; it does not offer the transient utility's collision prompt.

When a prompt offers several abbreviated responses, entering `?` may display an explanation of the available choices.

If you're preparing a batch file, examine whether the command provides options for specifying its behavior without prompting.

For batch-oriented copying, invoke the transient explicitly with `:COPY` and use its `/B` option. The resident command does not support `/B` or automatically switch to `COPY.COM`.

Chapter 6 explains batch processing in more detail.

12.2 File and Disk Problems

File Not Found

If a command reports that a file cannot be found, begin by checking the current drive and user area.

For example:

```
A0>DIR
```

displays the files in the current location.

If you believe the file is on another drive or in another user area, specify that location:

```
A0>DIR B3:
```

Remember that files in different user areas are ordinarily treated as separate collections.

A file named `REPORT.TXT` in `A0:` is distinct from a file with the same name in `A5:`.

You can also use BetterCP/M's extended DIR facilities to search for a file.

For example:

```
A0>:DIR /WHEREIS REPORT.TXT
```

This searches the configured drives and user areas according to the transient DIR utility's file-location rules.

Read-Only Files

If BetterCP/M refuses to modify or erase a file, check whether it has the read-only attribute.

Read-only protection is intended to prevent accidental changes.

Some utilities provide options for overriding this protection, but you should use them only when you're certain that the file should be modified.

If the file belongs to the operating system or an installed application, consider why it was protected before removing the attribute.

The STAT utility provides facilities for examining and changing file attributes.

A Disk Has Become Read-Only

BetterCP/M may treat a disk as read-only when it cannot safely determine that the disk currently in a drive matches the directory information already held by the operating system.

This can happen after removable media have been changed.

The read-only condition helps prevent directory information from one disk being written to another.

If a disk becomes read-only unexpectedly, complete any active operation and use the appropriate disk-reset or warm-boot procedure before attempting further writes.

Do not attempt to bypass the protection by forcing a write.

If the problem persists, check the disk's write-protection setting and the drive configuration.

A Disk Cannot Be Read

A disk-read failure can have several causes.

The disk may be damaged, the drive may not be ready, or BetterCP/M may be using the wrong disk-format definition.

Begin by checking that the correct disk is inserted and that the drive is available.

If the disk was prepared on another CP/M computer, verify that the assigned format matches the disk's actual organization.

Use CONFIG to examine the current drive configuration:

```
A0>CONFIG /SHOW
```

If the format is incorrect, select the appropriate definition before attempting further access.

Avoid writing to a disk whose format you're uncertain about. An incorrect format assignment can cause existing data to be damaged.

Disk Full

A disk can run out of space even when its files appear relatively small.

CP/M allocates storage in blocks, and each file consumes one or more allocation blocks.

A disk can also run out of directory entries before its data area is completely full.

If BetterCP/M reports insufficient disk space or directory capacity, remove unnecessary files or copy them to another disk.

You may also be able to use a disk format with greater capacity, provided your hardware supports it.

Formatting a disk to change its organization destroys its existing contents, so preserve any important files first.

Suspected Disk Errors

If a disk repeatedly produces read errors, DUP can examine its physical readability.

For example:

```
A0>DUP /CHECK B:
```

This checks the disk in drive B without modifying its contents.

DUP reports unreadable sectors and provides a summary when the check completes.

A successful check means the examined sectors could be read. It does not guarantee that the filesystem or individual files are logically correct.

If the disk contains important data, preserve a readable copy before attempting repairs or other changes.

Chapter 10 describes disk checking in more detail.

12.3 Configuration Problems

An incorrect configuration can make a drive unavailable, prevent a startup command from running, or cause a peripheral device to behave unexpectedly.

Fortunately, many configuration problems can be corrected without rebuilding the operating system.

Examining the Current Configuration

When a configuration problem occurs, begin by examining the settings currently in use.

Enter:

```
A0>CONFIG /SHOW
```

CONFIG displays information about the active configuration and the settings saved for subsequent cold boots.

Pay particular attention to differences between the current settings and the saved defaults.

A drive that works during the current session may not be configured the same way after a cold boot.

Likewise, a temporary change may have been made without being saved.

A Drive Is No Longer Available

If a logical drive becomes unavailable after a configuration change, check its assignment.

The drive may have been unbound, associated with an incompatible physical drive, or assigned an unsuitable disk format.

Use CONFIG to examine the available drives and their definitions.

If necessary, restore the appropriate assignment.

For example, a drive that was temporarily unbound can be assigned again using CONFIG's drive-configuration facilities.

The correct settings depend on the computer and the disk format.

Configuration Changes Disappear After Restarting

Some CONFIG operations affect only the current session.

If you change a drive assignment but don't save the new configuration, the previous saved settings may be restored during the next cold boot.

This is normal behavior.

To make a configuration permanent, use CONFIG's save facilities.

Chapter 7 explains the distinction between temporary and saved settings.

Device Assignments Don't Work

BetterCP/M retains CP/M's IOBYTE mechanism for selecting logical character devices.

However, selecting a device does not guarantee that the corresponding hardware exists or is supported by the BIOS.

If a printer, reader, punch, or other device does not respond, verify that the selected assignment is supported by your computer.

Remember that STAT changes the live IOBYTE, while CONFIG can save a value for subsequent disk cold boots.

Changing one does not necessarily change the other.

12.4 Extension Problems

BetterCP/M's resident extensions provide additional commands and system services, but problems can occur if an extension is missing, damaged, incompatible, or loaded in the wrong order.

A CPX Command Is Missing

If a command normally provided by a CPX is unavailable, examine the configured extensions:

```
A0>CPX LIST
```

Check whether the expected extension is listed.

If it is missing, you may need to load it.

If it is listed but its commands remain unavailable, the extension may have failed to load.

An extension's appearance in the configured list does not necessarily guarantee that it is active.

The CPX? Warning

If an optional CPX cannot be loaded or fails validation, BetterCP/M may display:

```
CPX?
```

The system continues to the command prompt with the remaining usable extensions.

Commands supplied by the rejected CPX are unavailable.

You can remove the unusable extension from the configured environment.

For example:

```
A0>CPX UNLOAD BAD
```

where `BAD` is the name of the affected extension.

Alternatively, replace the extension file with a valid copy.

Until the problem is corrected, the warning may recur when BetterCP/M restores the command environment.

An RSX Cannot Be Loaded

If an RSX fails to load, first check that its file is available and compatible with the running system.

Some RSXs also require supporting extensions.

For example, a terminal personality needs a suitable screen backend.

If you attempt to load a terminal personality without its required backend, the operation may fail.

Load the supporting component first, then try again.

An RSX Cannot Be Unloaded

Some resident extensions depend on others.

BetterCP/M prevents certain unload operations when the extension is still required by another resident component.

For example, a terminal personality may depend on a screen backend.

To remove the monitor, unload the terminal personality first, then unload the backend.

Chapter 8 explains extension dependencies in greater detail.

A Full-Screen Program Displays Incorrectly

If a full-screen application displays misplaced text, unexpected characters, or incorrect cursor movement, check its terminal configuration.

First, examine the installed RSXs:

```
A0>RSX LIST
```

Make sure the expected terminal personality and screen backend are loaded.

Next, check that the application is configured for the same terminal type.

For example, an application configured for VT52 should be used with BetterCP/M's VT52 personality.

You can also run the corresponding terminal test utility:

```
A0>VT52TST
```

If the test works correctly but the application does not, the application may be using unsupported terminal facilities or expecting different screen dimensions.

Chapter 9 provides more information about terminal configuration and troubleshooting.

Insufficient Program Memory

RSXs occupy protected memory that would otherwise be available to transient programs. CPX command code is reclaimable.

If a program reports insufficient memory, use `STAT MEM` and `RSX LIST` to check memory use and loaded services.

You may be able to unload facilities that aren't needed for the current task.

For example, if you're no longer using a full-screen application, unloading its terminal personality and screen backend may recover enough memory to run a larger program.

12.5 Startup and Boot Problems

Startup problems can arise from damaged system disks, incorrect saved configurations, missing files, or unsuitable startup commands.

The appropriate recovery procedure depends on how far the system gets before the problem occurs.

BetterCP/M Doesn't Boot

If BetterCP/M fails to start, begin with the simplest possibilities.

Check that the correct system disk or disk image is being used.

On physical hardware, make sure the disk is inserted correctly and that the drive is functioning.

On an emulator, verify that the appropriate disk image is attached to the expected drive.

If the disk was prepared for another computer, it may not contain the BIOS or bootstrap code required by your machine.

BetterCP/M system images are platform-specific, even though the operating environment is largely portable.

Consult the appropriate platform documentation for the expected boot procedure.

The System Boots but Reports Errors

If BetterCP/M reaches the command prompt but displays errors during startup, the problem may involve the configured startup command.

The standard distribution uses:

```
SUBMIT STARTUP
```

This executes `STARTUP.SUB`.

If the file is missing or contains an invalid command, startup may report an error even though the operating system itself is functioning.

Check the startup file and any programs or extensions it invokes.

A Startup Extension Fails

If `STARTUP.SUB` loads resident extensions, make sure the required files are present.

Also check their loading order.

For example, a terminal personality must be loaded after its supporting screen backend.

If a newly added startup command causes problems, remove it from `STARTUP.SUB` and test it separately at the command prompt.

Once the command works correctly, it can be restored to the startup procedure.

Saved Configuration Prevents Normal Operation

An incorrect saved drive configuration can make disks inaccessible after a cold boot.

If the system still reaches the command prompt, use CONFIG to examine and correct the settings.

If the system cannot boot far enough to run CONFIG, you may need to start from a known working system disk.

This is one reason it's worth keeping an unmodified backup of a working BetterCP/M installation.

12.6 Recovery Procedures

When a problem occurs, the most important thing is to avoid making it worse.

In particular, don't begin formatting disks, changing drive definitions, or overwriting files until you've identified the likely cause.

Begin with the Current State

Before making changes, determine what still works.

Can BetterCP/M reach the command prompt?

Can it display a directory?

Can it read files from the affected disk?

Can CONFIG display the current configuration?

Can the relevant utility run without performing a destructive operation?

These observations can help distinguish a command problem from a configuration problem, a filesystem problem, or a hardware failure.

Preserve Important Files

If a disk is producing errors but remains readable, preserve important files before attempting repairs.

When possible, copy them to another disk.

Avoid unnecessary writes to the affected medium.

If the disk appears physically unreliable, repeated attempts to modify it may make recovery more difficult.

Recovering from an Interrupted Disk Operation

Formatting, disk copying, and system installation can leave a destination disk incomplete if interrupted.

For example, an interrupted DUP format may leave a partially formatted disk.

An interrupted disk copy may leave a destination containing only part of the source's contents.

An interrupted SYSGEN installation may leave the system area incomplete, preventing the disk from booting.

In these situations, don't assume that the destination is usable simply because some operations completed successfully.

Correct the cause of the interruption and repeat the appropriate operation using a reliable source.

Remember that formatting and disk copying are destructive operations.

Recovering a System Disk

If the operating system's reserved system area has been damaged, SYSGEN may be able to reinstall BetterCP/M without replacing ordinary files.

For example:

```
A0>SYSGEN SYSTEM.SYS B:
```

installs the operating system from a valid system-image package onto drive B.

The destination must be properly prepared and compatible with the image.

SYSGEN replaces the reserved system area while preserving records outside that area.

However, it cannot repair a damaged filesystem or recover files that have been lost.

If the disk's contents are extensively damaged, restoring a complete disk backup may be necessary.

Chapter 10 explains the distinction between system installation and complete disk copying.

Returning to a Known Working Configuration

If a problem began after changing system settings, consider restoring the previous configuration.

For temporary changes, a cold boot may restore the saved defaults.

For saved changes, you may need to use CONFIG or reinstall a previously saved system image.

If you're experimenting with disk formats or resident extensions, maintaining a separate working system disk makes recovery considerably easier.

When to Consult the Technical Documentation

Most ordinary problems can be resolved using the procedures described in this manual.

Some problems, however, involve hardware behavior, BIOS configuration, or operating-system facilities beyond the scope of the User's Guide.

If a problem persists after checking the relevant commands and configuration settings, consult the platform documentation or the more detailed BetterCP/M technical guides.

Overall System Architecture explains the system organization. The Build disk instructions, platform material, and engineering records support development and maintenance. Appendix H distinguishes available references from planned manuals.

Part II - Command and Utility Reference

Chapter 13 - Introduction to the Command Reference

Part I introduced BetterCP/M's operating environment and explained how to use its principal facilities. Part II provides a detailed reference to the commands and utilities available under BetterCP/M 1.0.

This part of the manual is intended primarily for consultation. You need not read it from beginning to end. When you want to know how a particular command works, which options it accepts, or how it handles an unusual situation, you can turn directly to the appropriate entry.

BetterCP/M retains the familiar commands and conventions of Digital Research CP/M 2.2, while adding new commands and extending the capabilities of many existing ones.

Some commands are resident, some are transient programs, and some are available in both forms. Where the resident and transient versions differ, the reference describes those differences.

The command descriptions are organized by function rather than by their internal implementation. Each entry explains what the command does, how to invoke it, what options are available, and any important restrictions or compatibility considerations.

13.1 Command Syntax Conventions

Throughout this reference, command syntax is presented using a consistent set of conventions.

For example, a command description might begin with:

```
COMMAND filespec [options]
```

This is a description of the command's syntax, not a command to enter literally.

Command Names

Command names are shown in uppercase letters:

```
DIR
ERA
REN
COPY
```

BetterCP/M accepts command names without regard to letter case.

For example:

```
A0>DIR
A0>dir
A0>Dir
```

all request the same command.

For clarity and consistency, the examples in this manual generally use uppercase command names.

Required Arguments

Arguments that must be supplied are shown without enclosing brackets.

For example:

```
TYPE filespec
```

indicates that TYPE requires a file specification identifying the file to display.

An example would be:

```
A0>TYPE README.TXT
```

The word `filespec` is a placeholder. Replace it with an actual filename or file specification.

Optional Arguments

Optional arguments are enclosed in square brackets.

For example:

```
COMMAND filespec [options]
```

indicates that the command requires a file specification but may also accept additional options.

The brackets are part of the syntax notation and are not normally typed.

Thus, if a command accepts an optional `/P` switch, you might enter:

```
A0>COMMAND README.TXT /P
```

or omit the switch:

```
A0>COMMAND README.TXT
```

Some BetterCP/M commands also use square brackets as part of their actual file-selection syntax. In those cases, the command description explicitly identifies the brackets as characters that must be entered.

Alternative Forms

When a command accepts several forms, they are normally shown separately.

For example:

```
DIR
DIR filespec
DIR DU:filespec
```

These represent different ways of invoking DIR.

The command reference explains the purpose and behavior of each supported form.

Where alternatives are shown using a vertical bar, such as:

```
A|B
```

the bar means that one of the alternatives is selected. It is not typed unless the command description explicitly says otherwise.

Repeated Arguments

Some commands accept more than one filename, option, or other argument.

When an argument can be repeated, the syntax description will indicate that possibility and explain any applicable limits.

For example, a command may accept a list of source files, while another may permit only one source specification.

Do not assume that a command accepting one filename can also accept several. The individual command entry defines the permitted forms.

Spaces

Spaces normally separate a command name from its arguments and separate individual arguments from one another.

For example:

```
A0>COPY SOURCE.TXT B:SOURCE.TXT
```

The command name, source file, and destination file are separate elements.

Some commands also accept arguments containing spaces when enclosed in quotation marks.

For example:

```
A0>CONFIG /STARTUP="SUBMIT STARTUP"
```

The quotation marks preserve the startup command as a single argument.

Quoting rules are command-specific. The presence of quotation marks in one command's syntax does not imply that every BetterCP/M command supports quoted arguments.

Numbers and Numeric Bases

Unless otherwise stated, numeric arguments are decimal.

Some commands, particularly those used to examine or modify memory, accept hexadecimal values.

For example, a memory address might be written as:

```
0100
```

and interpreted as hexadecimal by the command concerned.

The command reference identifies the expected numeric base.

Where ambiguity is possible, the manual explicitly identifies hexadecimal values with the suffix `H`, as in `0100H`.

The suffix is an explanatory convention and should not be included in a command unless that command's syntax permits it.

Examples and the Command Prompt

Command examples generally include the BetterCP/M prompt:

```
A0>DIR
```

You type only the characters following the prompt.

The prompt itself identifies both the current drive and user area, including zero.

For example:

```
B3>DIR
```

shows DIR being entered while drive B, user area 3, is selected.

When several commands are shown together, each command is entered after the preceding operation has completed and the prompt has returned, unless otherwise indicated.

13.2 Command Options

Many BetterCP/M commands accept options that modify their behavior.

Options are commonly introduced by a slash (`/`).

For example:

```
A0>DIR /P
```

The `/P` option requests paged directory output in the transient DIR utility.

Other options may control sorting, file selection, confirmation, output formatting, or the treatment of existing files.

Options and Their Arguments

Some options consist of a switch alone:

```
/P
```

Others require a value:

```
/C=2
```

The command reference identifies which options require values and explains the accepted forms.

For example, an option might accept a number, a filename, or a selection from several predefined values.

Do not assume that an option requiring a value can be used without one.

Combining Options

Some commands allow several options to be supplied together.

For example:

```
A0>:COPY B1:*.DAT C3: /B /O
```

Here, `/B` requests batch-oriented operation, while `/O` specifies how existing writable destination files should be handled.

The two options work together to perform the operation without asking for overwrite confirmation.

Other combinations may be invalid or mutually exclusive.

Each command reference identifies the combinations it supports.

Option Placement

The permitted position of an option depends on the command.

Some commands accept options before or after a file specification. Others require a particular order.

For this reason, the examples and syntax descriptions should be followed rather than assuming that all BetterCP/M utilities use identical option-placement rules.

Default Behavior

When an option is omitted, the command uses its default behavior.

For example, a file-copying command may normally request confirmation before replacing an existing destination.

An option may change that behavior by instructing the command to overwrite, skip, or reject the existing file.

The command reference explains the defaults where they are important.

Interactive and Noninteractive Operation

Some commands request information from the user while executing.

For example, a utility may ask whether an existing file should be replaced.

BetterCP/M provides options for certain utilities that allow such decisions to be specified in advance.

This is particularly useful in SUBMIT files.

However, placing a command in a SUBMIT file does not automatically make it noninteractive.

If a command may prompt for input, consult its reference entry before using it in an unattended batch procedure.

File Selection and Attributes

Several BetterCP/M utilities support extended file-selection facilities.

These can include drive/user specifications, wildcards, and file-attribute conditions.

For example, a command may allow you to select files with particular attributes or search across several user areas.

These facilities are not necessarily available in every command.

The reference entry identifies which selectors are supported and where they may appear.

For an introduction to file specifications, wildcards, and attributes, see Chapter 4, *Working with Drives and Files*.

13.3 Resident and Transient Versions

BetterCP/M distinguishes between resident commands and transient programs.

A **resident command** is provided by the active command environment. It can be executed without loading an ordinary `.COM` program from disk.

A **transient command** is an executable program that BetterCP/M loads when it is invoked.

Some commands are available in both forms.

Why Two Versions?

The resident versions of frequently used commands provide convenient access to common operations.

For example, the standard resident command package supplies familiar commands such as DIR, ERA, REN, TYPE, COPY, and MOVE.

Their transient counterparts may offer additional capabilities.

This arrangement allows BetterCP/M to provide useful commands without permanently occupying memory with every available option.

The resident version can remain compact, while the transient version provides more extensive functionality when needed.

Normal Command Execution

When you enter:

```
A0>DIR
```

BetterCP/M uses the normal command-search procedure.

If DIR is available as a resident command, that version is selected.

If no resident command handles the name, BetterCP/M attempts to locate and execute the corresponding transient program.

For an unqualified transient program name, BetterCP/M searches the current drive and user area first, followed by `A0:`.

A program found in the current drive/user takes precedence over one found in `A0:`.

For more information about command search and execution, see Chapter 5.

Explicit Transient Execution

Sometimes you may want to execute a transient program even though a resident command with the same name is available.

BetterCP/M provides two prefixes for this purpose:

```
:
.
```

Place either prefix immediately before the command name.

For example:

```
A0>:DIR
```

or:

```
A0>.DIR
```

Both forms bypass the resident command and request the transient `DIR.COM` program.

The prefix can also be used with arguments:

```
A0>:DIR /WHEREIS ED.COM
```

This invokes the transient DIR utility's file-location facility.

Drive/User Qualifications

An explicit transient invocation can include a drive/user qualification.

For example:

```
A0>:B3:MYPROG ARGUMENTS
```

This requests `MYPROG.COM` from drive B, user area 3.

The transient-execution prefix appears before the drive/user qualification.

A qualified invocation restricts the program search to the specified location.

By contrast, an unqualified transient invocation follows the normal search procedure, including the fallback to `A0:.`

When the Resident Command Lacks an Option

BetterCP/M 1.0 does not automatically transfer execution from a resident command to its transient counterpart when an option is unavailable.

Instead, certain resident commands recognize options that require the transient version and report that the transient program should be used.

For example:

```
A0>DIR /WHEREIS ED.COM
DIR: Use :DIR.
```

You can then enter:

```
A0>:DIR /WHEREIS ED.COM
```

The command is not automatically reissued.

If the transient program cannot be found, BetterCP/M reports the normal command-lookup failure.

Differences Between Versions

The resident and transient versions of a command may differ in several respects.

They may accept different options, provide different output formats, or support different file-selection facilities.

The command reference distinguishes these differences explicitly.

Where appropriate, each entry provides separate syntax descriptions for the resident and transient forms.

Unless otherwise stated, examples apply to the form identified in the surrounding text.

13.4 Getting Help and Version Information

BetterCP/M provides several facilities for identifying commands, obtaining assistance during interactive operations, and examining software versions.

Not every command supports the same help facilities, so the individual reference entries identify the available options.

Version Information

BetterCP/M utilities generally support the `/VER` option for displaying version and build information.

For example:

```
A0>:DIR /VER
```

identifies the transient DIR utility.

A version display might resemble:

```
BetterCP/M DIR 1.0 (Build 002)
```

The exact build number depends on the installed utility.

Resident commands can also report version information.

For example:

```
A0>DIR /VER
```

identifies the resident DIR implementation when it is available.

A resident version display identifies the command package containing the command.

This distinction is useful when comparing the resident and transient versions of a utility.

Version Numbers and Build Numbers

BetterCP/M distinguishes between a software version and a build number.

The **version number** identifies the software release.

The **build number** identifies a particular revision of the executable.

For example:

```
BetterCP/M DIR 1.0 (Build 002)
```

identifies version 1.0 of DIR, build 002.

Different utilities maintain their own build sequences.

Resident commands supplied by the same command processor extension share the build identity of that extension.

Consequently, the resident and transient versions of a command may report different build numbers even though both belong to BetterCP/M 1.0.

Using /VER

For BetterCP/M-owned utilities, `/VER` is normally used by itself.

For example:

```
A0>TIME /VER
```

The utility displays its identification and exits without performing its ordinary operation.

Do not combine `/VER` with filenames or other options unless the command reference explicitly permits it.

An invalid combination may produce:

```
Invalid /VER usage.
```

The `/VER` convention applies to BetterCP/M utilities. Third-party CP/M programs may use different conventions or provide no version-reporting facility.

System Version Information

BetterCP/M also provides the VER command for identifying the operating system.

For example:

```
A0>VER
```

displays the system version.

Additional VER options provide more detailed identification information, including subsystem information and the system identification banner.

The complete forms are documented in the VER command reference.

Help at Interactive Prompts

Some BetterCP/M utilities provide help while waiting for a response.

For example, a command that encounters an existing destination file may display a prompt offering several possible actions.

When the prompt includes `?` as an available response, entering it displays a short explanation of the choices.

For example, COPY's file-collision prompt may offer:

```
Y/N/O/S/R/?
```

Entering `?` displays information about the available responses and returns to the same decision.

Requesting help does not itself select an action or change the command's current operation.

Not every prompt supports this facility.

Simple yes/no confirmations may require only `Y` or `N`, while paging prompts generally display their available controls directly.

When a Command Reports an Error

If a command rejects an option or file specification, consult its reference entry.

The entry describes the accepted syntax, supported options, restrictions, and important error conditions.

For problems involving the operating environment rather than a particular command, see Chapter 12, *Troubleshooting*.

Appendix F provides a reference to system messages and errors.

Chapter 14 - File and Directory Commands

BetterCP/M provides a collection of commands for listing, displaying, copying, renaming, moving, and deleting files. Most will be familiar to anyone who has used Digital Research CP/M, although BetterCP/M adds several useful capabilities.

This chapter is a reference to six of the most frequently used commands:

Command	Purpose
DIR	Display directory contents and locate files
ERA	Erase files
REN	Rename files
TYPE	Display text files
COPY	Copy files
MOVE	Move files

Several commands are available in both resident and transient forms. The resident versions provide convenient access to common operations, while their transient counterparts may offer more extensive options.

Where the two forms differ, the descriptions below identify those differences.

14.1 DIR - Display Directory

Purpose: Displays files in the current or a specified drive and user area.

DIR is available as both a resident command and a transient utility, `DIR.COM`.

Resident DIR

The resident command accepts:

```
DIR
DIR filespec
DIR DU:
DIR DU:filespec
```

For example:

```
A0>DIR
A0>DIR *.COM
A0>DIR B:
A0>DIR B3:*.ASM
```

Without arguments, DIR displays the files in the current drive and user area.

A file specification restricts the listing to matching files. Wildcards may be used to select groups of files.

BetterCP/M also supports extended drive/user selectors:

```
A0>DIR B[-]:*.COM
A0>DIR B[3-5]:*.ASM
A0>DIR [A0,B3]:*.DOC
```

Here, `B[-]:` selects all user areas on drive B, while `B[3-5]:` selects user areas 3 through 5.

The resident command displays each selected drive/user area separately and restores the original drive and user area when finished.

By default, system files are omitted.

Selecting Files by Attribute

DIR supports file-attribute expressions.

For example:

```
A0>DIR *.COM[$ARC]
A0>DIR *.DOC[$RO]
A0>DIR B[-]:*.COM[$ARC+!$SYS]
```

The available attribute names are:

Attribute	Meaning
`$RO`	Read-only
`$RW`	Writable
`$SYS`	System
`$DIR`	Not marked as a system file
`$ARC`	Archive

Attribute expressions use `!` for NOT, `+` for AND, and a comma for OR.

For example:

```
[$ARC+!$SYS]
```

selects files marked ARC but not SYS.

An explicit attribute expression replaces DIR's normal suppression of system files. A system file satisfying the expression can therefore appear in the listing.

Attribute selection does not change file attributes.

Transient DIR.COM

The transient DIR utility provides additional display and file-selection options.

Invoke it explicitly with:

```
A0>:DIR
```

or:

```
A0>.DIR
```

Both forms bypass the resident command.

The general syntax is:

```
DIR [options] [location:filespec[attribute-expression]]
```

The transient utility supports the same extended drive/user selectors and attribute expressions as resident DIR.

It also provides sorting, file sizes, attribute display, paging, and directory summaries.

DIR.COM Options

Option	Description
`/WHEREIS`	Locate matching files across drives and user areas; see below
`/A`	Display file attributes
`/P`	Pause after each screenful
`/Z` or `/Z=K`	Display allocated space in kilobytes
`/Z=S`	Display allocated 128-byte records
`/Z=E`	Display directory entries used
`/C=1`	One-column display
`/C=2`	Two-column display
`/C=4`	Four-column display
`/S=N`	Sort by filename
`/S=T`	Sort by file type
`/S=Z`	Sort by allocated size
`/S=U`	Preserve directory order

Sorting options `/S=N`, `/S=T`, and `/S=Z` may be followed by `+` for ascending or `-` for descending order.

For example:

```
A0>:DIR /S=N
A0>:DIR /S=Z-
A0>:DIR /S=T+ *.COM
```

The first sorts by filename, the second displays the largest files first, and the third sorts matching files by file type.

`/S=U` preserves the disk's directory-search order and does not accept a direction suffix.

Name-ascending order is the default.

Locating Files with `/WHEREIS`

Use transient DIR to find every location containing a matching filename:

```
A0>:DIR /WHEREIS ED.COM
```

`/WHEREIS` requires exactly one filespec. It searches directory names, not file contents, and does not change files or attributes. Wildcards and attribute expressions are supported.

A completely unqualified filespec searches all configured accessible drives and users 0 through 31. Configured drives may still have missing or unreadable media. An explicit drive/user selector restricts the search:

```
A0>:DIR /WHEREIS B:ED.COM
A0>:DIR /WHEREIS B[-]:ED.COM
A0>:DIR /WHEREIS [A0,B[3-5]]:*.COM[$SYS]
```

`B:` uses the user area that was current when the command started. `B[-]:` searches every user area on B. The last example searches only A0 and B users 3-5 for system `.COM` files.

Unlike ordinary DIR, `/WHEREIS` includes SYS files by default. An attribute expression can restrict the results; `[$DIR]`, for example, selects only files without SYS.

Each logical file appears once per location, even if it occupies several directory extents. Results use `DU:FILENAME` form, such as `B3:ED.COM`, followed by a match count. Drives and users are visited in ascending order, with native directory order within each location. Even an exact filename search reports all matches in the selected scope.

With no matches, DIR reports `DIR: No matches found.`. Recoverable unavailable-drive or directory-read errors are reported and the search continues elsewhere when safe. A skipped failure or directory-capacity limit adds `Search incomplete.`; a zero-match result in that case does not establish that the file is absent everywhere.

This mode has no ordinary size, attribute-column, free-space, sorting, column-layout, or paging display. Do not combine `/WHEREIS` with `/A`, `/P`, `/Z`, `/C=`, `/S=`, or `/VER`; these combinations are rejected. Attribute predicates in the filespec remain valid. No `/B`, `/Y`, or `/Q` option is needed for unattended searches.

Ctrl-C during console output follows the normal warm-boot cancellation behavior. On normal completion or recoverable errors, DIR preserves the caller's drive/user context; fatal system faults can end the search. Resident DIR does not implement this mode and reports `DIR: Use :DIR.`; use `:DIR` as shown above.

Displaying File Sizes

Transient DIR displays the allocated space occupied by each file.

The default unit is kilobytes.

To display allocated 128-byte records:

```
A0>:DIR /Z=S
```

To display the number of physical directory entries occupied by each file:

```
A0>:DIR /Z=E
```

These values describe allocated storage or directory usage, not necessarily the exact logical length of the file.

Displaying Attributes

Use `/A` to include a three-character attribute field:

```
A0>:DIR /A
```

The field contains the SYS, read-only, and ARC attributes, in that order.

For example:

Display	Meaning
`---`	No attributes set
`-R-`	Read-only
`S--`	System
`--A`	Archive
`S-A`	System and archive

The `/A` option displays attributes but does not select files based on them.

To select files by attribute, use an attribute expression.

Paging and Columns

For long listings, use:

```
A0>:DIR /P
```

DIR pauses after each screenful.

Press Space or Return to continue, or Ctrl-C to stop the listing.

The transient utility normally chooses a suitable column layout automatically.

You can request a particular number of columns:

```
A0>:DIR /C=1
```

A one-column display is particularly convenient when file sizes and attributes are shown.

If the requested number of columns cannot fit, DIR reports an error rather than silently changing the layout.

Directory Summaries

Transient DIR reports file counts and allocated-space totals.

When several user areas are selected, each is listed separately.

Free space is reported once per selected drive rather than repeated for every user area.

For example, selecting both B3 and B4 produces separate listings but only one free-space total for drive B.

Examples

Display all `.COM` files in user area 3 on drive B:

```
A0>:DIR B3:*.COM
```

Display archived files, excluding system files:

```
A0>:DIR *.*[$ARC+!$SYS]
```

Display the largest archived `.COM` files first, including their attributes:

```
A0>:DIR /A /S=Z- B3:*.COM[$ARC]
```

Display a paged, one-column listing:

```
A0>:DIR /C=1 /P
```

For ordinary directory listings, resident DIR is usually sufficient. Use transient DIR when you need sorting, sizes, paging, or more detailed information.

14.2 ERA - Erase Files

Purpose: Removes one or more files from disk.

ERA is available as a resident command and as the more capable transient utility `ERA.COM`.

Files erased by ERA may be unrecoverable. Check wildcard specifications and drive/user selections carefully before confirming an erase operation.

Resident ERA

The resident command accepts file specifications and BetterCP/M's extended drive/user selectors.

For example:

```
A0>ERA OLD.TXT
A0>ERA *.BAK
A0>ERA B3:*.TMP
A0>ERA B[-]:*.BAK
```

It also accepts attribute expressions:

```
A0>ERA B[-]:*.DOC[$ARC]
```

The complete selection is checked before any files are erased.

Resident ERA protects read-only and system files. These remain protected even when an attribute expression explicitly selects them.

When an all-wildcard specification is used, ERA requests confirmation:

```
ALL (Y/N)?
```

Only `Y` proceeds.

Resident ERA does not provide the extended confirmation, reporting, and destructive-erasure options available in the transient utility.

Transient ERA.COM

Invoke the transient utility explicitly:

```
A0>:ERA
```

Its general syntax is:

```
ERA [options] filespec[attributes]
```

For example:

```
A0>:ERA OLD.DOC
A0>:ERA *.BAK
A0>:ERA B[-]:*.DOC
```

Unlike the resident command, transient ERA normally displays the files selected for deletion and requests confirmation.

For a single file, it may ask:

```
TOM.DOC - Erase? Y/N
```

For multiple files, it displays a list of the selected files and a summary before asking whether to proceed.

ERA.COM Options

The documented erase controls are:

Option	Description
`/Y`	Assume yes for ordinary confirmations
`/Q`	Suppress ordinary informational output
`/C`	Confirm each file
`/R`	Authorize erasure of selected read-only files
`/D`	Destructive erase

These options affect how files are selected, confirmed, and removed.

Some options are intended for unattended operations, while others provide additional safeguards.

Confirmation

By default, transient ERA asks before deleting files.

For example:

```
A0>:ERA *.BAK
```

displays the selected files and requests confirmation.

To request confirmation for each file:

```
A0>:ERA /C *.BAK
```

ERA first obtains the ordinary group confirmation and then asks about individual files.

To suppress ordinary confirmation prompts:

```
A0>:ERA /Y *.BAK
```

This is useful in SUBMIT files, but should be used carefully.

Read-Only and System Files

Transient ERA handles protected files in three stages:

1. **Selection:** the filespec and attribute expression must match the file.
2. **Admission:** every protected attribute present (SYS and RO) must be explicitly admitted by the same OR branch that matches the file.
3. **Read-only authorization:** an admitted RO file still needs `/R` or an interactive read-only confirmation before erasure.

A comma separates OR branches; `+` joins conditions within one branch. Positive `$SYS` and `$RO` admit their respective protected attributes. `!$DIR` and `!$RW` are equivalent admissions; additional negations follow the normal Boolean rules.

Expression	Selection and protection admission
`[$SYS+$RO]`	Selects and admits both protected attributes in one matching branch
`[$SYS,$RO]`	Matches the file, but does not admit it: neither branch admits both attributes
`[$SYS+$RO,$ARC]`	Admits a SYS+RO file through the first branch; a SYS+ARC file without RO matches only the ARC branch and remains protected

For example, to select and authorize erasure of SYS+RO `.DOC` files:

```
A0>:ERA /R *.DOC[$SYS+$RO]
```

Without `/R`, an admitted read-only file receives a separate read-only confirmation. With `/Y` but without `/R`, ERA skips read-only files without asking. `/R` does not broaden selection or admission and does not admit SYS files by itself. `/Y` suppresses confirmations; it does not bypass these protection rules.

For read-only files without SYS, the familiar form remains:

```
A0>:ERA /R *.DOC[$RO]
```

A file that also carries SYS needs SYS admitted in the same matching branch, as shown above. Check the selection carefully before authorizing an erase.

Quiet Operation

Use `/Q` to suppress ordinary informational output:

```
A0>:ERA /Q *.BAK
```

This does not suppress required confirmation.

For unattended operation, combine it with `/Y`:

```
A0>:ERA /Y /Q *.BAK
```

Destructive Erase

The transient ERA utility provides a destructive-erasure option:

```
A0>:ERA /D SECRET.DAT
```

This attempts to overwrite the file's allocated data with zeroes before removing its directory information.

Destructive erase is intended to make recovery of the selected file's contents more difficult than ordinary deletion.

However, **it should not be treated as a guarantee of secure data destruction**.

Some allocation blocks may be shared with other live files. BetterCP/M preserves shared blocks rather than overwriting data belonging to another file.

Hardware characteristics and disk errors can also prevent complete overwriting.

Without `/Y`, destructive erase asks for confirmation even for one exact filename. For multiple files, it uses a destructive-erasure warning and confirmation; `/Q` hides the manifest but does not authorize erasure.

`/Y` suppresses the destructive confirmation as well as ordinary confirmations. Use it only after checking the selection carefully. It does not imply `/R`: selected read-only files are skipped with `/Y` unless `/R` is also supplied. Without `/Y` or `/R`, ERA asks for separate read-only authorization. Neither option broadens the selection or bypasses file-selection protection rules. ERA reports incomplete destructive operations.

Error Handling

If an erase operation fails, ERA may offer:

```
R/S/A/?
```

The responses are:

Response	Meaning
R	Retry
S	Skip the file
A	Abort the remaining operation
?	Display help

With `/Y`, ERA does not stop for this recovery prompt. It reports the failed file and continues when safe.

Completion Summary

Transient ERA reports the results of the operation.

For example:

```
12 FILES ERASED, 2 SKIPPED, 1 FAILED, 47K FREED
```

The reported space freed represents allocated disk space released, rather than the exact logical lengths of the deleted files.

Examples

Erase a single file:

```
A0>ERA OLD.TXT
```

Erase backup files in user area 3 on drive B:

```
A0>ERA B3:*.BAK
```

Erase archived files with individual confirmation:

```
A0>:ERA /C B[-]:*.DOC[$ARC]
```

Erase selected files without ordinary confirmation:

```
A0>:ERA /Y *.BAK
```

Destructively erase a file:

```
A0>:ERA /D SECRET.DAT
```

For ordinary file deletion, resident ERA is usually sufficient. Use transient ERA when you need its additional confirmation, selection, or destructive-erasure facilities.

14.3 REN - Rename Files

Purpose: Changes the names or file types of existing files without changing their contents.

REN is available as both a resident command and a transient utility, `REN.COM`.

Unlike COPY and MOVE, REN does not transfer files between drives or user areas. Each renamed file remains in its original location.

Resident REN

Resident REN accepts two equivalent forms:

```
REN source template
REN template=source
```

For example:

```
A0>REN NEW.TXT=OLD.TXT
A0>REN OLD.TXT NEW.TXT
```

Both rename `OLD.TXT` to `NEW.TXT`.

BetterCP/M also supports wildcard rename templates:

```
A0>REN *.DOC *.TXT
```

This changes the file type of matching `.DOC` files to `.TXT`, retaining their original names.

You can select files in another drive or user area:

```
A0>REN *.TXT=B3:*.DOC
```

The files remain in `B3:` after being renamed.

Renaming Several Files

REN supports BetterCP/M's extended drive/user selectors.

For example:

```
A0>REN B[-]:*.DOC *.TXT
```

renames matching files across all user areas on drive B.

You can also use attribute expressions:

```
A0>REN B[-]:*.DOC[$RW] *.TXT
```

This selects writable `.DOC` files.

Read-only and system files are protected by the resident command, even when explicitly selected.

Rename Templates

Rename templates determine how the new filename is constructed.

A literal character replaces the corresponding character in the source name. A question mark retains a character from the corresponding position, while a terminal asterisk retains the remainder of that filename field.

For example:

```
A0>REN X?*.BAK=B3:FOOBAR.DOC
```

produces `XOOBAR.BAK`.

The filename and file type are treated as separate fields.

A template cannot specify another drive or user area. Use COPY or MOVE when you want to change a file's location.

Existing Destination Names

REN never overwrites an existing file.

If the requested new name already exists, resident REN stops rather than replacing it.

Any files successfully renamed before the conflict remain renamed.

REN does not automatically undo earlier operations.

Transient REN.COM

The transient utility provides additional confirmation, preview, protection, and reporting options.

Invoke it explicitly:

```
A0>:REN
```

It accepts the same two operand arrangements:

```
REN source template
REN template=source
```

REN.COM Options

Option	Description
`/C`	Confirm each eligible rename
`/Y`	Proceed without ordinary confirmation
`/N`	Display proposed mappings without renaming
`/DRY`	Preview the operation without changing files
`/P`	Permit renaming selected protected files
`/Q`	Suppress routine successful-operation messages
`/QQ`	Suppress most normal output
`/VER`	Display version information

Previewing a Rename

When renaming several files, it's often useful to see what will happen before making changes.

Use `/DRY`:

```
A0>:REN /DRY B[-]:*.DOC *.TXT
```

REN displays the proposed filename changes without modifying the files.

A visible dry run ends with:

```
Dry run only: no files were renamed.
```

The `/DRY` option takes precedence over options that would otherwise authorize renaming.

This makes it particularly useful when experimenting with wildcard templates.

Confirming Renames

Use `/C` to request confirmation for each eligible file:

```
A0>:REN /C *.DOC *.TXT
```

Use `/Y` when the operation should proceed without ordinary confirmation:

```
A0>:REN /Y *.DOC *.TXT
```

Neither option overrides file protection.

Protected Files

By default, REN skips read-only and system files.

Transient REN provides `/P` to permit renaming explicitly selected protected files.

For example:

```
A0>:REN /P *.DOC[$RO] *.TXT
```

This selects read-only `.DOC` files and permits their renaming.

The files retain their attributes after the rename.

The `/P` option does not broaden the file selection or override a read-only drive.

Name Conflicts

If a destination name already exists, transient REN can offer:

```
File exists. N/S/A/?
```

The responses are:

Response	Meaning
N	Enter a new name
S	Skip this file
A	Abort the remaining operation
?	Display help

A replacement name must be a valid filename without a drive/user qualification.

REN never silently overwrites an existing target.

Errors and Completion

If an operation fails, transient REN may offer Retry, Skip, or Abort.

Successful renames are not rolled back when a later operation fails.

On completion, REN reports the results, for example:

```
12 FILES RENAMED, 2 SKIPPED, 1 FAILED
```

Examples

Rename one file:

```
A0>REN NEWNAME.TXT=OLDNAME.TXT
```

Change all `.DOC` files to `.TXT`:

```
A0>REN *.DOC *.TXT
```

Preview renaming across all user areas on drive B:

```
A0>:REN /DRY B[-]:*.DOC *.TXT
```

Rename selected read-only files:

```
A0>:REN /P *.DOC[$RO] *.TXT
```

For ordinary renaming, resident REN is sufficient. Use transient REN when you need confirmation controls, previews, or protected-file handling.

14.4 TYPE - Display Text Files

Purpose: Displays the contents of text files on the console.

TYPE is available as a resident command and as the more capable transient utility `TYPE.COM`.

Resident TYPE

The basic form is:

```
TYPE filespec
```

For example:

```
A0>TYPE README.TXT
```

TYPE displays the contents of `README.TXT`.

BetterCP/M also supports extended file selection:

```
A0>TYPE B3:README.TXT
A0>TYPE B[-]:*.TXT
A0>TYPE [A0,B3]:*.DOC
```

When several files are selected, TYPE displays them in sequence.

TYPE treats the conventional CP/M end-of-text character, Ctrl-Z (`1AH`), as the end of the text.

It does not modify the files being displayed.

Resident Paging

Resident TYPE supports forward paging with `/P`:

```
A0>TYPE README.TXT /P
```

The display pauses after a screenful of text.

Press Space or `.` to advance.

Ctrl-C cancels the resident paged display.

The resident command provides simple forward paging rather than the more extensive navigation facilities of `TYPE.COM`.

Transient TYPE.COM

Invoke the transient utility explicitly:

```
A0>:TYPE README.TXT
```

The transient version provides bidirectional paging, navigation between files, and several display options.

TYPE.COM Options

Option	Description
`/P`	Enable interactive paging and file navigation
`/7`	Strip the high bit from displayed characters
`/T`	Expand tabs to eight-column stops
`/CTL`	Display control characters using caret notation
`/VER`	Display version information

Options may appear before or after the file specification.

For example:

```
A0>:TYPE README.TXT /P
A0>:TYPE README.TXT /7
A0>:TYPE README.TXT /T
A0>:TYPE README.TXT /CTL
```

Paging and Navigation

With `/P`, transient TYPE provides the following controls:

Key	Action
Ctrl-R or ,	Previous page
Ctrl-C, . or Space	Next page
<	Beginning of current file, or previous file
>	Beginning of next file
Ctrl-Q	Quit

The meaning of Ctrl-C here is specific to TYPE.COM's paging prompt. During ordinary output, normal CP/M console handling applies.

At the end of the final file, TYPE leaves the last page displayed until you quit.

You can move backward through earlier pages without a fixed page-history limit, although revisiting distant pages may be slow on floppy disks.

Displaying Control Characters

The `/CTL` option displays control characters in a readable form.

For example, a tab is displayed as `^I`, while a form feed appears as `^L`.

This can be useful when examining text files containing unexpected control characters.

The `/T` option expands tabs into spaces for display. When `/CTL` is used, its control-character display takes precedence over ordinary tab expansion.

Displaying Several Files

TYPE.COM can display files selected from several drive/user areas:

```
A0>:TYPE B[-]:*.DOC /P
```

When several files are selected, the utility identifies each file as it is displayed.

You can use < and > to move between file boundaries.

This makes TYPE.COM useful for examining a collection of text files without returning to the command prompt after each one.

Limitations

TYPE is intended for ordinary text files.

It does not provide binary inspection, decompression, text searching, or editing.

For those operations, use an appropriate specialized utility.

Examples

Display a file:

```
A0>TYPE README.TXT
```

Display a file with simple paging:

```
A0>TYPE README.TXT /P
```

Use the transient viewer with backward paging:

```
A0>:TYPE README.TXT /P
```

Display control characters:

```
A0>:TYPE README.TXT /CTL
```

Display all `.DOC` files across drive B's user areas:

```
A0>:TYPE B[-]:*.DOC /P
```

14.5 COPY - Copy Files

Purpose: Copies one or more files to another filename, drive, or user area.

COPY is available as both a resident command and a transient utility, `COPY.COM`.

The resident version provides the file-copying facilities needed for ordinary use, while the transient utility offers additional capabilities, including wildcard destination templates, file verification, attribute management, and backup operations.

COPY leaves the original files in place. If you want to transfer files to another location and remove the originals, use MOVE instead.

Resident COPY

Resident COPY accepts two equivalent command forms:

```
COPY source destination
COPY destination=source
```

For example:

```
A0>COPY README.TXT README.BAK
A0>COPY README.BAK=README.TXT
```

Both commands create `README.BAK` from `README.TXT`.

You can also copy files between drives:

```
A0>COPY README.TXT B:
```

This copies `README.TXT` to drive B, retaining its original filename.

To copy a file into another user area:

```
A0>COPY A3:REPORT.TXT B5:
```

The destination file is created on drive B, user area 5.

Copying Several Files

COPY accepts wildcard file specifications.

For example:

```
A0>COPY *.ASM B:
```

copies the matching assembly-language source files to drive B.

BetterCP/M also supports extended drive/user selectors:

```
A0>COPY B[-]:*.DOC C3:
```

This selects `.DOC` files across all user areas on drive B and copies them to drive C, user area 3.

You can combine several locations:

```
A0>COPY [A0,B[5-7],C3]:*.COM D3:
```

The destination must identify a single drive/user area.

When several source files would produce the same destination filename, COPY rejects the conflicting selection rather than silently replacing one file with another.

Selecting Files by Attribute

Resident COPY supports file-attribute expressions.

For example:

```
A0>COPY B[-]:*.DOC[$RW+!$SYS] C4:
```

This selects writable, non-system `.DOC` files from drive B's user areas.

Attribute expressions use the same conventions described in Chapter 4 and Section 14.1.

They determine which files are selected; they do not automatically change the attributes of the source files.

Existing Destination Files

Resident COPY normally refuses to replace an existing destination file.

For example, if `B:README.TXT` already exists, an ordinary copy to that name will not silently overwrite it.

Resident COPY provides `/O` to permit replacement of existing writable destination files:

```
A0>COPY README.TXT B: /O
```

Read-only destination files remain protected.

The option does not permit a copy operation to overwrite one of its own selected source files.

COPY cannot roll back a replacement: after the old writable target is deleted, a later failure may leave neither the old target nor a complete replacement. Preserve a separate copy of an important existing destination before using `/O`. COPY preserves the source file; it does not restore the old destination.

Copying to a New Filename

When copying a single file, you can specify a different destination name:

```
A0>COPY REPORT.TXT B:REPORT.BAK
```

This creates `REPORT.BAK` on drive B.

For multiple-file selections, resident COPY retains the source filenames when the destination specifies only a drive/user area.

Wildcard destination templates are provided by the transient COPY utility.

Transient COPY.COM

The transient utility provides additional file-copying capabilities.

Invoke it explicitly:

```
A0>:COPY
```

or:

```
A0>.COPY
```

The general command forms remain:

```
COPY source destination
COPY destination=source
```

However, COPY.COM supports additional source-selection, destination-naming, and file-management options.

COPY.COM Options

Option	Description
`/O`	Replace existing writable destination files
`/S`	Skip existing writable destination files
`/B`	Operate without file-collision prompts
`/V`	Verify copied files by reading and comparing their contents
`/BACKUP`	Clear the archive attribute after successful backup copying

The `/O` and `/S` options are mutually exclusive.

The `/B` and `/V` options can be combined with either.

Overwriting and Skipping Files

By default, transient COPY requests instructions when an existing writable destination file would be replaced.

The available responses include:

Response	Meaning
Y	Replace this file
N	Skip this file
O	Replace this and subsequent writable collisions
S	Skip this and subsequent writable collisions
R	Supply a new destination filename
?	Display help

The O and S responses apply only to the current COPY operation.

They do not permanently change COPY's behavior.

Read-only destination files remain protected.

Renaming a Destination During Copying

If an existing destination file would be overwritten, the R response allows you to supply a different filename.

For example, suppose you're copying `REPORT.TXT` to drive B, but a file with that name already exists.

At the collision prompt, enter `R`.

COPY asks for a new name:

```
New Name:
```

You might enter:

```
REPORT2.TXT
```

COPY then uses that filename in the same destination drive/user area.

The replacement name must be an exact CP/M filename, without wildcards or a drive/user qualification.

If the new name also exists, COPY applies its normal collision handling to that name.

Batch-Oriented Copying

The `/B` option is intended for operations that should not stop for file-collision prompts.

For example:

```
A0>:COPY B1:*.DAT C3: /B
```

If a writable destination already exists, COPY reports `FILE EXISTS` for that file and continues with the remaining files.

It does not automatically overwrite or skip the collision as a successful operation.

To specify the desired policy, combine `/B` with `/O` or `/S`.

To overwrite existing writable destinations:

```
A0>:COPY B1:*.DAT C3: /B /O
```

To skip existing destinations:

```
A0>:COPY B1:*.DAT C3: /B /S
```

Read-only destinations remain protected in both cases.

With `/O`, replacing a writable target can delete the old file before the new copy is complete. A later failure or abort does not restore it. `/V` detects verification failures but provides no rollback; preserve important existing destinations separately first.

The `/B` option is useful in SUBMIT files, but it applies only to COPY. It does not make other commands in the batch noninteractive.

Verifying Copied Files

Use `/V` to request verification:

```
A0>:COPY REPORT.TXT B: /V
```

After copying the file, COPY reopens the source and destination and compares their contents.

A successful verification provides additional confidence that the destination contains the same data as the source.

If verification fails, COPY reports the problem and provides the appropriate recovery choices.

With `/B`, a failed copy or verification does not stop for an interactive retry prompt.

An incomplete destination is removed when possible, and COPY continues according to its error-handling rules.

For important backups, `/V` is recommended.

Wildcard Destination Templates

Transient COPY supports wildcard templates in destination filenames.

For example:

```
A0>:COPY C3:*.DOC=A4:*.COM
```

This copies matching `.COM` files from A4 to C3, changing their file type to `.DOC`.

A more selective template might be:

```
A0>:COPY C3:X?*.BAK=A4:FOOBAR.COM
```

This creates `XOOBAR.BAK`.

The destination template uses literal characters to replace source positions, question marks to retain corresponding characters, and a terminal asterisk to retain the remainder of a filename field.

The command validates the resulting destination names before copying.

Copying Files from Several User Areas

Transient COPY supports extended source selectors.

For example:

```
A0>:COPY [A[-],B[-],C[-]]:*.COM D0:
```

This selects `.COM` files from all user areas on drives A, B, and C and copies them to D0.

The destination must still identify one drive/user area.

COPY checks the combined selection before beginning.

If different source files would produce the same destination filename, the operation is rejected.

Transient COPY.COM supports a maximum of 64 matched source files in a single invocation.

Larger selections must be divided into smaller operations.

If a selection exceeds the limit, COPY reports `COPY BATCH TOO LARGE` and does not begin writing.

Source files can also be selected by attribute. For example:

```
A0>:COPY B[-]:*.COM[$ARC] D0:
```

This selects archive-marked `.COM` files using the attribute-expression conventions described under DIR.

Destination Attributes

Transient COPY allows destination attributes to be specified explicitly.

For example:

```
A0>:COPY A0:*.COM D0:[$RW,!$ARC]
```

This requests writable destination copies with the archive attribute cleared.

Attributes not explicitly changed are preserved.

The destination attribute expression is different from a source-selection expression: it specifies the attributes to assign to completed destination files.

It does not override protection on an existing read-only destination.

Backup Operations

The `/BACKUP` option supports workflows using CP/M's archive attribute.

For example:

```
A0>:COPY /V /BACKUP B[-]:*.DOC[$ARC] D0:
```

This selects `.DOC` files marked with the archive attribute, copies them to D0, verifies the copies, and clears the archive attribute on both source and destination after successful completion.

Files that are skipped, fail to copy, or are interrupted retain their source archive status.

The `/BACKUP` option does not automatically select archive-marked files. The `[$ARC]` expression performs that selection.

Similarly, `/BACKUP` does not automatically enable verification. Include `/V` when verification is desired.

The archive attribute should not be assumed to identify every file changed since the last backup unless the software maintaining those files updates the attribute appropriately.

Copy Errors and Recovery

COPY reports errors such as missing source files, unavailable destinations, insufficient disk space, read/write failures, and verification failures.

The transient utility attempts to remove incomplete destination files when an operation fails.

That cleanup does not restore an overwritten file. If replacement had already started, failure may leave neither the old target nor a complete replacement, even with `/V`. The source remains available, but an important old destination needs its own separate backup before `/O`.

If that cleanup cannot be completed, COPY reports the problem and stops rather than leaving an incomplete file without warning.

Previously completed copies remain available.

Completion Information

Transient COPY reports successful copies and their allocated sizes.

For example:

```
B1:FOO.COM -> D0:FOO.COM [2K]
```

For multiple files, it also reports totals for copied, skipped, and failed files.

Examples

Copy a file:

```
A0>COPY README.TXT B:
```

Copy a file under another name:

```
A0>COPY REPORT.TXT B:REPORT.BAK
```

Copy all assembly-language source files:

```
A0>COPY *.ASM B:
```

Copy files across several user areas:

```
A0>COPY B[-]:*.DOC C3:
```

Copy and verify:

```
A0>:COPY /V *.COM B:
```

Copy without overwrite prompts, replacing existing writable files:

```
A0>:COPY /B /O *.DAT B:
```

Back up archive-marked files:

```
A0>:COPY /V /BACKUP B[-]:*.DOC[$ARC] D0:
```

14.6 MOVE - Move Files

Purpose: Moves one or more files to another filename, drive, or user area.

MOVE is available as a resident command in BetterCP/M 1.0.

It provides a convenient way to reorganize files without separately copying them and deleting the originals.

Unlike COPY, MOVE removes the source file after successfully creating the destination.

Command Syntax

MOVE accepts two equivalent forms:

```
MOVE source destination
MOVE destination=source
```

For example:

```
A0>MOVE README.TXT B:
```

moves `README.TXT` from the current drive and user area to drive B.

You can also write:

```
A0>MOVE B:README.TXT=README.TXT
```

Both commands have the same effect.

Moving Files Between User Areas

BetterCP/M allows files to be moved between user areas.

For example:

```
A0>MOVE A3:REPORT.TXT B5:
```

moves `REPORT.TXT` from A3 to B5.

The original file is removed only after the destination has been successfully created.

This is particularly useful when reorganizing project files among user areas.

Moving Several Files

MOVE accepts wildcard specifications:

```
A0>MOVE *.BAK B:
```

This moves matching backup files to drive B.

Extended drive/user selectors are also supported:

```
A0>MOVE B[-]:*.DOC C3:
```

This selects `.DOC` files across all user areas on drive B and moves them to C3.

The destination must identify a single drive/user area.

If multiple source files would produce the same destination filename, MOVE rejects the conflicting selection.

Selecting Files by Attribute

MOVE supports BetterCP/M's shared file-selection expressions.

For example:

```
A0>MOVE B[-]:*.DOC[$RW+!$SYS] C4:
```

This selects writable, non-system `.DOC` files from drive B's user areas.

Read-only source files are protected.

Attribute expressions determine which files are selected; they do not automatically override file protection.

Moving a File to a New Name

MOVE can assign a new filename when moving a single file.

For example:

```
A0>MOVE REPORT.TXT B:REPORT.BAK
```

This moves the file to drive B and renames it `REPORT.BAK`.

The original `REPORT.TXT` is removed after the destination has been successfully written.

Existing Destination Files

MOVE does not overwrite existing destination files.

If a destination name already exists, MOVE reports the conflict rather than replacing the file.

Unlike transient COPY, resident MOVE does not provide an overwrite option.

If you need to replace an existing destination, handle that file separately before moving the new one.

Be careful when removing existing files, particularly when working with wildcard specifications.

How MOVE Handles Files

MOVE copies the file to its destination and removes the original only after the destination has been successfully written and closed.

This applies even when moving a file within the same drive/user area.

If the destination cannot be created or written successfully, MOVE does not intentionally erase the source.

However, an unexpected failure can interrupt a multi-file move after some files have already been transferred.

Earlier completed moves are not automatically reversed.

Source Deletion Failures

Occasionally, MOVE may successfully create the destination but be unable to remove the original.

In that case, it reports:

```
COPY MADE; SOURCE NOT ERASED
```

This means that the destination copy was created, but the source remains in place.

You should examine both files before deciding whether to delete the original manually.

Moving Several Files Safely

When moving a large group of files, it's wise to check the source and destination directories first.

For example:

```
A0>DIR B3:*.DOC
```

```
A0>DIR C4:*.DOC
```

This allows you to identify possible filename conflicts before moving the files.

You can then enter:

```
A0>MOVE B3:*.DOC C4:
```

Afterward, examine the destination:

```
A0>DIR C4:*.DOC
```

If MOVE reported an error, also examine the source to determine which files remain.

COPY or MOVE?

COPY and MOVE serve different purposes.

Use **COPY** when you want to create another copy of a file while retaining the original.

Use **MOVE** when you want to relocate a file and no longer need it in its original location.

For important files, copying and verifying them before manually deleting the originals may provide additional reassurance.

MOVE is convenient for routine file organization, but it should not be treated as a substitute for maintaining backups.

Examples

Move a file to another drive:

```
A0>MOVE README.TXT B:
```

Move a file to another user area:

```
A0>MOVE A3:REPORT.TXT B5:
```

Move a file and change its name:

```
A0>MOVE REPORT.TXT B:REPORT.BAK
```

Move all backup files:

```
A0>MOVE *.BAK B:
```

Move selected files from several user areas:

```
A0>MOVE B[-]:*.DOC C3:
```

Move writable, non-system files:

```
A0>MOVE B[-]:*.DOC[$RW+!$SYS] C4:
```

Chapter 15 - System and Environment Commands

This chapter describes the commands used to manage the BetterCP/M operating environment, examine system information, and work directly with memory.

Several of these commands will be familiar to users of Digital Research CP/M 2.2. Others are BetterCP/M extensions that provide convenient access to facilities traditionally associated with debugging and system maintenance.

The commands covered are:

Command	Purpose
USER	Select the current user area
CLS	Clear the console screen
VER	Display system version information
SAVE	Save memory to a file
GET	Load a file into memory
GO	Execute a program at address 0100H
JUMP	Transfer control to a specified address
PEEK / P	Examine memory
POKE	Modify memory

The memory-oriented commands are particularly useful when examining programs, testing small machine-language routines, or working with program images.

Because these commands can access memory directly, some require more care than ordinary file-management commands. In particular, executing arbitrary memory addresses or modifying operating-system memory can cause a program to fail or make the current session unusable.

15.1 USER - Select User Area

Purpose: Changes the current user area on the selected drive.

BetterCP/M retains the conventional CP/M USER command while extending the available range of user areas.

Syntax

```
USER number
```

The number specifies a user area from 0 through 31.

For example:

```
A0>USER 3
A3>
```

This selects user area 3 on drive A.

To select user area 17:

```
A3>USER 17
A17>
```

The prompt changes to reflect the new user area.

Direct User-Area Selection

BetterCP/M also allows you to select a user area directly by entering its number followed by a colon.

For example:

```
A0>3:
A3>
```

This is equivalent to:

```
A0>USER 3
```

You can also select a drive and user area together:

```
A3>B17:
B17>
```

This selects drive B, user area 17.

User Areas and File Access

Changing the current user area affects commands that do not explicitly specify another location.

For example, after selecting user area 3:

```
A3>DIR
```

displays files in user area 3 on drive A.

To examine another user area without changing the current selection:

```
A3>DIR B5:
```

The current drive and user area remain unchanged after the command.

Compatibility

The USER command is retained for compatibility with conventional CP/M usage.

BetterCP/M supports user areas 0 through 31, extending the traditional CP/M 2.2 range of 0 through 15.

USER is a resident command. There is no separate transient `USER.COM`.

For more information about drives and user areas, see Chapter 4.

15.2 CLS - Clear Screen

Purpose: Clears the console display and returns the cursor to the upper-left corner.

Syntax

```
CLS
```

Example

```
A0>CLS
```

CLS clears the current display.

This is useful when the screen has become cluttered with directory listings, program output, or command messages.

Resident and Transient Versions

CLS is normally supplied as a resident command by `RCP.CPX`.

A transient `CLS.COM` can be built from the matching source, but is not selected System Disk content. Resident CLS through `RCP.CPX` is the standard supplied form. The following invocation requires a separately installed transient:

To invoke the transient version explicitly:

```
A0>:CLS
```

Terminal Considerations

The exact screen-clearing operation depends on the active console environment.

On systems using BetterCP/M's terminal-monitor facilities, the operation is performed through the appropriate display support.

The visible result is normally a cleared screen with the cursor positioned at the beginning of the display.

The underlying method may differ between supported computers and terminal environments.

For more information about terminal support, see Chapter 9.

15.3 VER - Display Version Information

Purpose: Displays information identifying the running BetterCP/M system.

VER is useful when checking the operating-system release, identifying the installed software, or reporting a problem.

Syntax

```
VER
VER /V
VER /BANNER
```

The available forms provide different levels of information.

Displaying the System Version

Enter:

```
A0>VER
```

VER displays the BetterCP/M system identification and release version.

This is the simplest way to determine which version of BetterCP/M is running.

Detailed Version Information

To display additional subsystem information:

```
A0>VER /V
```

The `/V` option provides more detailed identification information about the operating-system environment.

This can be useful when comparing installations or determining whether a particular subsystem has been updated.

Displaying the Startup Banner

The transient VER utility provides the system identification banner used by the standard distribution's startup procedure.

Enter:

```
A0>:VER /BANNER
```

The `/BANNER` option is provided by `VER.COM`.

If you attempt to use it with the resident command:

```
A0>VER /BANNER
```

BetterCP/M reports:

```
VER: Use :VER.
```

You can then invoke the transient utility explicitly.

Resident and Transient Versions

VER is normally available as a resident command through `RCP.CPX`.

The transient `VER.COM` provides additional facilities, including the startup banner.

As with other commands available in both forms, prefixing the command name with a colon or period requests the transient version.

Version Information for Individual Utilities

Most BetterCP/M utilities also support `/VER` to display their own version and build information.

For example:

```
A0>:DIR /VER
```

identifies the transient DIR utility.

This is different from the VER command, which identifies the operating system itself.

For additional information about utility version conventions, see Chapter 13.

15.4 SAVE - Save Memory to a File

Purpose: Saves a specified number of 256-byte memory pages to a disk file.

SAVE is a core resident command retained from conventional CP/M.

It is particularly useful when working with programs or data that have been loaded or modified directly in memory.

Syntax

```
SAVE pages filespec
```

The `pages` argument specifies the number of 256-byte pages to save.

The memory is saved beginning at address `0100H`, the conventional start of the CP/M Transient Program Area.

Examples

Save ten pages:

```
A0>SAVE 10 TEST.COM
```

This writes 2,560 bytes from memory beginning at `0100H` into `TEST.COM`.

Save forty pages to drive B:

```
A0>SAVE 40 B:PROGRAM.COM
```

Save twenty pages to drive B, user area 3:

```
A0>SAVE 20 B3:IMAGE.COM
```

BetterCP/M extends the conventional SAVE command by allowing drive/user-qualified destination file specifications.

Calculating the Number of Pages

Each page contains 256 bytes.

To determine the number of pages required, divide the number of bytes by 256 and round upward.

For example:

Bytes to save	Pages
256	1
1,024	4
2,560	10
4,096	16
10,240	40

If the data does not occupy an exact number of pages, the final saved page includes any remaining bytes within that page.

Using SAVE with GET and POKE

SAVE can be used with BetterCP/M's memory commands.

For example, you might load a program with GET, modify several bytes with POKE, and then save the modified image under another filename.

```
A0>GET PROGRAM.COM
A0>POKE 100 C3 00 80
A0>SAVE 10 MODIFIED.COM
```

This example illustrates the general procedure. The number of pages saved must be appropriate to the size of the program or data.

SAVE does not determine the logical size of the program automatically.

Resident Command

SAVE is implemented directly by the command processor.

It is not provided as an ordinary transient utility because loading a transient `SAVE.COM` could overwrite the very memory that SAVE is intended to preserve.

Important Considerations

SAVE writes the contents of memory as they exist when the command executes.

It does not determine whether those contents form a valid executable program.

If you save a memory region containing incomplete or unrelated data, the resulting file will contain that data.

Likewise, saving too few pages truncates the intended image, while saving too many pages includes additional memory contents.

15.5 GET - Load a File into Memory

Purpose: Loads a file into memory without executing it.

GET is useful when examining or modifying program images, preparing machine-language routines for execution, or loading data into a particular memory location.

Unlike ordinary program execution, GET returns to the command prompt after loading the file.

Syntax

```
GET filename
GET address filename
```

With one argument, GET loads the file beginning at address `0100H`.

With two arguments, the first specifies the hexadecimal memory address at which loading begins.

Examples

Load a program at `0100H`:

```
A0>GET PROGRAM.COM
```

Load a file at address `8000H`:

```
A0>GET 8000 DATA.BIN
```

The address is hexadecimal.

Leading zeroes are optional.

Examining Loaded Data

After loading a file, you can examine its contents with PEEK.

For example:

```
A0>GET PROGRAM.COM
A0>PEEK 100
```

This displays memory beginning at address `0100H`.

You can also modify the loaded data with POKE.

Executing a Loaded Program

A program loaded at `0100H` can subsequently be executed with GO:

```
A0>GET PROGRAM.COM
A0>GO
```

Alternatively, JUMP can transfer control to a specified address.

Memory Protection

GET checks whether the requested load would overwrite memory required by the active BetterCP/M command environment.

If the file cannot be loaded safely, GET rejects the operation.

This protection helps prevent accidental damage to resident operating-system components.

Nevertheless, GET is intended for users who understand the memory requirements of the programs or data being loaded.

15.6 GO - Execute at 0100H

Purpose: Transfers control to address `0100H`, the conventional entry point for CP/M transient programs.

GO is useful when executing a program image that has already been loaded into memory.

Syntax

```
GO [command tail]
```

The command tail is optional.

Examples

Execute the program currently loaded at `0100H`:

```
A0>GO
```

Execute it with a command-line argument:

```
A0>GO INPUT.DAT
```

The command tail is supplied to the program using the normal CP/M transient-program conventions.

Using GO with GET

A common sequence is:

```
A0>GET PROGRAM.COM
A0>GO
```

GET loads the program into memory.

GO then transfers control to it.

This allows a program to be loaded, examined, or modified before execution.

Relationship to JUMP

GO is equivalent to:

```
JUMP 0100
```

Both transfer control to address `0100H`.

GO is provided as a convenient shorthand for the most common CP/M program entry point.

Program Environment

Before transferring control, BetterCP/M establishes the normal CP/M transient-program environment.

This includes the expected command tail, default File Control Blocks, and other program-entry conditions.

Ordinary CP/M programs therefore receive the familiar execution environment.

Caution

GO does not verify that memory at `0100H` contains a valid program.

If the memory contains unrelated data or an incomplete program image, execution may fail.

Use GO only when you know that an appropriate program has been loaded.

15.7 JUMP - Execute at an Address

Purpose: Transfers control to a specified memory address.

JUMP extends the functionality of GO by allowing execution to begin at an address other than `0100H`.

Syntax

```
JUMP address [command tail]
```

The address is hexadecimal.

Examples

Execute at `0100H`:

```
A0>JUMP 100
```

Execute at `8000H`:

```
A0>JUMP 8000
```

Execute at `0100H` with an argument:

```
A0>JUMP 100 INPUT.DAT
```

Using JUMP with GET

JUMP can be used to execute a routine loaded at a nonstandard address.

For example:

```
A0>GET 8000 ROUTINE.BIN
A0>JUMP 8000
```

This loads `ROUTINE.BIN` at address `8000H` and transfers control to that address.

The routine must be suitable for execution at the selected address.

Program Environment

Like GO, JUMP establishes the normal CP/M transient-program environment before transferring control.

This includes the standard command-tail and default FCB arrangements.

Address Range

JUMP accepts hexadecimal addresses from `0000H` through `FFFFH`.

However, **not every address is a valid or safe execution address**.

Some locations contain operating-system code, data, interrupt vectors, or other system structures.

Transferring control to an arbitrary address can cause the system to malfunction.

JUMP should therefore be used only when the contents and purpose of the target address are known.

Error Handling

If the address is missing or invalid, JUMP reports an error and returns to the command prompt.

It does not attempt to execute an invalid address specification.

15.8 PEEK / P - Display Memory

Purpose: Displays the contents of memory in hexadecimal and ASCII form.

PEEK provides a convenient way to examine memory without modifying its contents. It is particularly useful when inspecting loaded programs, examining data structures, or checking the results of changes made with POKE.

The command can also be abbreviated to `P`.

Syntax

```
PEEK
PEEK address
PEEK first last
```

Or, using the abbreviated form:

```
P
P address
P first last
```

All addresses are hexadecimal.

Displaying Memory

To display memory beginning at address `0100H`, enter:

```
A0>PEEK 100
```

PEEK displays 256 bytes beginning at the specified address.

The display includes hexadecimal byte values and their corresponding ASCII representations, making it possible to examine both machine-language instructions and ordinary character data.

For example:

```
A0>P 8000
```

displays 256 bytes beginning at address `8000H`.

Displaying an Address Range

You can specify both the first and last addresses to display.

For example:

```
A0>PEEK 100 1FF
```

This displays the inclusive range from `0100H` through `01FFH`.

The first address identifies the beginning of the range, while the second identifies its final byte.

PEEK does not wrap around from `FFFFH` to `0000H`.

Continuing a Memory Display

If you enter PEEK without an address:

```
A0>PEEK
```

the first invocation begins at `0100H`.

Subsequent invocations continue from the byte following the previous display.

This allows you to examine consecutive regions of memory without repeatedly calculating addresses.

For example:

```
A0>PEEK
A0>PEEK
A0>PEEK
```

displays successive portions of memory.

Paging Through Memory

PEEK supports interactive paging when displaying memory.

The following keys control navigation:

Key	Action
Space	Display the next page
.	Display the next page

Key	Action
>	Display the next page
,	Display the previous page
<	Display the previous page
Ctrl-C	Return to the command prompt

These controls allow you to move forward and backward through the displayed memory.

Using PEEK with GET

PEEK is especially useful after loading a program or data file with GET.

For example:

```
A0>GET PROGRAM.COM
A0>PEEK 100
```

GET loads `PROGRAM.COM` into memory beginning at `0100H`.

PEEK then displays the first 256 bytes of the loaded image.

You can examine additional portions by specifying other addresses or continuing the display.

Using PEEK with POKE

PEEK can also be used to verify memory changes.

For example:

```
A0>POKE 200 48 45 4C 4C 4F
A0>PEEK 200
```

The POKE command stores five bytes beginning at address `0200H`.

PEEK then displays the corresponding memory region.

The hexadecimal values represent the ASCII characters `HELLO`.

Using PEEK with GET and POKE

PEEK is particularly useful alongside the other resident memory commands.

For example:

```
A0>GET PROGRAM.COM
A0>PEEK 100
A0>POKE 100 C3 00 80
A0>PEEK 100
```

This loads a program, examines its beginning, modifies three bytes, and displays the modified memory.

PEEK itself does not change memory.

Important Considerations

PEEK displays the contents of memory as they exist when the command executes.

It does not interpret the contents as program instructions or determine whether the data is meaningful.

Memory may contain executable code, program data, operating-system structures, or unused values.

PEEK itself does not modify memory, but the contents of some memory locations may change as programs or the operating system execute.

For disassembly and more advanced debugging operations, use an appropriate debugging utility such as DDT.

15.9 POKE - Modify Memory

Purpose: Stores hexadecimal byte values or literal text directly into memory.

POKE provides a convenient way to modify memory without loading a separate debugger.

It can be used to patch a program image, enter small machine-language routines, or modify data that has been loaded with GET.

Unlike PEEK, **POKE changes memory contents**. Incorrect modifications can damage a loaded program or interfere with the operating system.

Syntax

```
POKE address byte [byte ...]
POKE address "text
POKE address byte [byte ...] "text
```

The address and hexadecimal byte values are required in their respective forms.

The address specifies where writing begins.

Successive bytes are stored in consecutive memory locations.

Storing Hexadecimal Bytes

For example:

```
A0>POKE 100 C3 00 80
```

stores three bytes beginning at address `0100H`:

Address	Value
0100H	C3H
0101H	00H
0102H	80H

These bytes form a Z80 absolute jump instruction to address `8000H`.

POKE stores the values exactly as specified. It does not interpret them as instructions while writing them.

Storing Text

POKE can also store literal text.

For example:

```
A0>POKE 200 "Hello
```

stores the ASCII characters `Hello` beginning at address `0200H`.

A quotation mark introduces literal text. All remaining characters on the command line are stored as text.

A closing quotation mark is not required.

Combining Bytes and Text

You can combine hexadecimal byte values with literal text.

For example:

```
A0>POKE 300 0D 0A "BetterCP/M
```

This stores a carriage return (`0DH`) and line feed (`0AH`), followed by the ASCII characters in `BetterCP/M`.

This is useful when constructing small data structures or preparing text containing control characters.

Examining the Results

Use PEEK to examine the memory after making a change.

For example:

```
A0>POKE 200 "Hello
A0>PEEK 200
```

PEEK displays the modified memory, including the hexadecimal values and ASCII representation.

Modifying a Loaded Program

POKE can be used with GET and SAVE to modify a program image.

For example:

```
A0>GET PROGRAM.COM
A0>POKE 120 00
A0>PEEK 120
A0>SAVE 10 MODIFIED.COM
```

This sequence:

1. Loads `PROGRAM.COM` into memory.
2. Changes the byte at address `0120H` to `00H`.
3. Displays the modified memory.
4. Saves ten 256-byte pages to `MODIFIED.COM`.

The example illustrates the procedure rather than a useful program modification. The address, replacement value, and number of pages must be appropriate to the actual program.

Input Validation

POKE checks the address and hexadecimal byte operands before modifying memory.

If an operand is invalid, the command rejects the operation without making a partial modification.

For example, an invalid hexadecimal byte does not cause POKE to write the preceding valid bytes and then stop.

This helps prevent accidental partial changes caused by mistyped commands.

However, POKE cannot determine whether a valid address and byte sequence are appropriate for the program being modified.

Memory Safety

POKE is intended for users who understand the memory locations they are changing.

Writing to the wrong address can corrupt program data, overwrite executable instructions, or interfere with operating-system structures.

A valid hexadecimal address is not necessarily a safe address to modify.

If you're experimenting with machine-language programs, work with known memory locations and keep copies of the original program files.

When modifying a loaded program, use PEEK to verify the changes before executing or saving it.

Using the Memory Commands Together

BetterCP/M provides a small collection of resident commands that can be used together for memory inspection, modification, loading, execution, and saving.

Command	Purpose
GET	Load a file into memory
PEEK / P	Examine memory
POKE	Modify memory
GO	Execute at `0100H`
JUMP	Execute at a specified address
SAVE	Save memory to a file

Together, these commands provide convenient facilities for simple memory-oriented tasks without requiring a separate debugger.

For more extensive debugging, including disassembly, breakpoints, and register examination, use a dedicated debugging utility such as DDT.

Chapter 16 - System Utilities

BetterCP/M provides several utilities for configuring the operating system, examining files and disks, managing the system clock, and preparing or maintaining system disks.

These utilities generally perform more specialized operations than the commands described in Chapters 14 and 15. Some modify the operating environment, while others work directly with disk formats or the operating system's reserved system area.

This chapter covers five utilities:

Utility	Purpose
CONFIG	Examine and modify system configuration
STAT	Display disk and file information and manage supported attributes and device assignments
TIME	Display the date and time and identify clock providers
DUP	Format, copy, and check disks
SYSGEN	Install BetterCP/M and save system images

Chapter 7 introduced system configuration, Chapter 10 explained disk operations, and Chapter 11 covered date and time facilities. This chapter provides the corresponding command references.

Because some of these utilities can modify disk structures or operating-system settings, pay particular attention to their confirmation procedures and restrictions.

16.1 CONFIG - System Configuration

Purpose: Examines and modifies BetterCP/M's system configuration, including startup commands, disk-drive definitions, disk formats, and the saved cold-boot IOBYTE.

CONFIG provides both an interactive menu and command-line operations.

It distinguishes between settings currently in use, changes prepared for saving, and settings already saved for subsequent cold boots.

Syntax

The principal forms are:

```
CONFIG
CONFIG /SHOW
CONFIG /DEFAULT
CONFIG /STARTUP="command"
CONFIG /STARTUP=""
CONFIG /IOBYTE=value
CONFIG /VER
```

The disk-configuration forms are:

```
CONFIG /PHYSICAL=n /field=value [/field=value ...]
CONFIG /LOGICAL=B /PHYSICAL=n /FORMAT="description"
CONFIG /LOGICAL=C /UNBIND
CONFIG /EDIT=B /field=value [/field=value ...]
CONFIG /EDIT=B /SLOT=n /ID=id /CODE=code
```

Use the slash-prefixed field names in the tables below. `B` and `C` are example logical drives; these operations accept B through D. Physical-profile selection accepts 0 through 3. The operation selector must come first. Names are case-insensitive and numeric values are unsigned decimal. Separate fields with whitespace. Do not

combine these forms with `/SHOW`, startup/IOBYTE options, `/DEFAULT`, or `/VER`. Duplicate or unknown fields, extra operands and numeric overflow are rejected before changes begin.

Interactive Configuration

Enter:

```
A0>CONFIG
```

CONFIG displays a menu containing the available configuration operations:

```
[ A ] Startup command

[ B ] Cold-boot IOBYTE

[ C ] Disk drive definitions
[ D ] Disk format definitions
[ E ] Create or edit a disk format

[ S ] Save current configuration on disk (SYSGEN)
```

Select the letter corresponding to the operation you want to perform.

Press Ctrl-C to leave the main menu.

CONFIG operates through the ordinary console and does not require a terminal-monitor extension.

Displaying the Configuration

To examine the current configuration without changing it:

```
A0>CONFIG /SHOW
```

This displays information about the active physical-drive profiles, logical-drive assignments, saved startup command, and saved drive defaults.

It also distinguishes between the live IOBYTE and the value saved for subsequent cold boots.

This is particularly useful when troubleshooting a configuration problem or determining whether temporary changes have been saved.

Startup Commands

BetterCP/M can execute a command automatically during a cold boot.

The standard distribution uses:

```
SUBMIT STARTUP
```

This executes `STARTUP.SUB`, allowing the system to establish a preferred working environment.

To change the saved startup command:

```
A0>CONFIG /STARTUP="SUBMIT STARTUP"
```

CONFIG requests confirmation before saving the change.

The new command takes effect during a subsequent cold boot. It is not executed immediately.

To disable automatic startup commands:

```
A0>CONFIG /STARTUP=""
```

An empty startup command prevents BetterCP/M from automatically executing a command during cold startup.

Restoring the Default Startup Command

Enter:

```
A0>CONFIG /DEFAULT
```

This prepares `SUBMIT STARTUP` as the pending startup command and opens the appropriate configuration menu.

The change is not automatically saved.

You can inspect or modify it before committing it to the system disk.

`/DEFAULT` affects the startup command; it does not restore every configuration setting to its original value.

Physical-Drive Configuration

CONFIG provides command-line facilities for changing physical-drive characteristics.

For example:

```
A0>CONFIG /PHYSICAL=1 /TRACKS=80 /SIDES=2 /STEP=6
```

This requests an 80-track, double-sided configuration for physical drive 1, with the specified step timing.

`/PHYSICAL=n` must come first, followed by at least one field. Supply each field at most once; the field order is otherwise free. Unspecified fields retain their current values.

Option	Accepted values	Units or meaning
`/PHYSICAL=`	0-3	Physical-drive index
`/TYPE=`	5 or 8	Drive type in inches
`/TRACKS=`	35, 40, 77 or 80	Track count
`/SIDES=`	1 or 2	Available sides
`/STEP=`	6, 12, 20 or 30	Milliseconds
`/SPINUP=`	0-16	Quarter-second units
`/SETTLE=`	0-255	Head-settling milliseconds

Timing effects depend on the backend. CONFIG checks the proposed profile against A's current binding and rejects a change that cannot support the boot drive. After a successful update, incompatible non-system bindings are detached and their BDOS login state reset. If later cleanup fails, `Drive profile changed; binding cleanup incomplete.` means the profile did change and affected bindings need attention.

This form changes active settings only, requires no `DISK.FDB`, and neither prompts nor saves to disk. To retain the change across cold boot, use the separate save-all-current operation described below.

Logical-Drive Configuration

To associate a logical drive with a physical drive and disk format:

```
A0>CONFIG /LOGICAL=C /PHYSICAL=1 /FORMAT="MM 80T DS DATA"
```

This assigns logical drive C to physical drive 1 using the named format.

The operands must appear in the order shown: `/LOGICAL=`, `/PHYSICAL=`, then `/FORMAT=`. Logical targets are B-D and physical indexes are 0-3. The quoted format must exactly match a catalogue display description, ignoring letter case and trailing space padding. It accepts 1-32 printable ASCII characters with at least one non-space character; quote escapes, prefixes, wildcards and row numbers are not accepted.

Named binding requires a valid `DISK.FDB`, using CONFIG's current-drive/A: catalogue lookup. Missing, corrupt, unknown, ambiguous or unsupported definitions leave the active binding unchanged.

CONFIG checks that the requested assignment is compatible with the selected drive.

To remove a logical-drive assignment:

```
A0>CONFIG /LOGICAL=C /UNBIND
```

This disconnects logical drive C from its active assignment without erasing the disk.

BetterCP/M protects drive A from binding, unbinding and format edits. `/UNBIND` requires no catalogue. Successful binding/unbinding resets only the selected drive's BDOS login state. These operations change active settings without saving, formatting or erasing media; use save-all-current to make them cold-boot defaults.

Editing Disk-Format Definitions

CONFIG provides facilities for editing the characteristics of an active disk-format assignment.

For example:

```
A0>CONFIG /EDIT=B /CKS=16
```

`/EDIT=B` through `/EDIT=D` must come first and requires at least one scalar or slot change. Scalar fields may follow in any order, once each. CONFIG starts from the active 64-byte binding, merges the named changes privately, and submits one validated update.

Field	Meaning	Decimal storage range
`/PHYSICAL=`	Physical-drive index in the binding	0-255; final binding must select 0-3
`/SPT=`	128-byte records per track	0-65535
`/BSH=`, `/BLM=`, `/EXM=`	Block shift, block mask, extent mask	0-255 each
`/DSM=`, `/DRM=`	Last allocation-block number, last directory-entry number	0-65535 each
`/AL0=`, `/AL1=`	Directory allocation masks	0-255 each
`/CKS=`, `/OFF=`	Directory check size, reserved tracks	0-65535 each
`/CYLINDERS=`, `/SECTORS=`	Cylinders, physical sectors per track	0-255 each
`/SIZE=`, `/FLAGS=`, `/SIZEMAP=`	Maximum sector-size code, format flags, explicit size-map flag	0-255 each

These are storage limits, not a promise that every value or combination is valid. The physical geometry and backend rules still apply. Start with a valid binding; an unbound drive needs a complete valid definition before it can be used. Rejection before commit leaves active settings unchanged. A successful edit resets only that drive's BDOS login state.

To edit one sector-map entry:

```
A0>CONFIG /EDIT=B /SLOT=1 /ID=200
A0>CONFIG /EDIT=B /SPT=35 /SLOT=1 /CODE=0
```

Option	Accepted values	Meaning
`/SLOT=`	1-32	One-based slot in physical sector order
`/ID=`	0-255	Physical sector ID for that slot
`/CODE=`	0-3	Packed size code: 0=128, 1=256, 2=512, 3=1024 bytes

A slot requires `/ID=`, `/CODE=`, or both. Neither value is accepted without `/SLOT=`. One slot is edited per invocation; scalar fields may accompany it after `/EDIT=`. Other slots and unspecified fields are preserved. The first explicit size edit seeds the other slots from the current uniform size. Duplicate IDs and inconsistent record/sector geometry are rejected. The examples show syntax; use values appropriate to the actual format.

Explicit size maps require `FDF.RSX`'s checked normalization service before applying the binding. FDF is separately buildable and is not selected System Disk content; install a compatible `A0:FDF.RSX` and load it with `RSX LOAD FDF` when this service is needed. Missing support or invalid maps leave the active binding unchanged.

These commands need no catalogue and make active changes without a confirmation or disk save. **Changing a binding definition does not format the disk.** An incorrect definition can make existing media inaccessible. Use the separate save-all-current operation to preserve validated changes across cold boot.

Saved IOBYTE

BetterCP/M retains CP/M's IOBYTE mechanism for selecting logical character devices.

STAT changes the live IOBYTE, while CONFIG can save a value for subsequent disk cold boots.

For example:

```
A0>CONFIG /IOBYTE=85
```

The value is an unsigned decimal byte, 0-255.

CONFIG requests confirmation before saving it.

This operation does not change the IOBYTE currently in use.

The selected device assignments must be supported by the computer's BIOS.

Saving Configuration Changes

CONFIG distinguishes between saving pending cold-boot changes and saving all current settings as the new startup defaults.

For example, you may have temporarily changed a drive assignment while also preparing a new startup command.

Saving only the pending startup change allows you to retain your existing saved drive configuration.

Saving all current settings makes the active configuration the new startup default.

From the main menu choose **S**, then:

```
[ A ] Save pending cold-boot changes only
[ B ] Save all current settings as startup defaults
BREAK (^C) returns without saving.
```

Choose **A** to retain saved drive defaults while saving pending cold-boot changes. Choose **B** to capture the active physical profiles and logical bindings as defaults as well. Follow the confirmation before the disk write. Active CLI disk changes survive leaving CONFIG but are lost on cold boot unless explicitly saved through **S**, **B**; they are not invocation-private pending edits.

CONFIG verifies saved configuration writes and reports failures.

Keep a backup of a working system disk before making significant changes to saved configuration information.

Version Information

Enter:

```
A0>CONFIG /VER
```

CONFIG displays its version and build identification without modifying the system.

The option must be used alone.

Limitations

CONFIG is not a general-purpose preferences editor.

BetterCP/M 1.0 does not use CONFIG to manage resident extensions, terminal personalities, keyboard mappings, or clock providers.

Those facilities are managed through their respective commands and extensions.

For an introduction to configuration procedures, see Chapter 7.

16.2 STAT - System and File Statistics

Purpose: Displays information about disks, files, and logical devices, and provides supported facilities for examining or changing file attributes and device assignments.

STAT retains the familiar role of Digital Research's CP/M utility while adding support for BetterCP/M's extended drive/user specifications.

Basic Use

To examine the status of the current disk:

```
A0>STAT
```

STAT displays disk information according to the supported CP/M conventions.

To examine a particular file:

```
A0>STAT REPORT.TXT
```

A drive-qualified file specification can also be used:

```
A0>STAT B:REPORT.TXT
```

Syntax and Status Reports

The supported forms are:

```
STAT
STAT filespec [$S|$R/O|$R/W|$SYS|$DIR]
STAT d:
STAT d:=R/O
STAT DSK:
STAT d:DSK:
STAT USR:
STAT DEV:
STAT VAL:
STAT MEM
STAT CON:=device:
STAT RDR:=device:
STAT PUN:=device:
STAT LST:=device:
STAT target:
```

Here `d:` means a drive selection. Conventional operands also accept numeric user selectors such as `5:` and `C3:`. Temporary selections are restored before STAT returns. The alternatives inside square brackets on the filespec line are optional file options; type just the option you need, without those brackets or vertical bars.

Command	Information or action
`STAT`	Read/write status and free space for logged drives
`STAT B:`	Free space on B at the current user
`STAT DSK:`	Drive characteristics for logged drives
`STAT B:DSK:`	Drive characteristics for B, including record/block capacity and directory limits
`STAT USR:`	Current user and user areas containing files on the current drive

Command	Information or action
`STAT DEV:`	Current logical-device assignments
`STAT VAL:`	Valid file options and device assignments
`STAT B:=R/O`	Temporarily mark B read-only

The drive read-only setting is cleared by the normal warm-start/reset path. STAT does not accept `B:=R/W`; use the normal reset procedure to clear the software lock. It does not override hardware write protection.

Memory Map

```
A0>STAT MEM
```

`STAT MEM` displays the current memory map, including protected system regions and RSXs, the CCP and CPXs, transient space, and page zero. Address ranges and byte counts identify what occupies memory.

The final maximum-loadable-COM figure is rounded down to complete 128-byte load records. It can include the memory currently occupied by the reclaimable CCP and CPX command code. That figure describes how large a transient program can be loaded; it is not an additional free region to add to the map. Protected RSXs reduce that maximum. Use `RSX LIST` to identify optional services when investigating a memory shortage.

Extended Drive/User Selection

BetterCP/M extends STAT's file-inspection facilities to support explicit user areas and compound selections.

For example:

```
A0>STAT B3:*.COM
```

examines matching files in drive B, user area 3.

To examine several user areas:

```
A0>STAT B[3,5]:*.COM
```

To examine all user areas on drive B:

```
A0>STAT B[-]:*.COM
```

Compound selections are also supported:

```
A0>STAT [A0,C[3,5,7-11],5]:*.DAT
```

STAT processes the selected locations and identifies each result by its drive and user area.

Duplicate locations are visited only once.

Displaying File Sizes

STAT provides the `$S` option for displaying logical file sizes.

For example:

```
A0>STAT B[3,5]:*.DAT $S
```

This examines matching files in the selected user areas and includes the additional size information.

The distinction between logical file size and allocated disk space is important. CP/M allocates storage in blocks, so a file's allocated space may exceed the amount of data it contains.

File Attributes

Use a file option to change attributes in one conventional drive/user location:

```
A0>STAT REPORT.TXT $R/O
```

```
A0>STAT REPORT.TXT $R/W
A0>STAT B3:*.COM $SYS
A0>STAT B3:*.COM $DIR
```

`$R/O` sets read-only protection; `$R/W` clears it. `$SYS` sets the system attribute, while `$DIR` clears SYS so ordinary directory listings can show the file. Wildcards apply the change to all matching files and their extents. These operations do not provide an ARC-setting option.

Check the filespec before changing attributes, especially with wildcards. Disk write protection can prevent changes. In a `.SUB` source file, double each dollar sign, for example `STAT REPORT.TXT $$R/O`.

Logical Device Assignments

STAT examines and changes the live IOBYTE, which selects the logical console, reader, punch, and list devices:

```
A0>STAT DEV:
A0>STAT LST:=CRT:
A0>STAT CRT:
A0>STAT VAL:
```

The second command maps list output to CRT. The third is an inverse query: it lists every logical device currently assigned to CRT, or reports that CRT is not assigned. Queries do not change assignments or test whether hardware is ready.

Logical device	Accepted targets
`CON:`	`TTY:`, `CRT:`, `BAT:`, `UC1:`
`RDR:`	`TTY:`, `PTR:`, `UR1:`, `UR2:`
`PUN:`	`TTY:`, `PTP:`, `UP1:`, `UP2:`
`LST:`	`TTY:`, `CRT:`, `LPT:`, `UL1:`

Several assignments may be supplied together, for example `STAT CON:=CRT: LST:=LPT:`. An inverse query accepts one target from this table, followed by a colon, with no additional operands. It reports matching logical devices in CON/RDR/PUN/LST order.

The available hardware routes depend on the BIOS. An accepted IOBYTE value does not create a device or guarantee that a particular interface is supported. `BAT:` selects the conventional batch console arrangement using reader input and list output where supported; it does not activate SUBMIT or XSUB.

Extended Selection Restrictions

BetterCP/M's compound drive/user selectors are supported for file inspection.

They are not accepted indiscriminately by every STAT operation.

In particular, extended selectors cannot be used with the conventional drive-wide operations or file-attribute modification forms.

For those operations, use the supported conventional single-location syntax.

STAT and CONFIG

STAT and CONFIG serve different purposes when managing device assignments.

STAT changes the live IOBYTE. Its changes affect the current operating environment.

CONFIG manages the saved cold-boot IOBYTE. Its saved value is established during a subsequent disk cold boot.

The two facilities complement one another.

For example, you can use STAT to test a supported device assignment before saving it as a startup default through CONFIG.

Examples

Examine the current disk:

```
A0>STAT
```

Examine files in user area 3:

```
A0>STAT B3:*.COM
```

Examine files across all user areas on drive B:

```
A0>STAT B[-]:*.COM
```

Display file sizes for selected user areas:

```
A0>STAT B[3,5]:*.DAT $S
```

For further information about drive/user specifications and file attributes, see Chapter 4.

16.3 TIME - Date and Time

Purpose: Displays the current date and time, identifies the installed clock provider, and provides an interface for requesting clock-setting operations.

TIME uses BetterCP/M's resident clock services. It does not communicate directly with individual clock devices.

BetterCP/M 1.0 includes read-only clock providers for FreHD and z80pack.

Syntax

```
TIME
TIME /PROVIDER
TIME /SET LL HH HR MN SC
```

The first form displays the date and time. The second identifies the installed clock provider. The third requests a clock-setting operation.

Displaying the Date and Time

Enter:

```
A0>TIME
```

TIME obtains the current date and time from the installed provider.

A provider must be loaded before TIME can obtain clock information.

On a TRS-80 Model 4 equipped with FreHD:

```
A0>RSX LOAD FREHDCLK
A0>TIME
```

Under z80pack/cpmsim:

```
A0>RSX LOAD ZPRTC
A0>TIME
```

If no clock provider is installed, TIME reports that the required service is unavailable.

Identifying the Provider

Enter:

```
A0>TIME /PROVIDER
```

TIME displays information about the installed clock provider, including its identity, interface version, and capabilities.

The two providers supplied with BetterCP/M 1.0 are:

Provider	Extension	Capabilities
FreHD	`FREHDCLK.RSX`	Read-only
z80pack	`ZPRTC.RSX`	Read-only

The provider information can help determine which clock service is active and whether it supports setting the clock.

Setting the Clock

TIME includes the following SET syntax:

```
TIME /SET LL HH HR MN SC
```

The arguments are five two-digit hexadecimal bytes representing BetterCP/M's native clock record.

Argument	Meaning
`LL`	Low byte of the day count
`HH`	High byte of the day count
`HR`	Hour, packed BCD
`MN`	Minute, packed BCD
`SC`	Second, packed BCD

The day count is measured from the CP/M clock epoch of December 31, 1977, with day 1 corresponding to January 1, 1978.

For example:

```
A0>TIME /SET 01 00 12 34 56
```

requests January 1, 1978, at 12:34:56.

The hour, minute, and second are represented in packed binary-coded decimal. Thus, hexadecimal byte `12` represents decimal 12, not decimal 18.

TIME checks the arguments before requesting the operation.

Malformed hexadecimal values, invalid BCD digits, and out-of-range time fields are rejected.

Clock-Setting Limitations

Neither clock provider supplied with BetterCP/M 1.0 supports setting the clock.

Consequently, a valid SET request is rejected as unsupported when either `FREHDCLK.RSX` or `ZPRTC.RSX` is installed.

The existing clock value remains unchanged.

The SET interface is implemented for compatibility with the native clock service and for use with providers that support writable clocks.

P2DOS Compatibility

An optional, separately buildable `P2DOS.RSX` supplies the P2DOS-compatible clock interface. It is not selected System Disk content; install a compatible carrier in A0 before using the example below.

To use it under z80pack:

```
A0>RSX LOAD ZPRTC
A0>RSX LOAD P2DOS
```

P2DOS obtains its clock services through the installed native provider.

Because the supplied providers are read-only, P2DOS-compatible requests to set the clock are also unsupported.

The P2DOS extension is not required for ordinary use of TIME.

For additional information about clock providers and their capabilities, see Chapter 11.

16.4 DUP - Disk Utilities

Purpose: Formats disks, copies entire disks, and checks disks for physical read errors.

DUP is BetterCP/M's disk-management utility. Unlike the COPY command, which operates on individual files, DUP works directly with the physical organization and contents of a disk.

It provides three principal operations:

Operation	Purpose
FORMAT	Prepare a disk using its assigned disk format
COPY	Copy the contents of one disk to another
CHECK	Examine a disk for unreadable sectors

All three operations are available through DUP's interactive menu or directly from the command line.

Formatting and disk copying can destroy existing data. Always verify the selected drives before confirming either operation.

Syntax

```
DUP
DUP /FORMAT drive:
DUP /COPY source: destination:
DUP /CHECK drive:
DUP /VER
```

For example:

```
A0>DUP
A0>DUP /FORMAT B:
A0>DUP /COPY A: B:
A0>DUP /CHECK C:
```

The command-line forms use the same operations, validations, and safeguards as the interactive menu.

There is no command-line option for bypassing destructive-operation confirmations.

Interactive Operation

To start DUP interactively, enter:

```
A0>DUP
```

DUP displays its main menu:

```
A  Format a disk
B  Copy a disk
C  Check a disk for errors
```

Select the desired operation by entering its letter.

DUP then asks for the appropriate drive or drives.

Press Ctrl-C to exit the main menu. At a drive-selection prompt, Ctrl-C returns to the menu.

The current DUP interface supports logical drives A through D.

Drive and Format Selection

DUP uses the disk-format definition already assigned to each logical drive.

It does not provide a separate disk-format selection menu.

If you need to change a drive's format, use CONFIG before starting DUP.

For example:

```
A0>CONFIG
```

You can then select the appropriate disk format and associate it with the desired logical drive.

DUP uses that assignment when formatting, copying, or checking the disk.

The available formats and operations depend on the computer's BIOS and disk hardware.

A format listed in the system's catalog is not necessarily supported for every operation on every platform.

DUP reports unsupported operations rather than substituting another format.

Protected System Drive

BetterCP/M protects physical drive 0 against destructive DUP operations.

Physical drive 0 cannot be used as the destination for FORMAT or COPY.

This protection follows the physical drive even if logical drive letters have been reassigned.

The protected drive may still be read by CHECK or used as the source of a disk-copy operation.

This safeguard helps prevent accidental destruction of the running system disk.

Formatting a Disk

To format a disk interactively, start DUP and select option A.

DUP asks you to identify the logical drive to format.

Alternatively, enter:

```
A0>DUP /FORMAT B:
```

This requests formatting of drive B using its currently assigned format.

DUP validates the drive and format before proceeding.

If the operation is supported, it displays a warning that the destination disk's contents will be destroyed and requests confirmation.

Only `Y` proceeds.

Formatting destroys the previous contents of the destination disk.

Make sure the correct disk is inserted and that its existing contents are no longer needed.

Format Verification

DUP verifies formatting by reading back the formatted tracks.

If formatting and verification complete successfully, DUP reports the number of tracks processed and displays:

```
Format complete.
```

If a write or verification error occurs, DUP stops the operation and reports the problem.

An interrupted format may leave the disk only partly formatted.

Do not treat such a disk as ready for normal use.

Interrupting Formatting

Ctrl-C can interrupt formatting between disk operations.

An operation already in progress may finish before the interruption takes effect.

If formatting is interrupted, the destination disk may require formatting again before it can be used reliably.

Copying a Disk

DUP's COPY operation duplicates the contents of an entire disk.

Unlike the file-oriented COPY command, it copies the sector data represented by the source disk's format, including system areas, directories, file data, and unused space.

This makes it useful for preparing duplicate disks and maintaining complete disk backups.

To copy a disk interactively, start DUP and select option B.

DUP asks for the source and destination drives.

Alternatively, enter:

```
A0>DUP /COPY A: B:
```

This requests a copy from drive A to drive B.

The source is read, while the destination is written.

The destination's previous contents will be replaced.

Two-Drive Requirement

DUP requires two different physical drives for disk copying.

It does not support copying between two disks by repeatedly exchanging them in a single drive.

The destination must also satisfy the system's drive-protection restrictions.

In particular, physical drive 0 cannot be used as the destination.

Copying Between Different Formats

DUP examines the format assignments of the source and destination.

If they match, DUP can copy the source without first reformatting the destination.

If they differ, DUP reports that the destination will be formatted to match the source.

The operation proceeds only if the required formatting and copying operations are supported.

DUP requests confirmation before replacing the destination's contents.

Check both the logical and physical drive assignments before proceeding.

Copy Verification

DUP verifies copied data by reading it back from the destination and comparing it with the source data.

A successful operation reports:

```
Copy complete; destination verified.
```

This indicates that the data copied by DUP has been verified.

If a read, write, or comparison error occurs, DUP reports the failure.

An interrupted or failed copy may leave the destination partly written.

Do not rely on such a disk as a complete backup.

What DUP Copies

DUP copies the sector contents represented by the source disk's configured format.

This includes the system area, directory information, file data, and unused disk space.

It is not a magnetic flux-level copying utility.

It does not reproduce physical characteristics outside the represented sector data, such as weak sectors or special deleted-data address marks.

For ordinary BetterCP/M and compatible CP/M disks, the operation provides a verified sector-data copy.

Temporary Format Changes

When copying between different disk formats, DUP may temporarily change the destination's active format assignment.

It restores the previous assignment when the operation completes.

However, restoring the assignment does not restore the destination disk's former physical format.

After copying, use CONFIG to assign the copied format before accessing the destination normally.

If you intend to use that format regularly, save the appropriate configuration as the new startup default.

Checking a Disk

DUP's CHECK operation examines a disk for physical read errors without modifying its contents.

It is useful when investigating suspected media problems, checking an unfamiliar disk, or determining whether the sectors of a disk can be read reliably.

To check a disk interactively, start DUP and select option C.

DUP asks you to select the drive to examine.

Alternatively, enter:

```
A0>DUP /CHECK B:
```

This checks the disk in drive B using its currently assigned format.

Unlike FORMAT and COPY, CHECK does not require confirmation because it performs no writes.

What CHECK Examines

CHECK reads the sectors represented by the disk-format definition assigned to the selected drive.

If a sector cannot be read successfully, DUP reports the error and its location.

Ordinary read errors do not necessarily stop the check. DUP continues examining the remaining sectors whenever it can safely do so.

When a physical sector contains several logical CP/M records, an unreadable sector is counted once rather than once for each record it contains.

Check Results

When CHECK completes, DUP displays:

```
Check complete. Unreadable sectors: n
```

The value of `n` indicates the number of physical sectors that could not be read successfully.

For example:

```
Check complete. Unreadable sectors: 0
```

indicates that all sectors examined during the check were readable.

A nonzero count indicates that one or more sectors could not be read.

What CHECK Does Not Do

CHECK examines physical readability. It does not repair disks or validate the logical consistency of the filesystem.

In particular, CHECK does not:

- Repair unreadable sectors.
- Reconstruct damaged files or directories.
- Restore deleted files.
- Compare the disk with another disk.
- Verify that individual files contain the expected data.

A disk can pass CHECK while still containing damaged or incorrect files.

If important data may have been corrupted, compare it with a known good copy or restore it from a backup.

Interrupting a Check

Press Ctrl-C to interrupt CHECK.

Because CHECK performs no writes, interrupting it does not change the disk.

DUP reports:

```
Check stopped. No disk contents changed.
```

An interrupted check is incomplete and should not be treated as evidence that the entire disk is readable.

Errors and Recovery

DUP reports errors encountered during disk operations.

These may include a drive that is not ready, write protection, missing sectors, read errors, unsupported operations, or verification failures.

Where possible, DUP identifies the physical location associated with the error.

For example, it may report the cylinder, side, sector, or record involved.

Drive Not Ready

If DUP reports that a drive is not ready, check that the correct disk is inserted and that the drive is available.

On an emulator, verify that the appropriate disk image is attached.

Write Protection

If a destination disk is write-protected, DUP cannot format it or write copied data to it.

Check the disk's write-protection setting before retrying.

Do not remove write protection unless you're certain the destination disk is the one you intend to modify.

Unsupported Operations

Some disk formats can be described by BetterCP/M's format catalog but cannot be formatted or copied using a particular hardware backend.

DUP reports unsupported operations rather than silently substituting another format.

If an operation is rejected, examine the drive configuration and consult the platform documentation.

Read and Verification Errors

A read error indicates that DUP could not obtain the requested data reliably.

A verification error indicates that data read back from the destination did not match the expected contents.

For FORMAT and COPY, errors that make continued operation unsafe cause DUP to stop.

The destination may be incomplete and should not be treated as successfully prepared.

Interrupted Operations

DUP recognizes Ctrl-C during physical disk operations.

An operation already in progress may finish before the interruption takes effect.

The consequences depend on the operation:

Operation	Effect of interruption
FORMAT	Destination may be partly formatted
COPY	Destination may be partly written
CHECK	Operation stops without changing disk contents

An interrupted FORMAT or COPY should be repeated from a reliable source after the cause of the interruption has been corrected.

Restoring Temporary Drive Assignments

When DUP temporarily changes a destination drive's format assignment during copying, it attempts to restore the previous assignment before returning to the command prompt.

If restoration fails, DUP displays:

```
Cannot restore destination configuration. Press a key to retry.
```

Press a key to retry the restoration.

DUP retains the information needed to restore the previous assignment and does not knowingly return to normal operation while the temporary assignment remains installed.

Do not reset the computer merely to dismiss this message. Follow the recovery prompt unless a hardware failure makes further recovery impossible.

Memory Requirements

DUP requires approximately 51K of available transient program memory.

If resident extensions have reduced the available TPA below this requirement, DUP reports insufficient memory and does not begin the requested operation.

You may be able to recover the required memory by unloading resident extensions that are no longer needed.

For information about managing extensions, see Chapter 8.

Version Information

To display DUP's version and build identification, enter:

```
A0>DUP /VER
```

DUP displays its identification and returns to the command prompt without performing a disk operation.

Examples

Start the interactive disk utility:

```
A0>DUP
```

Format drive B:

```
A0>DUP /FORMAT B:
```

Copy the contents of drive A to drive B:

```
A0>DUP /COPY A: B:
```

Check drive C for unreadable sectors:

```
A0>DUP /CHECK C:
```

Display version information:

```
A0>DUP /VER
```

For an introduction to disk formats, system disks, and backup procedures, see Chapter 10, *Working with Disks*.

16.5 SYSGEN - System Generation

Purpose: Installs BetterCP/M onto a prepared system disk or saves an installed operating system as a system-image file.

SYSGEN is used when preparing a bootable BetterCP/M disk, replacing an installed operating system, or preserving a working system configuration.

Unlike DUP, which can format and copy entire disks, SYSGEN works specifically with the operating system's reserved system area.

It can obtain the operating system from another bootable disk or from a `SYSTEM.SYS` package.

SYSGEN also provides the reverse operation: reading the installed operating system from a disk and saving it as a package file for later use.

SYSGEN does not format disks or copy ordinary files. It installs or saves the operating system itself.

Syntax

BetterCP/M 1.0 supports the following forms:

```
SYSGEN
SYSGEN source: destination:
SYSGEN [drive:]package destination:
SYSGEN source: [drive:]package
SYSGEN source:
SYSGEN /VER
```

The principal operations are:

Form	Operation
`SYSGEN`	Install from drive A, prompting for a destination

Form	Operation
`SYSGEN A: B:`	Install from one system disk onto another
`SYSGEN SYSTEM.SYS B:`	Install from a system-image package
`SYSGEN A:`	Save the installed system from A as `SYSTEM.SYS`
`SYSGEN A: B:BACKUP.SYS`	Save the installed system under a specified filename
`SYSGEN /VER`	Display version information

Source drives may be A through D. Installation destinations may be B through D, subject to physical-drive restrictions.

Interactive Installation

To start SYSGEN interactively, enter:

```
A0>SYSGEN
```

SYSGEN uses drive A as the source system and prompts:

```
Destination drive [ B - D ], or BREAK (^C) to exit:
```

Select the destination drive.

For example, entering `B` requests installation onto drive B.

SYSGEN then examines the source and destination to determine whether the operation can be performed safely.

If the destination is suitable, SYSGEN displays the installation details and requests confirmation before writing.

Press Ctrl-C at the destination prompt to exit without modifying any disk.

Installing from Another System Disk

To specify both drives directly, use:

```
SYSGEN source: destination:
```

For example:

```
A0>SYSGEN A: B:
```

This reads the installed BetterCP/M system from drive A and installs it onto drive B.

Another example is:

```
A0>SYSGEN C: D:
```

This installs the system from drive C onto drive D.

The source and destination must refer to different physical drives.

Using different logical drive letters is not sufficient if both letters refer to the same physical device.

SYSGEN checks the physical assignments and refuses an installation that would use the same physical drive as both source and destination.

Installing from SYSTEM.SYS

SYSGEN can also install BetterCP/M from a system-image package.

For example:

```
A0>SYSGEN SYSTEM.SYS B:
```

This reads `SYSTEM.SYS` from the current drive and user area and installs its operating-system image onto drive B.

A drive-qualified package name can also be used:

```
A0>SYSGEN C:SYSTEM.SYS B:
```

This reads the package from drive C.

The package filename need not be `SYSTEM.SYS`. Any valid, explicitly specified CP/M 8.3 filename can be used.

For example:

```
A0>SYSGEN B:BASE090.SYS C:
```

This installs the saved system image contained in `BASE090.SYS` onto drive C.

Package filenames do not support wildcard specifications, compound drive/user selectors, or explicit user-area qualifications.

An unqualified package name refers to the current drive and user area.

System-Image Packages

A BetterCP/M system-image package contains the operating-system image together with information needed to identify and install it correctly.

The standard package is named:

```
SYSTEM.SYS
```

A valid BetterCP/M 1.0 package contains a header followed by the operating-system image.

SYSGEN checks the package before installation.

It verifies that the expected header and system-image structure are present, that the complete package can be read, and that the image is suitable for the destination.

If the package is invalid or incompatible, SYSGEN refuses the operation before writing to the destination.

A structurally valid package is not necessarily free from every possible form of corruption. Use system images obtained from a trusted source.

Preparing the Destination Disk

Before using SYSGEN, the destination disk must already have a suitable format.

SYSGEN does not format disks.

If the disk needs to be formatted, use CONFIG to establish the appropriate disk-format assignment and DUP to perform the formatting.

For example:

```
A0>CONFIG
A0>DUP /FORMAT B:
A0>SYSGEN SYSTEM.SYS B:
```

This illustrates the general preparation sequence.

The correct format depends on the computer and the system image being installed.

The destination must provide a compatible bootstrap arrangement and sufficient reserved space for BetterCP/M.

Compatible Disk Formats

SYSGEN does not require the source and destination disks to have identical filesystem capacities.

For example, it may be possible to install BetterCP/M onto a disk with a larger filesystem than the source, provided the destination has compatible bootstrap geometry and sufficient reserved system space.

BetterCP/M 1.0 requires at least 160 reserved 128-byte logical records for the installed system.

This corresponds to 20,480 bytes of reserved system space.

A destination may have more reserved space, but SYSGEN writes only the required system records.

If the reserved area is too small, SYSGEN reports:

```
Destination reserved system area is too small (160 records required).
```

If the destination's bootstrap geometry is incompatible, SYSGEN refuses the installation.

It does not attempt to convert the disk into a different format.

Protected System Drive

BetterCP/M protects physical drive 0 against SYSGEN destination writes.

This protection follows the physical drive, regardless of its logical drive letter.

For example, assigning physical drive 0 to another logical letter does not make it an acceptable SYSGEN destination.

If the requested destination resolves to physical drive 0, SYSGEN reports:

```
Physical drive 0 is protected as the running system disk.
```

This safeguard helps prevent accidental replacement of the operating system from which the computer is currently running.

Confirming Installation

Before writing to the destination, SYSGEN displays the installation information and requests confirmation.

The warning includes:

```
The destination system area will be replaced.
Files outside the reserved system tracks will be preserved.
Proceed (Y/N)?
```

Only `Y` begins installation.

Any other response cancels the operation without modifying the destination.

The confirmation is required whether SYSGEN was started interactively or with command-line arguments.

BetterCP/M 1.0 does not provide an option for bypassing this confirmation.

Preserving Existing Files

SYSGEN replaces the operating system in the reserved system area.

It does not erase or rewrite ordinary filesystem records.

This means that a compatible disk containing existing files can receive a new BetterCP/M system without necessarily losing those files.

For example, a disk containing application programs and data files may be suitable for system installation if it has the required reserved system area.

SYSGEN preserves records outside the system area it installs.

However, this protection does not mean that every disk is suitable.

A disk with no compatible system area, or with insufficient reserved space, cannot be made bootable merely by running SYSGEN.

Installation and Verification

SYSGEN reads the complete source image before beginning destination writes.

It also prepares the image for the destination's configuration.

During installation, SYSGEN writes the system records and verifies each one by reading it back and comparing the data.

The boot record is written last.

When the operation completes successfully, SYSGEN reports:

```
System installed and verified. The destination is bootable.
```

This indicates that the operating-system image was installed and verified.

It does not mean that the disk contains every utility or supporting file required for your preferred working environment.

Those files must be copied separately.

For example, a complete system disk may also require the appropriate command utilities, resident extensions, and `STARTUP.SUB`.

Saving an Installed System to a Package

SYSGEN can also read the operating system from a disk and save it as a `SYSTEM.SYS` package.

This is useful for preserving a working configuration, maintaining several system versions, or preparing an image for installation onto another compatible disk.

To save the installed system from drive A:

```
A0>SYSGEN A:
```

SYSGEN reads the system area from drive A and creates `SYSTEM.SYS` in the current drive and user area.

You can specify another filename and destination drive:

```
A0>SYSGEN A: B:BASE090.SYS
```

This saves the installed system from drive A as `BASE090.SYS` on drive B.

The resulting package can later be installed using:

```
A0>SYSGEN B:BASE090.SYS C:
```

What a Saved Package Contains

A saved system package includes the installed BetterCP/M operating system and its saved configuration information.

This includes the saved startup command, cold-boot IOBYTE, and saved drive configuration.

It does not include unsaved changes made only to the running system.

It also does not include ordinary files from the source disk.

For example, changes made to `STARTUP.SUB` are not captured in the system-image package because that file resides in the ordinary filesystem.

To preserve both the operating system and its supporting files, use a complete disk backup.

Replacing an Existing Package

If the requested output filename already exists, SYSGEN asks:

```
Replace existing package (Y/N)?
```

Only `Y` permits replacement.

Read-only and system files are protected against replacement.

SYSGEN verifies the package before reporting successful export.

A successful operation displays:

```
Package exported and verified.
```

Interrupted Package Export

SYSGEN takes precautions to avoid replacing an existing package with an incomplete one.

It first writes the new package to a temporary file, verifies it, and then publishes it under the requested filename.

During replacement, it may use the temporary filenames:

```
SYSGEN.$$$
SYSGEN.BAK
```

These names are reserved for recovery.

If a package export fails, one or both files may remain.

Do not delete these files without first examining the failed operation. They may contain a recoverable package or the previous version of the file.

SYSGEN reports publication failures and identifies the files that should be inspected.

The export procedure reduces the risk of losing an existing package, but it cannot guarantee recovery from every possible disk or power failure.

Interrupting Installation

SYSGEN recognizes Ctrl-C during installation.

At the interactive destination prompt, Ctrl-C exits without modifying the disk.

During installation, Ctrl-C is checked between disk operations.

An operation already in progress may finish before the interruption takes effect.

If installation is interrupted after writing has begun, the destination system area may be incomplete.

The disk may no longer boot.

SYSGEN does not automatically restore the previous operating system after a partial installation.

Correct the cause of the interruption and repeat installation from a valid source.

Installation Errors

SYSGEN reports errors encountered while reading the source, validating the destination, writing system records, or verifying the installed image.

Common causes include incompatible disk formats, insufficient reserved system space, unavailable drives, write protection, and read or write failures.

Invalid System Package

If a system-image package is invalid or cannot be installed on the selected destination, SYSGEN may report:

```
The system package is invalid or cannot represent the destination.
```

Check that the package is a valid BetterCP/M system image and that it is compatible with the destination.

Incompatible Bootstrap Geometry

If the destination's bootstrap geometry is unsuitable, SYSGEN reports:

```
Destination bootstrap geometry is not supported by this system package.
```

Use CONFIG to examine the destination's format assignment.

SYSGEN cannot convert an incompatible disk format into a bootable one.

Insufficient Reserved Space

If the destination does not provide enough reserved system space, SYSGEN reports:

```
Destination reserved system area is too small (160 records required).
```

A disk with insufficient reserved space cannot accept the BetterCP/M 1.0 system image.

Read and Write Failures

If a source read fails, SYSGEN stops before beginning destination installation.

If a destination write or verification fails after installation has begun, the destination may contain an incomplete system.

Do not assume that such a disk is bootable.

Memory Requirements

Saving an installed system to a package requires at least 36K of available transient program memory.

If resident extensions have reduced the available TPA below this requirement, SYSGEN refuses the export operation.

Unload unnecessary extensions before retrying.

Version Information

To display SYSGEN's version and build identification:

```
A0>SYSGEN /VER
```

The utility displays its identification and returns to the command prompt without performing an installation or export.

Examples

Start interactive installation:

```
A0>SYSGEN
```

Install from drive A onto drive B:

```
A0>SYSGEN A: B:
```

Install from a system-image package:

```
A0>SYSGEN SYSTEM.SYS B:
```

Install from a package on another drive:

```
A0>SYSGEN C:SYSTEM.SYS B:
```

Save the installed system from drive A:

```
A0>SYSGEN A:
```

Save the installed system under a specified filename:

```
A0>SYSGEN A: B:BASE090.SYS
```

Restore a saved system image:

```
A0>SYSGEN B:BASE090.SYS C:
```

Display version information:

```
A0>SYSGEN /VER
```

SYSGEN, DUP, and SYSBUILD

BetterCP/M provides three utilities with related but distinct purposes.

Utility	Purpose
DUP	Format disks, copy entire disks, and check physical readability
SYSGEN	Install BetterCP/M onto prepared media or save an installed system image
SYSBUILD	Construct a system-image package from operating-system build products

For ordinary system maintenance, DUP and SYSGEN provide the facilities needed to prepare and install BetterCP/M disks.

SYSBUILD is primarily intended for developers and system builders preparing new operating-system images.

For an introduction to disk preparation, installation, and backup procedures, see Chapter 10, *Working with Disks*.

Chapter 17 - Batch and Extension Commands

BetterCP/M provides facilities for automating command sequences and extending the operating environment without rebuilding the operating system.

Two utilities, **SUBMIT** and **XSUB**, provide batch processing. SUBMIT executes commands stored in a text file, while XSUB allows compatible programs to receive input from a batch file instead of the keyboard.

Two additional utilities, **CPX** and **RSX**, manage resident extensions. CPXs add commands to the command processor, while RSXs provide services that remain available while other programs execute.

This chapter covers four utilities:

Utility	Purpose
SUBMIT	Execute commands from a batch file
XSUB	Supply batch-file input to compatible programs
CPX	Manage Command Processor Extensions
RSX	Manage Resident System Extensions

Chapter 6 introduced batch processing, and Chapter 8 explained resident extensions. This chapter provides the corresponding command references.

17.1 SUBMIT - Batch Command Processing

Purpose: Executes a sequence of commands stored in a text file.

SUBMIT allows frequently used command sequences to be saved and executed without entering each command individually.

It retains the familiar CP/M batch-processing conventions, including `.SUB` files and positional parameter substitution.

Syntax

```
SUBMIT filename [parameters]
```

The filename identifies a SUBMIT command file. The `.SUB` extension is normally omitted when invoking the utility.

For example:

```
A0>SUBMIT BACKUP
```

executes the commands contained in `BACKUP.SUB`.

Creating a SUBMIT File

A SUBMIT file is an ordinary text file containing commands.

Each command appears on a separate line.

For example, a file named `BACKUP.SUB` might contain:

```
DIR *.ASM
:COPY *.ASM B:
DIR B:*.ASM
```

To execute it:

```
A0>SUBMIT BACKUP
```

SUBMIT arranges for the commands to be executed in sequence.

The commands are interpreted by the normal BetterCP/M command environment.

Consequently, a SUBMIT file can contain resident commands, transient programs, and commands provided by installed CPXs.

Parameters

SUBMIT files can accept parameters supplied when the file is invoked.

Parameters are referenced using a dollar sign followed by a number.

For example, suppose `BUILD.SUB` contains:

```
ASM $1
LOAD $1
$1
```

Enter:

```
A0>SUBMIT BUILD TEST
```

SUBMIT substitutes `TEST` for `$1`, producing the equivalent of:

```
ASM TEST
LOAD TEST
TEST
```

This allows the same command file to be used with different programs.

Multiple Parameters

A SUBMIT file can refer to several parameters.

For example, `COPYFILE.SUB` might contain:

```
:COPY $1 $2
```

You could invoke it with:

```
A0>SUBMIT COPYFILE REPORT.TXT B:REPORT.TXT
```

The resulting command would be:

```
:COPY REPORT.TXT B:REPORT.TXT
```

Parameter substitution is textual. SUBMIT does not interpret filenames or options while performing the substitution.

For a literal dollar sign in the `.SUB` source, write `$$`. For example, write `[$$ARC]` so the executed command receives `[$ARC]`. A bare `$ARC` causes `Parameter Error`. See Chapter 6, *Literal Dollar Signs in Command Files*, for a complete backup example.

The resulting command is interpreted by the command processor or utility that executes it.

Queue Location and Limits

SUBMIT reads the `.SUB` source in the invoking user area, using the filename's drive qualifier when supplied. It always writes the shared command/input queue as `A0:$$$.SUB`. A0 must be writable and have room for the queue, even if the script and all working files are on another drive. Queue access restores the caller's drive/user context; commands may change drive or user without relocating the queue.

The CCP consumes command lines from this queue. XSUB uses the same queue for buffered program input. Run one submitted procedure at a time; starting another procedure replaces the shared queue rather than creating an independent job.

- The nine ordinary positional parameters are `$1` through `$9`; `$0` refers to the submitted command-file name from the invocation.
- Parameters are separated by blanks. There is no quoting language for grouping a parameter containing spaces, and no named variables or defaults. A missing referenced parameter expands to empty text, so check that all required arguments are supplied. Invalid dollar references produce `Parameter Error`.
- Each expanded command is limited to 125 bytes, after parameter, `$$`, and caret-control expansion. An overlong command produces `Command Too Long`.
- The complete expanded procedure must fit the 2K aggregate command buffer, including its bookkeeping. This is not a 2K source-file allowance; a procedure with many short lines can also reach the limit and produce `Command Buffer Overflow`. Split a long procedure into smaller separately submitted jobs.
- Empty source lines are omitted. `^A` through `^Z` in the source produce the corresponding control bytes; literal dollar signs require `$$`.

The queue is deleted at normal exhaustion, after a bad submitted command, on console abort, and after malformed-queue or queue read/close failures. An ordinary supported warm boot can reconstruct the command environment and continue pending batch lines. Cancelling a procedure does not undo work already completed. With XSUB, queue exhaustion or an unusable queue returns buffered-input requests to the keyboard.

Command Execution

SUBMIT commands execute in the normal BetterCP/M command environment.

A command may load a transient program, invoke a resident command, or use an installed extension.

Ordinary warm boots preserve the batch-processing mechanisms needed to continue supported SUBMIT operations.

A cold boot establishes a new operating session rather than continuing a previous batch.

Interactive Commands

A command executed from a SUBMIT file does not automatically become noninteractive.

For example, transient `COPY.COM` may ask whether an existing writable destination file should be replaced. Resident COPY instead refuses the collision unless `/O` is supplied. Use `:COPY` to select the transient utility explicitly.

If the batch is intended to run unattended, use the command's supported options to specify the desired behavior.

For example:

```
:COPY B1:*.DAT C3: /B /O
```

uses COPY's batch option and requests replacement of existing writable destination files.

The `/B` option belongs to COPY. It does not change SUBMIT's behavior or make other commands noninteractive.

Errors and Interruptions

Errors during batch execution may interrupt the procedure or affect subsequent commands.

The exact behavior depends on the command and the error encountered.

A failed command does not necessarily undo earlier successful operations.

For example, if a batch copies several files and encounters an error partway through, some destination files may already have been created.

Before restarting an interrupted batch, examine the results to determine which operations have completed.

Startup Files

BetterCP/M uses SUBMIT for its standard startup procedure.

The default cold-start command is:

```
SUBMIT STARTUP
```

This executes `STARTUP.SUB`.

Section 7.3 lists the supplied scripts in order. cpmsim displays the banner only; Model 4 already loads M4SCRN and ADM3A, clears the screen, then displays the banner. Edit `A0:STARTUP.SUB` to customize that sequence.

The startup command is executed during a cold boot, not during an ordinary warm boot.

For information about configuring startup behavior, see Chapter 7.

Examples

Execute a command file:

```
A0>SUBMIT BACKUP
```

Execute a command file with one parameter:

```
A0>SUBMIT BUILD TEST
```

Execute a command file with two parameters:

```
A0>SUBMIT COPYFILE REPORT.TXT B:REPORT.TXT
```

For an introduction to batch processing, see Chapter 6.

17.2 XSUB - Extended Batch Input

Purpose: Allows compatible programs to receive input from a SUBMIT file through the CP/M buffered console-input interface.

Ordinary SUBMIT processing supplies commands to the command processor.

Some programs, however, request additional input after they have started.

For example, an editor or utility might ask for a filename or an option.

XSUB extends batch processing by allowing submitted input to be supplied to such programs.

How XSUB Works

XSUB provides a resident input facility that cooperates with SUBMIT.

When a compatible program requests buffered console input, XSUB can supply the next submitted input line.

This allows certain interactive procedures to be automated.

For example, a program that normally asks:

```
Source file:
Destination file:
```

may be able to receive those responses from the batch file.

XSUB does not automatically support every interactive program.

Programs that read the keyboard directly, use private input routines, or bypass the supported BDOS console-input interface may still require ordinary keyboard input.

Using XSUB

The activation command is:

```
XSUB
```

Place it before the program to be automated. `XSUB.COM` belongs to the **Utilities Disk**, not the current System Disk. Its protected companion, `BATCHIO.RSX`, must be in A0, along with the matched-build `R3PACK.RSX` manager runtime. If your utility media lacks BATCHIO, obtain the matching companion before using XSUB. The current distribution manifest assigns XSUB to the Utilities Disk but does not separately specify where BATCHIO is packaged.

Before running the example, make `XSUB.COM`, `ED.COM`, `TYPE.COM`, and `SUBMIT.COM` available on the command search path and `BATCHIO.RSX` and `R3PACK.RSX` available in A0. Keep A0 writable for the shared queue, and use a writable working drive/user where `XDEMO.TXT` does not already exist.

Save this complete procedure as `XDEMO.SUB` in the working location:

```
XSUB
ED XDEMO.TXT
IHELLO FROM XSUB^L^Z
E
:TYPE XDEMO.TXT
```

Type the caret sequences `^L` and `^Z` as two ordinary source characters each; SUBMIT converts them to control bytes. Run:

```
A0>SUBMIT XDEMO
```

XSUB activates BATCHIO. ED then receives `IHELLO FROM XSUB`, followed by the expanded controls, as its first buffered command line. Inline `I` inserts the text, Ctrl-L adds a line ending, and Ctrl-Z ends the inline insertion. ED receives `E` as its next command, saves the new file, and exits. The command processor resumes the remaining queue and TYPE displays:

```
HELLO FROM XSUB
```

The response lines belong immediately after the program invocation because each BDOS Function 10 request consumes the next queue line. They are not separate CCP commands while the program is reading them. Supply exactly the responses the program will consume; leftover responses can otherwise be interpreted as commands, and extra input requests can consume intended later commands.

This example deliberately uses ED's inline insertion command. ED's interactive `I` mode reads individual characters through Function 1, which XSUB does not supply. XSUB supports **BDOS Function 10 buffered console input**, not arbitrary keyboard input.

Returning to Keyboard Input

When the submitted input has been exhausted, XSUB allows the program to continue receiving input from the keyboard.

For example, a program may receive its initial responses automatically and then continue interactively.

This is useful when only the repetitive portion of an operation needs to be automated.

A malformed or unusable submitted-input queue is discarded before returning to ordinary console input.

Resident BATCHIO Support

BetterCP/M's XSUB facility uses a resident component named `BATCHIO`.

This component may remain installed after the batch has finished.

Under the supported warm-boot conditions, an idle BATCHIO provider can retire automatically.

However, some program-termination paths may leave it resident.

You can examine installed RSXs with:

```
A0>RSX LIST
```

To remove BATCHIO explicitly:

```
A0>RSX UNLOAD BATCHIO
```

This releases its resident memory when it is no longer needed.

Memory Considerations

Like other resident extensions, BATCHIO occupies memory that would otherwise be available to transient programs.

Unloading it when no longer needed can restore available program memory.

The exact amount recovered depends on the other resident extensions installed.

Limitations

XSUB is intended for programs that use compatible buffered console-input services.

It is not a general keyboard-recording or keyboard-simulation utility.

It does not guarantee automation of programs that communicate directly with keyboard hardware or use unsupported input methods.

For ordinary command sequences that do not require input to running programs, SUBMIT alone is sufficient.

For an introduction to XSUB and batch processing, see Chapter 6.

17.3 CPX - Command Processor Extension Manager

Purpose: Displays, loads, and unloads Command Processor Extensions.

A Command Processor Extension, or CPX, adds commands to BetterCP/M's command environment.

A CPX remains available between commands in the active command environment. Its command code can be reclaimed by transient programs and is reconstructed on warm boot. This allows frequently used commands to be provided at the prompt without loading a separate `.COM` file each time.

The standard BetterCP/M command environment includes `RCP.CPX`, which supplies familiar commands such as DIR, ERA, REN, TYPE, COPY, and MOVE.

The CPX manager allows additional command packages to be installed or removed.

Command Forms

The principal CPX management commands are:

```
CPX LIST
CPX LOAD name
CPX UNLOAD name
```

The `name` argument is an unqualified CPX filename stem, with optional `.CPX`. Blank names, drive qualifiers, wildcards and other extensions are rejected.

For example:

```
A0>CPX LOAD TOOLS
```

requests the command package `TOOLS.CPX`.

Listing CPXs

To examine the configured command processor extensions, enter:

```
A0>CPX LIST
```

BetterCP/M displays information about the configured CPX packages.

This is useful when determining which command packages are available or investigating a missing resident command.

An important distinction is that **a configured CPX is not necessarily a successfully loaded CPX**.

BetterCP/M may retain an extension's configured entry even when the corresponding file cannot be loaded.

Consequently, the presence of a name in `CPX LIST` does not by itself establish that its commands are available.

Loading a CPX

To load an additional command package, enter:

```
CPX LOAD name
```

For example:

```
A0>CPX LOAD TOOLS
```

BetterCP/M attempts to load the specified extension.

The file must be available as `A0:TOOLS.CPX` and compatible with the running system. Keep configured carriers in A0 for reconstruction after warm boot or transient execution.

If loading succeeds, the commands supplied by the extension become available through the normal command environment.

For example, if `TOOLS.CPX` provides a command named `TOOLS`, that command can be invoked directly:

```
A0>TOOLS
```

The actual commands provided depend on the extension.

Loading Several CPXs

BetterCP/M maintains an ordered **four-entry reconstruction profile**. The standard RCP entry occupies one slot, leaving three for additional packages. A configured entry still occupies its slot if its carrier is missing or fails validation.

`CPX LOAD name` appends a new name after the existing entries. Loading an already configured name is a successful no-op: it neither adds another entry nor changes the order. A fifth distinct entry is rejected without changing the profile.

Successfully reconstructed CPXs are consulted in profile order after the CCP's built-in commands and before transient lookup. The first CPX that handles a command wins. Appending a package therefore does not give it precedence over an earlier package providing the same command. `CPX LIST` shows the configured order.

`CPX UNLOAD name` removes the named entry and shifts later entries left, preserving their relative order. Unloading an absent name is a successful no-op. To deliberately reorder packages, remove and reload the affected entries in the desired order, keeping the four-entry limit in mind.

If you need to execute a particular transient utility regardless of the resident command environment, use BetterCP/M's explicit transient-execution prefix:

```
A0>:DIR
```

This bypasses resident command lookup.

For more information about command search and execution, see Chapter 5.

Unloading a CPX

To remove an optional command package, enter:

```
CPX UNLOAD name
```

For example:

```
A0>CPX UNLOAD TOOLS
```

After the extension has been removed, its commands are no longer available as resident commands.

Corresponding transient programs, if present, remain available through the ordinary program-search mechanism.

Unloading a CPX can be useful when you no longer need its commands or want to change the resident command environment.

Removing the Standard Command Package

RCP is part of the selected standard environment, but the current CPX profile manager does not reserve or lock its entry. `CPX UNLOAD RCP` is allowed. Removing it takes away commands supplied by RCP, including resident DIR, ERA, REN, TYPE, COPY, MOVE, CLS and VER. Built-in CCP commands and available transient utilities remain usable; a transient is available only if its file has been installed.

For example, the System Disk supplies `DIR.COM` but does not supply `CLS.COM`. To restore the normal command package, keep `A0:RCP.CPX` available and enter `CPX LOAD RCP`. It is appended in the usual way, so check the resulting order with `CPX LIST`.

Recovering from an Unusable CPX

If an optional CPX file fails validation, BetterCP/M may display:

```
CPX?
```

The operating system continues to the command prompt with its remaining usable extensions.

Commands supplied by the rejected CPX are unavailable.

The rejected extension may nevertheless remain in the configured profile.

For example:

```
A0>CPX LIST
```

may still show the extension.

To remove an unusable optional CPX, enter:

```
A0>CPX UNLOAD BAD
```

where `BAD` is the name of the affected extension.

Alternatively, replace the damaged file with a valid copy.

Until the problem is corrected, the warning may recur when BetterCP/M restores the command environment.

A missing optional CPX file may simply be skipped without producing the same warning.

CPXs and Available Memory

Command Processor Extensions occupy memory in the active command environment, but their command code is reclaimable, along with the CCP, when a transient program starts. BetterCP/M shuts down the active CPX chain before transferring control and reconstructs the command environment on warm boot.

Keeping a CPX configured makes its commands available at the prompt without permanently subtracting its command-code size from the maximum transient program size. RSXs, by comparison, occupy protected

memory while applications run.

Use `STAT MEM` to see the current CCP, CPX, and RSX regions and the maximum loadable COM size. The maximum can include the reclaimable CCP and CPX regions shown in the map.

CPX Examples

List configured command extensions:

```
A0>CPX LIST
```

Load an optional extension:

```
A0>CPX LOAD TOOLS
```

Unload an optional extension:

```
A0>CPX UNLOAD TOOLS
```

Request a transient utility rather than a resident command:

```
A0>:DIR /WHEREIS ED.COM
```

For an introduction to CPXs and their relationship to the command environment, see Chapter 8, *Resident Extensions*.

17.4 RSX - Resident System Extension Management

Purpose: Lists, loads, and unloads Resident System Extensions (RSXs).

Resident System Extensions provide services that remain available while other programs execute. Unlike Command Processor Extensions, which add commands to the command environment, RSXs extend the services available to applications and the operating system.

BetterCP/M uses RSXs for several purposes, including terminal emulation, clock support, batch input, and other optional system facilities.

The RSX utility allows you to manage these extensions without rebuilding the operating system.

Normal carrier loading reads `A0:name.RSX`, regardless of the current drive/user. Keep the matched-build `R3PACK.RSX` runtime container in A0 too; it is required by the manager, not an extension to load with `RSX LOAD`. See Section 8.2 for installation details.

Syntax

The principal command forms are:

```
RSX LIST
RSX LOAD name
RSX UNLOAD name
```

The `name` argument identifies the extension to be loaded or unloaded. The `.RSX` extension is normally omitted.

For example:

```
A0>RSX LOAD ADM3A
```

loads the ADM-3A terminal personality from `ADM3A.RSX`.

Listing Installed RSXs

To display the currently installed Resident System Extensions, enter:

```
A0>RSX LIST
```

BetterCP/M displays the installed extensions.

This is useful when determining which system services are available, checking whether a particular extension is loaded, or investigating memory usage.

For example, you might use RSX LIST to determine whether a terminal personality or clock provider is installed before running an application that requires it.

Loading an RSX

To load an extension, enter:

```
RSX LOAD name
```

For example:

```
A0>RSX LOAD FREHDCLK
```

loads the FreHD clock provider.

Once successfully loaded, the provider makes its supported clock services available to programs using BetterCP/M's clock interface.

Another example is:

```
A0>RSX LOAD VT52
```

which loads the VT52 terminal personality.

Some extensions require other resident components to be loaded first.

The extension's documentation identifies these requirements.

Extension Dependencies

Some RSXs depend on services provided by other extensions.

BetterCP/M's terminal-monitor system provides a common example.

A terminal personality interprets the control sequences used by an application, while a screen backend communicates with the actual display environment.

The backend must be loaded before the terminal personality.

For example, on a TRS-80 Model 4:

```
A0>RSX LOAD M4SCRN
A0>RSX LOAD ADM3A
```

The first command loads the Model 4 screen backend.

The second loads the ADM-3A terminal personality.

On a system using the ANSI screen backend, the corresponding commands are:

```
A0>RSX LOAD ANSISCRN
A0>RSX LOAD ADM3A
```

These commands establish the supported ADM-3A display environment using the appropriate backend.

Unloading an RSX

To remove an installed extension, enter:

```
RSX UNLOAD name
```

For example:

```
A0>RSX UNLOAD ADM3A
```

removes the ADM-3A terminal personality.

Once the extension has been removed, its services are no longer available.

Unloading an extension also releases the memory occupied by that extension when the resident manager can reclaim it.

Unloading Dependent Extensions

When several extensions depend on one another, remove them in the reverse order from which they were loaded.

For example:

```
A0>RSX UNLOAD ADM3A
A0>RSX UNLOAD M4SCRN
```

This removes the terminal personality before removing its supporting screen backend.

Attempting to remove a supporting extension while another component still requires it may be rejected.

If RSX reports that an extension cannot be unloaded, examine the installed extensions and their dependencies before trying again.

Changing Terminal Personalities

Only one terminal personality can be active at a time.

To change from ADM-3A to VT52 while retaining the same screen backend:

```
A0>RSX UNLOAD ADM3A
A0>RSX LOAD VT52
```

The screen backend need not be reloaded if it remains installed and supports the new personality.

To verify the new environment, run the corresponding terminal test utility:

```
A0>VT52TST
```

For further information about terminal personalities, see Chapter 9, *Terminal Support*.

Clock Providers

BetterCP/M provides loadable clock providers through the RSX system.

For example, on a TRS-80 Model 4 with FreHD:

```
A0>RSX LOAD FREHDCLK
A0>TIME
```

Under z80pack:

```
A0>RSX LOAD ZPRTC
A0>TIME
```

These providers allow TIME and compatible applications to obtain the current date and time.

Both providers supplied with BetterCP/M 1.0 are read-only.

You can identify the active provider with:

```
A0>TIME /PROVIDER
```

For more information, see Chapter 11, *Date and Time*, and Section 16.3, *TIME*.

P2DOS Clock Compatibility

The optional `P2DOS.RSX` implements compatibility with the P2DOS date-and-time interface. It is not on the selected System Disk; the following example requires a compatible separately built carrier installed in A0.

For example:

```
A0>RSX LOAD ZPRTC
A0>RSX LOAD P2DOS
```

The first command loads the native clock provider.

The second installs the P2DOS-compatible interface.

The P2DOS extension uses BetterCP/M's native clock service rather than communicating directly with the underlying clock hardware.

Consequently, its capabilities depend on the installed provider.

With the standard read-only providers, clock-setting requests are not supported.

Batch Input Support

BetterCP/M's XSUB facility uses a resident extension named `BATCHIO`.

This extension allows compatible programs to receive submitted input through the CP/M buffered console-input interface.

BATCHIO may remain resident after the batch procedure has finished.

To check whether it is installed:

```
A0>RSX LIST
```

To remove it explicitly:

```
A0>RSX UNLOAD BATCHIO
```

This releases the resident batch-input facility when it is no longer needed.

For additional information, see Section 17.2, *XSUB*.

RSXs and Program Memory

Resident System Extensions occupy memory that would otherwise be available to transient programs.

Loading an extension can therefore reduce the available Transient Program Area (TPA).

For example, a terminal monitor may be useful while running a full-screen editor but unnecessary when running an ordinary command-line utility.

If a program reports insufficient memory, examine the installed RSXs:

```
A0>RSX LIST
```

You may be able to unload extensions that are no longer needed.

Some utilities have substantial memory requirements. DUP, for example, requires approximately 51K of available transient memory.

Unloading unnecessary extensions may provide sufficient space for such programs.

Warm Boots and Extension Persistence

BetterCP/M preserves supported resident extensions across ordinary warm boots.

For example, suppose you load a terminal monitor:

```
A0>RSX LOAD M4SCRN
A0>RSX LOAD VT52
```

You can then run a compatible full-screen application.

When the application terminates and control returns to the command processor, the terminal monitor remains available.

There is ordinarily no need to reload it after every program.

A cold boot starts a new session. Use the supplied script and editing procedure in Section 7.3 to arrange automatic loading; normal carrier placement and dependencies are described in Section 8.2.

Extension Errors

An RSX may fail to load because its file is missing, damaged, incompatible, or unsuitable for the current system.

A load may also fail because the required supporting services are unavailable.

If an extension fails to load, check that its file is present and that any required supporting extensions have already been installed.

For example, a terminal personality requires a suitable screen backend.

If an extension cannot be unloaded, another resident component may still depend on it.

Remove the dependent extension first.

Recovering from Extension Problems

If an RSX causes problems, begin by examining the installed extensions:

```
A0>RSX LIST
```

If the affected extension can be safely removed, unload it.

For example:

```
A0>RSX UNLOAD VT52
```

If the problem began after adding an extension to `STARTUP.SUB`, remove the corresponding startup command and test the extension manually.

A cold boot can restore the configured startup environment, but it will also execute the configured startup procedure again.

If that procedure contains an unsuitable extension-loading command, the problem may recur until the startup file is corrected.

Examples

List installed RSXs:

```
A0>RSX LIST
```

Load a terminal screen backend:

```
A0>RSX LOAD M4SCRN
```

Load an ADM-3A terminal personality:

```
A0>RSX LOAD ADM3A
```

Change to VT52:

```
A0>RSX UNLOAD ADM3A
A0>RSX LOAD VT52
```

Load the FreHD clock provider:

```
A0>RSX LOAD FREHDCLK
```

Load z80pack clock support:

```
A0>RSX LOAD ZPRTC
```

Load P2DOS-compatible clock services after separately installing the optional `P2DOS.RSX` in A0 and loading a native clock provider:

```
A0>RSX LOAD P2DOS
```

Remove the batch-input extension:

```
A0>RSX UNLOAD BATCHIO
```

Unload a terminal monitor:

```
A0>RSX UNLOAD VT52
A0>RSX UNLOAD M4SCRN
```

For an introduction to resident extensions and their use, see Chapter 8, *Resident Extensions*.

For detailed information about developing RSXs, consult the BetterCP/M RSX and CPX Programmer's Guide.

Appendices

Appendix A - Command Summary

Use this appendix to find a command form or switch, then follow its reference for defaults, limits and protection rules. Bracketed syntax is optional; source/destination and filenames are placeholders. A leading `:` explicitly selects a transient utility. See Section 13.1 for notation and Section 13.3 for command selection.

A.1 Command Syntax and Switches

Command	Common syntax	Options or limits to check	Reference
CLS	`CLS`	Resident in RCP; transient is separately buildable	§15.2
CONFIG	`CONFIG`, `CONFIG /SHOW`	Startup: `/STARTUP="command"`, `/DEFAULT`; saved byte: `/IOBYTE=n`; disk forms below	§16.1
COPY	`COPY source destination`, `COPY destination=source`	Resident `/O`; transient `:COPY` adds `/B`, `/S`, `/V`, `/BACKUP`; `/O` and `/S` are exclusive	§14.5
CPX	`CPX LIST`, `CPX LOAD name`, `CPX UNLOAD name`	Four profile entries including RCP; append order	§17.3
DIR	`DIR [filespec]`	Transient `/A`, `/P`, `/Z=K`, `/Z=S`, `/Z=E`, `/C=1`, `/C=2`, `/C=4`, `/S=N`, `/S=T`, `/S=Z`, `/S=U`	§14.1
DIR locator	`:DIR /WHEREIS filespec`	Exactly one operand; global default scope; do not combine with ordinary display switches	§14.1
DUP	`DUP`, `DUP /FORMAT d:`, `DUP /COPY s: d:`, `DUP /CHECK d:`	Physical drive 0 is protected against FORMAT and COPY destinations	§16.4
ERA	`ERA filespec`, `:ERA [options] filespec`	Transient `/Y`, `/Q`, `/C`, `/R`, `/D`; matching-branch protection still applies	§14.2
GET	`GET filename`, `GET address filename`	Hexadecimal address; returns without execution	§15.5
GO	`GO`	Executes at `0100H`	§15.6
JUMP	`JUMP address`	Hexadecimal address	§15.7
MOVE	`MOVE source destination`, `MOVE destination=source`	Deletes source only after successful copy; no destination overwrite	§14.6
PEEK / P	`PEEK`, `PEEK address`, `PEEK first last`	Hexadecimal addresses; `P` is the short name	§15.8
POKE	`POKE address byte [byte ...]`	Hexadecimal bytes; a double quote starts literal text	§15.9
REN	`REN source template`, `REN template=source`	Transient `/Y`, `/C`, `/P`, `/DRY`; preview with `/DRY`	§14.3
RSX	`RSX LIST`, `RSX LOAD name`, `RSX UNLOAD name`	Carriers and matched `R3PACK.RSX` in A0; respect dependencies	§17.4
SAVE	`SAVE pages filename`	Decimal count of 256-byte pages from `0100H`	§15.4
STAT	`STAT [filespec]`, `STAT MEM`	File options `$S`, `$R/O`, `$R/W`, `$SYS`, `$DIR`; status/device forms below	§16.2
SUBMIT	`SUBMIT filename [parameters]`	`$1`-`$9`, literal `$$`, 125 expanded bytes per line, 2K aggregate buffer; writable A0 queue	§17.1

Command	Common syntax	Options or limits to check	Reference
SYSGEN	`SYSGEN source: target:`, `SYSGEN source:`, `SYSGEN package target:`	Install from disk/package or export installed system; target geometry must match	§16.5
TIME	`TIME`, `TIME /PROVIDER`, `TIME /SET LL HH HR MN SC`	SET takes five hexadecimal bytes; supplied providers are read-only	§16.3
TYPE	`TYPE filespec`, `:TYPE filespec /P`	`/P` selects transient paging	§14.4
USER	`USER n`	Decimal user 0-31; `B:`, `5:` and `C3:` also navigate	§15.1
VER	`VER`, `VER /V`	Transient `:VER /BANNER` displays the startup banner	§15.3
XSUB	`XSUB` before the program invocation in a `.SUB` procedure	Function-10 buffered input only; Utilities Disk component; full example in §17.2	§17.2

A.2 Configuration and Status Forms

Task	Syntax or choices	Reference
Physical profile	`CONFIG /PHYSICAL=n /TRACKS=80 /SIDES=2 /STEP=6`	§16.1
Bind a logical drive	`CONFIG /LOGICAL=C /PHYSICAL=1 /FORMAT="MM 80T DS DATA"`	§16.1
Unbind	`CONFIG /LOGICAL=C /UNBIND`	§16.1
Edit an active binding	`CONFIG /EDIT=B /CKS=16`	§16.1
Edit one physical-order map slot	`CONFIG /EDIT=B /SLOT=1 /ID=200`	§16.1
Save active disk settings	Interactive CONFIG: **S**, then **B**; **A** saves pending cold-boot changes only	§16.1
Disk characteristics / users	`STAT DSK:`, `STAT B:DSK:`, `STAT USR:`	§16.2
Temporary drive read-only	`STAT B:=R/O` (cleared by normal warm-start/reset)	§16.2
Device inspection / legal values	`STAT DEV:`, `STAT VAL:`, `STAT CRT:` (inverse query)	§16.2
Live list-device assignment	`STAT LST:=CRT:`	§16.2
File protection / visibility	`STAT REPORT.TXT $R/O`, `$R/W`, `$SYS`, or `$DIR`	§16.2
Memory use	`STAT MEM`	§16.2

CONFIG disk commands change active state; saving is separate. Use the field tables in Section 16.1 for accepted values and dependencies. Bracketed multi-DU STAT selectors support inspection, optionally `$S`, rather than attribute changes. Double literal dollars when putting STAT or attribute selectors in a `.SUB` source.

A.3 Resident and Transient Commands

Use `:COMMAND` or `.COMMAND` to bypass resident lookup. BetterCP/M does not automatically hand an unsupported resident option to a transient. DIR, ERA, REN, TYPE, COPY and VER have selected System Disk transients; `CLS.COM` is separately buildable. See Section 13.3 for the exact lookup rules and Section 3.2 for disk contents.

A.4 Short Examples

Find a utility, preview a rename, or copy with explicit collision and verification policy:

```
A0>:DIR /WHEREIS ED.COM
A0>:REN /DRY B[-]:*.DOC *.TXT
A0>:COPY /B /O /V B1:*.DAT C3:
```

`/O` can remove an existing writable target without rollback if the replacement fails; preserve important old targets separately. ERA's protection admission and `/Y`/`/R` rules are in Section 14.2. Complete worked batch and XSUB procedures are in Chapter 6 and Section 17.2.

A.5 Utility Version Information

Most utility programs accept standalone `/VER`, for example `CONFIG /VER`. Use `:DIR /VER` for the transient DIR identity. `VER` identifies the operating system; `VER /V` adds subsystem detail. See Section 13.4 for version conventions and restrictions.

A.6 Additional Utilities

Section 3.2 lists selected System Disk files, the separate Utilities Disk, and separately obtained or buildable tools. Appendix H links the available utility, programming and build documentation. Implementation and development-disk presence do not establish distribution membership.

Appendix B - Command-Line Editing Keys

BetterCP/M provides command-line editing and command-history facilities that make it easier to enter, correct, and reuse commands.

Instead of retyping an entire command after making a mistake, you can move the cursor to the error, correct it, and continue typing. You can also recall previously entered commands, modify them, and execute them again.

BetterCP/M uses editing conventions familiar to users of WordStar and other CP/M programs, although some assignments differ.

This appendix summarizes the command-line editing facilities described in Chapter 5.

B.1 Entering and Editing Commands

Commands are entered at the BetterCP/M command prompt.

For example:

```
A0>COPY REPORT.TXT B:
```

Press **Return** to execute the command.

Before pressing Return, you can edit the command line. Editing changes only the command being entered; it does not affect files or execute any part of the command.

Basic Editing Operations

The command editor provides facilities for moving the cursor, inserting characters, and deleting unwanted text.

For example, suppose you've typed:

```
A0>COPY REPROT.TXT B:
```

You can move the cursor back to the misspelled filename, correct it to `REPORT.TXT`, and then press Return.

You don't need to erase and reenter the entire command.

Editing-Key Conventions

BetterCP/M follows several conventions familiar to users of WordStar-style editors.

However, the command editor is not a complete WordStar editor, and its key assignments should not be assumed to match WordStar in every respect.

The exact assignments for cursor movement and line editing should be taken from the command processor's supported keyboard interface.

The key combinations listed below are those established for BetterCP/M 1.0.

B.2 Command History

BetterCP/M maintains a history of previously entered commands.

You can recall an earlier command, edit it, and execute it again without retyping the entire line.

The command-history controls in BetterCP/M 1.0 are:

Key	Function
Ctrl-K	Recall the previous command
Ctrl-J	Move forward to a newer command

Key	Function
Return	Execute the displayed command

For example, suppose you've entered:

```
A0>COPY REPORT.TXT B:
```

To repeat the command, press Ctrl-K to recall it.

You can then press Return to execute it again or edit the command before executing it.

To move farther back through the history, press Ctrl-K again.

To move toward more recently entered commands, press Ctrl-J.

Arrow Keys

The original Model 4 keyboard supplies these command-editor aliases:

Physical key	Function
Left / Right	Move one character left / right
Up / Down	Previous / next command in history (Ctrl-K / Ctrl-J)
Clear	Toggle insert/overwrite (Ctrl-V)
Shift-Left / DEL	Delete the character left of the cursor (Ctrl-H)

Physical Left is distinct from Ctrl-H. Ctrl-E and Ctrl-X retain physical-line continuation and line-discard behavior; they do not navigate history.

Host-emulator keyboard translation and other console keyboards may produce different codes or escape sequences. The Model 4 assignments describe the original machine's keyboard signals. Use Ctrl-K/J for history if host arrow keys do not behave as expected.

Editing Recalled Commands

A recalled command can be edited just like a newly entered command.

For example, suppose the history contains:

```
A0>COPY REPORT.TXT B:
```

You could recall it, change the destination drive, and execute:

```
A0>COPY REPORT.TXT C:
```

The recalled command is not executed until you press Return.

History Persistence

Command history is maintained by the command environment rather than by individual transient programs.

It remains available when a program finishes and returns control to the command processor.

It also survives an ordinary warm boot.

A cold boot begins with an empty command history.

History storage is limited. As new commands are entered, older entries are eventually discarded.

B.3 Control Keys and Terminal Differences

Control-key combinations are entered by holding Ctrl while pressing the corresponding key.

For example, Ctrl-K means holding Ctrl and pressing K.

On most terminals, control-key combinations produce character codes rather than distinct keyboard events.

This distinction matters because different keyboards and terminal emulators may generate different codes for the same physical key.

Physical Keys and Control Characters

An arrow key is not necessarily equivalent to a particular control-key combination.

For example, one computer may generate a single control character when its Up arrow is pressed, while a terminal emulator may transmit a sequence of several characters.

BetterCP/M can recognize a key only if the console environment supplies a representation that its command editor understands.

Consequently, the behavior of physical cursor keys can vary between platforms.

WordStar Conventions

Some command-editor controls follow WordStar conventions, but Ctrl-E/X keep CP/M continuation/discard behavior. Use the complete editor table in Section B.5. Section C.4 describes traditional application-level WordStar controls.

B.4 Keyboard Differences Between Platforms

BetterCP/M runs on computers with different keyboard arrangements.

Some provide dedicated cursor keys, while others rely heavily on control-key combinations.

Terminal emulators may also translate host keyboard input before passing it to the emulated computer.

For example, a modern keyboard may provide characters or keys that did not exist on the original computer.

The emulator must translate those inputs into codes recognized by the emulated hardware or operating system.

TRS-80 Model 4 Keyboard

The TRS-80 Model 4 keyboard lacks several punctuation characters found on modern keyboards.

BetterCP/M retains Montezuma Micro-style keyboard combinations for entering these characters.

Character	Model 4 key combination
[	Ctrl-8
]	Ctrl-9
{	Ctrl-comma
}	Ctrl-period
\	Ctrl-/
|	Ctrl-1
~	Ctrl-3
^	Ctrl-5
`	Ctrl-7
_	Ctrl-minus

These combinations refer to the original Model 4 keyboard.

When using an emulator, the host keyboard may require different physical keys to produce the corresponding Model 4 key signals.

Shift may also be held with the indicated control-key combinations.

Terminal Emulators

When using BetterCP/M through a terminal emulator, the emulator's keyboard configuration can affect which control characters reach the operating system.

If a key does not behave as expected, check the terminal emulator's keyboard settings.

In particular, do not assume that its arrow keys produce the same control codes as the original computer's cursor keys.

For command-history navigation, Ctrl-K and Ctrl-J remain the documented BetterCP/M 1.0 controls.

B.5 Quick Reference

These assignments apply while the BetterCP/M command editor owns input at the prompt. Applications can assign different meanings to the same controls.

Key	Function
Return	Execute the current command
Ctrl-S / Ctrl-D	Move one character left / right
Ctrl-A / Ctrl-F	Move to the previous / next word
Ctrl-T	Delete the next word and adjacent spacing
Ctrl-V	Toggle insert/overwrite
Ctrl-G	Delete the character under the cursor
Ctrl-H	Delete the character left of the cursor
Ctrl-U / Ctrl-X	Discard the complete command line and redraw an empty prompt
Ctrl-E	Continue the same command on a new physical screen line; do not execute it
Ctrl-K / Ctrl-J	Previous / next command in history
Ctrl-C	Abandon input, echo `^C`, and warm boot
Ctrl-P	Toggle shared console printer echo to `LST:`

Ctrl-S moves the cursor at the prompt. During cooked console output it pauses scrolling until the next key; the resume key is consumed. Ctrl-P changes the shared console printer-echo state, so its effect can continue into transient programs.

For the control characters used by CP/M programs and terminal devices, see Appendix C, *Control Characters and Keyboard Conventions*.

Appendix C - Control Characters and Keyboard Conventions

CP/M was designed to operate with a wide variety of keyboards, consoles, printers, and terminals. Because these devices differed considerably, many programs relied on ASCII control characters rather than dedicated function keys.

BetterCP/M retains the traditional CP/M character-oriented console interface while providing additional facilities for command-line editing, command history, and terminal support.

This appendix describes the control characters commonly encountered when using BetterCP/M, explains their relationship to keyboard input, and identifies important differences between conventional CP/M behavior and BetterCP/M's command environment.

For command-line editing keys, see Appendix B. For information about terminal emulation, see Chapter 9, *Terminal Support*.

C.1 Control Characters

A control character is a character code traditionally used to perform an operation rather than display a printable symbol.

ASCII defines 32 control characters, numbered `00H` through `1FH`, together with the Delete character (`7FH`).

These include familiar operations such as carriage return, line feed, backspace, and horizontal tab.

On many keyboards, a control character can be entered by holding the Ctrl key while pressing a letter.

For example:

```
Ctrl-M
```

produces ASCII carriage return (`0DH`) on a conventional ASCII keyboard.

Similarly:

```
Ctrl-J
```

produces ASCII line feed (`0AH`).

The notation `^M` is often used as an abbreviation for Ctrl-M, particularly in CP/M documentation and program displays.

Thus, `^M`, Ctrl-M, and ASCII `0DH` refer to the same character code.

ASCII Control Character Table

Hex	Decimal	Name	Common notation
00	0	Null	NUL, ^@
01	1	Start of Heading	SOH, ^A
02	2	Start of Text	STX, ^B
03	3	End of Text	ETX, ^C
04	4	End of Transmission	EOT, ^D
05	5	Enquiry	ENQ, ^E
06	6	Acknowledge	ACK, ^F
07	7	Bell	BEL, ^G
08	8	Backspace	BS, ^H
09	9	Horizontal Tab	HT, ^I
0A	10	Line Feed	LF, ^J
0B	11	Vertical Tab	VT, ^K
0C	12	Form Feed	FF, ^L
0D	13	Carriage Return	CR, ^M
0E	14	Shift Out	SO, ^N
0F	15	Shift In	SI, ^O
10	16	Data Link Escape	DLE, ^P
11	17	Device Control 1	DC1, ^Q
12	18	Device Control 2	DC2, ^R
13	19	Device Control 3	DC3, ^S
14	20	Device Control 4	DC4, ^T
15	21	Negative Acknowledge	NAK, ^U
16	22	Synchronous Idle	SYN, ^V
17	23	End of Transmission Block	ETB, ^W
18	24	Cancel	CAN, ^X
19	25	End of Medium	EM, ^Y
1A	26	Substitute	SUB, ^Z
1B	27	Escape	ESC, ^[
1C	28	File Separator	FS, ^\
1D	29	Group Separator	GS, ^]
1E	30	Record Separator	RS, ^^
1F	31	Unit Separator	US, ^_
7F	127	Delete	DEL, ^?

The names in this table are the standard ASCII names. They do not imply that BetterCP/M assigns a special function to every character.

The meaning of a control character depends on the program or console facility receiving it.

For example, Ctrl-K has a particular meaning in BetterCP/M's command-history editor, but another program may interpret the same character differently.

C.2 Common CP/M Control Characters

Several control characters have established uses in the traditional CP/M operating environment.

These conventions are important because many CP/M applications expect them.

Ctrl-C - Interrupt or Warm Boot

Ctrl-C (`03H`) is traditionally associated with interrupting an operation or returning to the command processor.

In conventional CP/M, its effect depends on the console-input function and the program's handling of keyboard input.

BetterCP/M retains the familiar Ctrl-C convention while allowing individual utilities to provide their own interruption behavior.

For example, Ctrl-C can interrupt a DUP operation between disk operations.

In other programs, Ctrl-C may terminate the current command and return control to the command processor.

However, **Ctrl-C does not have identical behavior in every program**.

Some utilities interpret it as part of their own interactive controls.

For example, transient TYPE.COM uses Ctrl-C to advance to the next page while waiting at its paging prompt.

Consult the individual utility reference when the distinction matters.

Ctrl-H - Backspace

Ctrl-H (`08H`) is the ASCII backspace character.

It is commonly used by character-oriented consoles to move the cursor backward or erase a character during input.

The precise effect depends on the input routine and the console environment.

Some keyboards provide a Backspace key that generates Ctrl-H. Others may generate DEL (`7FH`).

Applications may recognize one or both.

Ctrl-I - Horizontal Tab

Ctrl-I (`09H`) is the ASCII horizontal-tab character.

In text files and console output, a tab normally advances to the next tab stop.

The tab width depends on the program or terminal.

For example, BetterCP/M's transient TYPE utility can expand tabs to eight-column stops using:

```
A0>:TYPE README.TXT /T
```

With `/CTL`, TYPE displays a tab as `^I` instead of expanding it.

Ctrl-J - Line Feed

Ctrl-J (`0AH`) is ASCII line feed.

Historically, line feed advances the output position to the next line without necessarily returning the cursor to the beginning of that line.

In BetterCP/M 1.0's command editor, Ctrl-J is also assigned to moving forward through command history.

This is a command-editor convention, not a redefinition of ASCII line feed.

Other programs may continue to interpret Ctrl-J as line feed.

Ctrl-M - Carriage Return

Ctrl-M (`0DH`) is ASCII carriage return.

Carriage return traditionally moves the output position to the beginning of the current line.

On CP/M consoles, pressing Return normally submits the current command or input line.

Text files commonly use carriage return followed by line feed (`0DH 0AH`) to mark the end of a line.

Ctrl-P - Printer Echo

In conventional CP/M, Ctrl-P is associated with printer echo, directing console output to the list device as well as the console.

This behavior depends on the operating system's console handling and the available list-device support.

Programs that bypass the normal console-output facilities may not participate in printer echo.

The actual destination of list output depends on the BIOS and the current logical-device assignment.

Ctrl-S and Ctrl-Q - Output Control

Ctrl-S (`13H`) and Ctrl-Q (`11H`) are commonly associated with pausing and resuming character output.

They are also known as XOFF and XON in serial communications.

In conventional CP/M console operation, Ctrl-S is commonly used to suspend scrolling output, with Ctrl-Q or another accepted key resuming it.

The precise behavior depends on the console driver and application.

BetterCP/M applications may also assign these keys to their own operations.

For example, transient TYPE.COM uses Ctrl-Q to quit its interactive paging mode.

Consequently, these characters should not be assumed to have one universal meaning throughout the system.

Ctrl-Z - End of Text

Ctrl-Z (`1AH`) is traditionally used as an end-of-text marker in CP/M text files.

CP/M stores files in 128-byte logical records, so the physical end of a text file does not always coincide with the end of its meaningful text.

A Ctrl-Z character can mark the point at which text ends.

For example, TYPE stops displaying a text file when it encounters the raw Ctrl-Z end-of-text marker.

This convention is particularly important when transferring text files between CP/M and other operating systems.

Not every CP/M file uses Ctrl-Z. Binary files may contain the value `1AH` as ordinary data.

Escape

Escape (`1BH`) is used by many terminal-control protocols.

For example, VT52 and ANSI/VT100 terminals use escape sequences to request operations such as cursor movement and screen clearing.

An escape sequence consists of the Escape character followed by additional characters identifying the requested operation.

BetterCP/M's terminal-monitor extensions interpret the supported sequences for their respective terminal personalities.

See Chapter 9 for more information.

C.3 BetterCP/M Command-Line Control Keys

The complete command-editor key table is in Section B.5, with original Model 4 aliases in Section B.2. These prompt-level assignments do not redefine ASCII or the controls used by applications. For example, Ctrl-J is newer history at the prompt and can still be line feed inside a program.

C.4 WordStar Keyboard Conventions

WordStar was one of the most influential applications in the CP/M environment, and its keyboard conventions were adopted by many other programs.

Because early computers often lacked dedicated cursor keys, WordStar used control-key combinations for movement and editing.

The familiar cursor movement keys are:

Key	WordStar function
Ctrl-E	Cursor up
Ctrl-X	Cursor down
Ctrl-S	Cursor left
Ctrl-D	Cursor right

These assignments form a convenient arrangement on a conventional keyboard.

Other WordStar commands use combinations of control keys for operations such as moving by words, deleting text, and navigating through documents.

WordStar and BetterCP/M

BetterCP/M uses some WordStar-style conventions in its command-line editor, but it is not a WordStar implementation.

In particular, BetterCP/M 1.0 uses Ctrl-K and Ctrl-J for command-history navigation rather than Ctrl-E and Ctrl-X.

Applications running under BetterCP/M retain their own keyboard conventions.

For example, a WordStar-compatible editor may continue to use Ctrl-E and Ctrl-X for vertical cursor movement.

The operating system does not require applications to adopt its command-history assignments.

Application-Specific Key Assignments

Control characters can have different meanings in different programs.

For example:

Key	Possible application-specific meaning
Ctrl-C	Interrupt, cancel, or another program-defined action
Ctrl-J	Line feed or BetterCP/M history navigation

Key	Possible application-specific meaning
Ctrl-K	WordStar command prefix or BetterCP/M history navigation
Ctrl-Q	WordStar command prefix, flow control, or TYPE.COM quit
Ctrl-S	Cursor left or output suspension
Ctrl-X	Cursor down or another program-defined action

These differences are normal in the CP/M environment.

When using an application, follow the keyboard conventions documented for that application.

C.5 Terminal Control Sequences

Many CP/M applications use control sequences to manage full-screen displays.

These sequences are distinct from ordinary command-line editing keys.

A terminal-control sequence is a series of characters sent to the console to request an operation such as moving the cursor, clearing the screen, or changing a display attribute.

ADM-3A

ADM-3A-compatible applications use the control sequences associated with the Lear Siegler ADM-3A terminal.

BetterCP/M provides an ADM-3A terminal personality through:

```
ADM3A.RSX
```

The supported implementation follows the Montezuma Micro-style ADM-3A conventions used by compatible TRS-80 Model 4 CP/M applications.

VT52

VT52-compatible applications use escape sequences associated with the DEC VT52 terminal.

BetterCP/M provides:

```
VT52.RSX
```

This interprets the supported VT52 control sequences and passes the corresponding display operations to the installed screen backend.

ANSI/VT100

ANSI and VT100-compatible applications commonly use escape sequences beginning with Escape followed by additional control characters.

BetterCP/M provides:

```
ANSI.RSX
```

This supports a useful subset of ANSI/VT100 terminal operations.

It does not provide complete VT100 emulation.

Terminal Personalities and Screen Backends

BetterCP/M separates terminal personalities from screen backends.

The terminal personality determines which control sequences are recognized.

The screen backend determines how the requested operations are performed on the computer's display.

For example:

```
A0>RSX LOAD M4SCRN
A0>RSX LOAD VT52
```

establishes the VT52 environment using the TRS-80 Model 4 screen backend.

For more information about supported terminal operations, see Chapter 9.

C.6 Keyboard Differences Between Platforms

BetterCP/M supports computers and emulated environments with different keyboard arrangements.

Some computers provide dedicated cursor keys, while others rely primarily on control-key combinations.

Modern terminal emulators may translate host keyboard input into codes appropriate for the emulated computer.

Consequently, the same physical key may produce different character codes on different systems.

TRS-80 Model 4

Section B.4 contains the original Model 4 punctuation chords. Host emulators may require different physical keys to produce those signals; consult the emulator's keyboard mapping.

Terminal Emulators

On systems using a terminal emulator, cursor keys may generate multi-character escape sequences rather than individual ASCII control characters.

For example, an Up arrow might transmit an escape sequence instead of Ctrl-E.

Whether BetterCP/M recognizes that sequence depends on the console environment.

A terminal personality that interprets display-control sequences does not necessarily provide corresponding keyboard translation.

The two facilities should not be confused.

When a Key Doesn't Work

If a keyboard command does not behave as expected, first determine whether the problem occurs at the BetterCP/M command prompt or within an application.

If it occurs at the command prompt, check the command editor's supported key assignments.

If it occurs within an application, check that application's keyboard configuration.

For full-screen applications, also verify that the appropriate terminal personality is loaded.

When using an emulator, examine its keyboard-mapping settings.

C.7 Control Characters in Text Files

Control characters can also appear as data within files.

This is particularly common in CP/M text files, which often use carriage return and line feed to separate lines and Ctrl-Z to mark the end of meaningful text.

Line Endings

CP/M text files conventionally use:

```
0DH 0AH
```

to mark the end of a line.

These are carriage return and line feed.

This convention is also commonly called CR/LF.

When transferring text files between CP/M and other operating systems, line-ending differences may need to be considered.

End-of-Text Marker

The Ctrl-Z character (`1AH`) traditionally marks the end of meaningful text.

For example, a file may contain:

```
FIRST LINE<CR><LF>
SECOND LINE<CR><LF>
<Ctrl-Z>
```

The Ctrl-Z marker indicates that the remaining bytes in the file's final record are not part of the text.

Displaying Control Characters

BetterCP/M's transient TYPE utility provides `/CTL` for displaying control characters in readable form.

For example:

```
A0>:TYPE README.TXT /CTL
```

This displays control characters using caret notation.

A tab appears as `^I`, while a form feed appears as `^L`.

The raw Ctrl-Z end-of-text marker still terminates the display rather than appearing as ordinary file content.

For additional information, see Section 14.4, *TYPE*.

Appendix D - File Attributes and File Specifications

BetterCP/M retains the familiar CP/M 2.2 file-naming conventions while extending the ways in which files can be selected and managed.

In conventional CP/M, a file is identified by its name, file type, drive, and user area. BetterCP/M preserves these conventions and adds facilities for selecting files across multiple user areas, combining drive/user specifications, and filtering files according to their attributes.

These extensions are particularly useful when working with large collections of files, performing backups, or managing files distributed across several user areas.

This appendix describes BetterCP/M's file specifications, wildcard conventions, drive/user selectors, and file attributes. It also explains how these facilities are used by commands such as DIR, ERA, REN, COPY, MOVE, TYPE, and STAT.

For an introduction to working with files, see Chapter 4, *Working with Drives and Files*. For the exact syntax accepted by individual utilities, consult Part II, *Command and Utility Reference*.

D.1 File Names and File Types

BetterCP/M uses the standard CP/M 8.3 filename convention.

A filename consists of a name containing up to eight characters and an optional file type containing up to three characters.

The name and type are separated by a period.

For example:

```
REPORT.TXT
PROGRAM.COM
SOURCE.ASM
BACKUP.BAK
```

The portion before the period is the filename. The portion after the period is the file type, sometimes called the extension.

A file type is optional.

For example:

```
README
```

is a valid filename without a file type.

Letter Case

BetterCP/M treats filenames without regard to letter case.

For example:

```
REPORT.TXT
report.txt
Report.Txt
```

all identify the same CP/M filename.

For consistency, this manual generally displays filenames in uppercase.

File Types

CP/M does not require every file type to have a particular meaning, but several conventions are widely used.

File type	Common purpose
`.COM`	Executable transient program
`.ASM`	Assembly-language source
`.HEX`	Intel hexadecimal object file
`.SUB`	SUBMIT command file
`.TXT`	Text file
`.DOC`	Documentation
`.BAK`	Backup copy
`.SYS`	System-related file
`.RSX`	BetterCP/M Resident System Extension
`.CPX`	BetterCP/M Command Processor Extension

These conventions help identify a file's intended purpose, but the file type alone does not guarantee that its contents have the expected format.

For example, renaming a text file to `.COM` does not make it an executable program.

Filename Restrictions

File specifications must conform to CP/M's filename rules.

Names longer than eight characters or file types longer than three characters are invalid.

BetterCP/M's file-selection facilities also distinguish between literal filenames and wildcard patterns.

Characters used as wildcard operators cannot be treated as ordinary filename characters in a file-selection expression.

When a command rejects a filename, check its length, punctuation, and any wildcard characters.

D.2 Drive and User-Area Specifications

CP/M organizes files by drive and user area.

A drive is identified by a letter, such as `A:` or `B:`.

A user area is identified by a number.

BetterCP/M supports user areas 0 through 31.

Drive Specifications

To identify a file on drive B:

```
B:REPORT.TXT
```

If the user area is omitted, the current user area is used.

For example, if the current user area is 3, the specification:

```
B:REPORT.TXT
```

refers to `REPORT.TXT` on drive B, user area 3.

User-Area Specifications

BetterCP/M allows a user area to be specified directly with the drive.

For example:

```
B3:REPORT.TXT
```

identifies `REPORT.TXT` on drive B, user area 3.

A user-area specification can also be used without a drive letter:

```
3:REPORT.TXT
```

This refers to user area 3 on the current drive.

Drive/User Abbreviations

BetterCP/M uses the term **drive/user**, abbreviated **DU**, for a particular drive and user-area combination.

For example:

Specification	Meaning
`A0:`	Drive A, user area 0
`A3:`	Drive A, user area 3
`B0:`	Drive B, user area 0
`B17:`	Drive B, user area 17
`3:`	User area 3 on the current drive
`B:`	Drive B, current user area

When no drive or user area is specified, the current drive/user is used.

Changing the Current Drive/User

You can change the current drive and user area directly at the command prompt.

For example:

```
A0>B:
B0>
```

selects drive B.

To select user area 3:

```
B0>3:
B3>
```

To select both at once:

```
B3>C5:
C5>
```

These commands change the current working location.

By contrast, supplying a drive/user qualification to a file-management command normally selects files in that location without changing the current drive/user.

For example:

```
A0>DIR B3:*.COM
```

lists matching files in B3 and then returns to the original prompt.

D.3 Wildcard File Specifications

Wildcards allow a command to select several files using a single specification.

BetterCP/M supports the familiar CP/M wildcard characters:

Character	Meaning
?	Match one filename position
*	Match the remainder of a filename or file-type field

The name and file-type fields are matched separately.

Question-Mark Wildcards

A question mark matches one position in the filename or file type.

For example:

```
TEST?.COM
```

can match names such as:

```
TEST1.COM
TEST2.COM
TESTA.COM
```

A pattern containing several question marks can match several corresponding positions.

For example:

```
FILE??.TXT
```

selects filenames matching the indicated positions.

Asterisk Wildcards

An asterisk matches the remaining positions in its field.

For example:

```
*.COM
```

selects files with the `.COM` type.

Similarly:

```
REPORT.*
```

selects files named `REPORT` with different file types.

The familiar specification:

```
*.*
```

selects all filenames and file types, subject to any attribute restrictions imposed by the command.

BetterCP/M's Bounded Wildcard Rules

BetterCP/M uses a consistent set of bounded 8.3 wildcard rules in its extended file-selection facilities.

An asterisk must occupy the remainder of the filename or file-type field.

For example, these patterns are valid:

```
F?*.DAT
F***.DAT
```

Repeated terminal asterisks are treated as equivalent to one asterisk.

However, characters cannot follow an asterisk within the same field.

Consequently, these patterns are invalid:

```
F*A.DAT
F**?.DAT
```

BetterCP/M reports:

```
Invalid filespec.
```

This restriction avoids ambiguous wildcard interpretation and provides consistent behavior among the utilities using the shared file-selection facilities.

Wildcards and Protected Files

Wildcards select filenames; they do not automatically override file protection.

For example:

```
ERA *.BAK
```

does not imply permission to erase read-only or system files.

The treatment of protected files depends on the command and any explicit attribute-selection or protection options.

For details, consult the ERA, REN, COPY, and MOVE references in Chapter 14.

D.4 Extended Drive/User Selectors

BetterCP/M extends conventional CP/M file specifications by allowing commands to select files from several drive/user areas in a single operation.

These extended specifications are called **drive/user selectors**, or **DU selectors**.

They are supported by the enhanced file-management utilities, although not every command accepts every form.

Selecting One User Area

A conventional BetterCP/M file specification identifies one drive/user area:

```
B3:*.COM
```

This selects matching `.COM` files in drive B, user area 3.

Selecting Several User Areas

A bracketed list selects several user areas on the same drive.

For example:

```
B[1,3,5]:*.COM
```

selects `.COM` files in user areas 1, 3, and 5 on drive B.

A range can be specified with a hyphen:

```
B[3-7]:*.ASM
```

This selects user areas 3 through 7.

Lists and ranges can be combined:

```
B[1,3,8-10]:*.DOC
```

This selects user areas 1, 3, 8, 9, and 10.

Selecting All User Areas

The special selector `[-]` selects all supported user areas on a drive.

For example:

```
B[-]:*.COM
```

selects `.COM` files from user areas 0 through 31 on drive B.

The notation:

```
B[*]:*.COM
```

is not an alternative spelling and is invalid.

Use `[-]` when you want all user areas.

Descending and Open-Ended Ranges

BetterCP/M's extended selector language also supports descending and open-ended ranges.

For example:

```
B[5-3]:*.DAT
```

selects user areas 3, 4, and 5.

The direction of the range does not change the order in which files are processed.

Open-ended ranges allow one endpoint to be omitted, using the beginning or end of the supported user-area range.

These facilities are useful when selecting groups of user areas without listing every number individually.

Selecting Several Drives

Compound selectors allow locations on different drives to be combined.

For example:

```
[A0,B3]:*.DOC
```

selects `.DOC` files from A0 and B3.

More elaborate selections are also possible:

```
[A0,B[3-5],C[4,5,9]]:*.COM
```

This selects matching files from A0, B3 through B5, and C4, C5, and C9.

Inherited Drive and User Areas

A compound selector can contain terms that omit either the drive or the user area.

An omitted drive inherits the drive that was current when the command began.

An omitted user area inherits the original current user area.

For example:

```
[A0,B[3-5],5]:*.COM
```

The final `5` refers to user area 5 on the original current drive.

This behavior allows convenient selections without repeatedly specifying the same drive.

Duplicate Locations

If a compound selector names the same drive/user area more than once, BetterCP/M processes that location only once.

For example:

```
[B3,B[3-5]]:*.COM
```

does not cause B3 to be processed twice.

Selected locations are normally processed in drive-major, user-minor order.

Unsupported Drives

The available logical drives depend on the computer and its configuration.

BetterCP/M's current supported drive bindings ordinarily use drives A through D.

An extended selector does not make an unavailable drive accessible.

Different utilities may handle an unavailable selected drive differently. For example, one may reject the entire operation, while another may report the unavailable location and continue examining the remaining locations.

Consult the command reference for the applicable behavior.

D.5 File Attributes

CP/M stores several file attributes in directory entries.

These attributes affect how files are displayed, protected, or selected by utilities.

BetterCP/M retains the traditional attributes and provides extended facilities for selecting files according to their attribute settings.

The principal attributes are:

Attribute	Meaning
RO	Read-only
RW	Writable
SYS	System
DIR	Directory-visible, meaning SYS is clear
ARC	Archive

RO and RW describe opposite states of the read-only attribute.

Similarly, SYS and DIR describe opposite states of the system attribute.

ARC is an independent attribute.

Read-Only Files

A file marked read-only is protected against ordinary modification or deletion.

This can help prevent accidental changes to important programs or data.

For example, a utility may refuse to erase a read-only file unless the user explicitly authorizes the operation.

BetterCP/M's enhanced ERA utility provides additional safeguards for read-only files.

System Files

The SYS attribute identifies a file as a system file.

In conventional CP/M, system files are normally omitted from ordinary directory listings.

BetterCP/M retains this convention.

For example:

```
A0>DIR
```

normally omits SYS files.

An explicit attribute expression can select them:

```
A0>DIR *.*[$SYS]
```

This displays matching system files.

The SYS attribute does not necessarily mean that a file is part of the operating system itself. It is a directory attribute that may also be used by applications or utilities.

Archive Attribute

The ARC attribute is commonly associated with backup operations.

BetterCP/M's COPY utility can use ARC to select files and manage their backup status.

For example:

```
A0>:COPY /V /BACKUP B[-]:*.DOC[$ARC] D0:
```

This selects archive-marked `.DOC` files, copies and verifies them, and clears ARC on the source and destination after successful completion.

However, ARC should not automatically be interpreted as a reliable record of every file changed since the previous backup.

That depends on whether the software modifying files maintains the attribute appropriately.

Displaying Attributes

The transient DIR utility provides `/A` to display file attributes.

For example:

```
A0>:DIR /A
```

The attributes appear as a three-character field in SYS, RO, ARC order.

Display	Meaning
`---`	No attributes set
`S--`	System
`-R-`	Read-only
`--A`	Archive
`SR-`	System and read-only
`S-A`	System and archive
`-RA`	Read-only and archive
`SRA`	All three attributes set

A hyphen indicates that the corresponding attribute is clear.

The `/A` option displays attributes; it does not change them or select files according to their values.

For attribute-based selection, use an attribute expression.

D.6 Attribute Expressions

BetterCP/M allows several file-management commands to select files according to their attributes.

An attribute expression is enclosed in square brackets immediately following the file specification.

For example:

```
*.COM[$ARC]
```

selects `.COM` files whose ARC attribute is set.

Attribute Names

The supported attribute predicates are:

Predicate	Selects
`$RO`	Read-only files
`$RW`	Writable files
`$SYS`	System files
`$DIR`	Files without SYS set
`$ARC`	Files with ARC set

The aliases `$R/O` and `$R/W` are also accepted for `$RO` and `$RW`.

Logical Operators

Attribute expressions support three logical operators:

Operator	Meaning
`!`	NOT
`+`	AND
`,`	OR

For example:

```
*.COM[$ARC+!$SYS]
```

selects `.COM` files that have ARC set and SYS clear.

Another example:

```
*.DOC[$RO,$SYS]
```

selects `.DOC` files that are read-only or marked as system files.

To select files without ARC:

```
*.*[!$ARC]
```

Operator Precedence

When several operators appear in an expression, BetterCP/M evaluates them in the following order:

1. NOT (`!`)
2. AND (`+`)
3. OR (`,`)

For example:

```
[$RO,$SYS+$ARC]
```

selects files that are read-only, or files that have both SYS and ARC set.

It does not require read-only files to have ARC set.

Parentheses are not supported in attribute expressions.

Attribute Selection Does Not Change Attributes

An attribute expression normally selects files; it does not modify their attributes.

For example:

```
A0>DIR *.COM[$ARC]
```

displays matching archive-marked files without changing them.

Similarly:

```
A0>COPY *.COM[$ARC] B:
```

selects archive-marked files for copying but does not clear ARC on the source merely because the files were selected.

Attribute modification requires a command or option that explicitly performs it.

Attribute Selection and File Protection

Selecting a protected file does not necessarily authorize modifying or deleting it.

For example:

```
A0>ERA *.DOC[$RO]
```

explicitly selects read-only files, but transient ERA may still require additional authorization before erasing them.

The `/R` option permits erasure of selected read-only files:

```
A0>:ERA /R *.DOC[$RO]
```

Likewise, transient REN requires `/P` when renaming protected files.

The exact protection rules depend on the command.

For ERA, selection, protection admission and read-only authorization are separate. All protected attributes present must be admitted by the same matching OR branch: `[$SYS+$RO]` admits a SYS+RO file, while `[$SYS,$RO]` does not. `/R` only authorizes admitted RO files; it neither broadens the selection nor admits SYS. `/Y` without `/R` skips RO files unattended.

Consult Section 14.2 for the complete ERA rules.

D.7 Destination Attribute Specifications

Some BetterCP/M utilities allow attributes to be assigned to destination files.

This is different from selecting source files by attribute.

Transient COPY.COM provides destination attribute specifications.

For example:

```
A0>:COPY A0:*.COM D0:[$RW,!$ARC]
```

This requests destination copies that are writable and have ARC clear.

The source files are selected normally, while the destination expression specifies the attributes to apply after copying.

Source and Destination Expressions

Consider:

```
A0>:COPY B2:*.DOC[$ARC] C0:[!$ARC] /V
```

The source expression:

```
[$ARC]
```

selects files whose archive attribute is set.

The destination expression:

```
[!$ARC]
```

requests that ARC be cleared on the completed copies.

The source files retain their original attributes unless another option explicitly changes them.

Destination Attribute Rules

Destination attribute specifications use comma-separated assignments.

They are not general Boolean selection expressions.

For example:

```
[$RW,!$ARC]
```

is valid as a destination attribute specification.

Conflicting assignments, such as requesting both RO and RW, are invalid.

Attributes not explicitly changed are preserved.

Destination attribute settings do not override protection on an existing read-only destination file.

The /BACKUP Option

COPY's `/BACKUP` option provides additional handling of the archive attribute.

For example:

```
A0>:COPY /V /BACKUP B[-]:*.DOC[$ARC] D0:
```

After a successful copy and any requested verification, COPY clears ARC on both source and destination.

Skipped, failed, or interrupted copies retain their source ARC status.

The `/BACKUP` option takes precedence over a destination request to set ARC.

For more information, see Section 14.5, *COPY*.

D.8 Compatibility and Command-Specific Restrictions

BetterCP/M's extended file-selection facilities are intended to provide consistent behavior across commands, but not every command supports every feature.

For example, DIR, ERA, REN, COPY, MOVE, and TYPE support extended source selection.

STAT supports compound drive/user selectors for file inspection, but not for every attribute-modification or drive-wide operation.

Some utilities also impose restrictions on the number of selected files or the destinations they accept.

Source and Destination Restrictions

Commands that operate on several source files may require a single destination drive/user area.

For example:

```
A0>COPY B[-]:*.DOC C3:
```

is valid because the destination identifies one location.

A destination selector resolving to several user areas is not accepted by COPY.

Similarly, REN changes filenames within their existing drive/user areas. It does not use destination drive/user specifications to relocate files.

Resident and Transient Differences

Some commands provide additional capabilities in their transient versions.

For example, transient COPY supports wildcard destination templates and destination attribute assignments.

Resident COPY provides a smaller set of operations.

When a command has both resident and transient forms, consult its reference entry to determine which features are available in each.

Use the colon or period prefix to request the transient version explicitly.

File-Selection Errors

BetterCP/M checks file specifications before carrying out operations.

Invalid filenames, malformed wildcard patterns, and unsupported selectors produce errors.

Commands that modify files generally validate the requested selection before beginning, helping prevent partial operations caused by malformed input.

However, validation does not guarantee that every subsequent file operation will succeed.

Disk errors, insufficient space, file protection, and destination conflicts may still interrupt an operation after some files have been processed.

For this reason, use particular care with wildcard operations that modify or delete many files.

Further Reference

For detailed command behavior, consult:

- Chapter 14 - File and Directory Commands
- Chapter 16 - System Utilities
- Appendix E - BetterCP/M and CP/M 2.2 Compatibility Notes

Appendix E - BetterCP/M and CP/M 2.2 Compatibility Notes

BetterCP/M is intended to provide a compatible implementation of Digital Research CP/M 2.2 while extending its capabilities in ways useful to contemporary CP/M users.

Compatibility is an important part of BetterCP/M's design. Existing CP/M programs should ordinarily be able to use the familiar operating-system services without modification, while users can take advantage of BetterCP/M's additional commands, utilities, and resident extensions.

Nevertheless, compatibility does not mean that BetterCP/M behaves identically to Digital Research CP/M 2.2 in every respect.

Some differences are deliberate extensions. Others reflect variations among historical CP/M implementations, differences in hardware, or the behavior of particular utilities.

This appendix summarizes the principal compatibility considerations that may affect users running conventional CP/M software under BetterCP/M 1.0.

It is not intended to reproduce the complete technical compatibility specification. Programmers requiring exact BDOS behavior, calling conventions, or implementation details should consult the *BetterCP/M Compatibility Documentation* and the appropriate programming references.

E.1 Compatibility Objectives

BetterCP/M preserves the principal conventions of Digital Research CP/M 2.2.

These include the familiar command environment, `.COM` executable format, drive and user-area organization, file-naming conventions, and standard operating-system interfaces.

Programs that use conventional CP/M services should ordinarily require no changes.

For example, a program that uses BDOS to open, read, write, and close files should continue to use those services in the expected manner.

Likewise, programs that use the conventional console interface can continue to communicate through the BIOS and BDOS facilities provided by BetterCP/M.

Compatibility and Extensions

BetterCP/M adds facilities beyond those provided by Digital Research CP/M 2.2.

These include extended drive/user specifications, enhanced file-management utilities, resident extensions, terminal monitors, and clock services.

These additions are intended to supplement conventional CP/M operation rather than require applications to adopt a new programming model.

An ordinary CP/M program does not need to know whether BetterCP/M has an extended DIR utility or a resident terminal monitor installed.

It can continue using the conventional interfaces.

Compatibility Is Not Hardware Emulation

BetterCP/M provides an operating-system environment, not a universal emulator for every computer that historically ran CP/M.

A program that communicates directly with a particular computer's hardware may not run correctly on another computer, even when both use compatible CP/M operating systems.

For example, a program written specifically for the TRS-80 Model 4's video hardware may depend on memory addresses or hardware behavior not present on another system.

BetterCP/M's terminal monitors can provide compatible terminal-control behavior, but they cannot make arbitrary hardware-specific programs portable.

E.2 Program Compatibility

Transient Programs

BetterCP/M retains the conventional CP/M transient-program model.

Executable programs normally use the `.COM` file type and begin execution at address `0100H`.

BetterCP/M establishes the expected program environment, including the command tail, default File Control Blocks, and conventional page-zero facilities.

Programs using these conventions should ordinarily run without modification.

Program Termination

Conventional CP/M programs terminate by returning control to the operating system through the supported termination mechanisms.

BetterCP/M preserves this behavior.

When a transient program terminates, BetterCP/M restores the command environment and displays the command prompt.

Resident extensions intended to survive ordinary warm boots remain available.

Available Memory

The amount of memory available to a transient program depends on the system configuration.

Loaded RSXs occupy protected memory that is unavailable to transient programs. The CCP and CPX command code are reclaimable and can be overwritten by a transient; BetterCP/M reconstructs the command environment on warm boot.

A program may therefore encounter insufficient memory when RSXs are loaded, even though it fits without those services.

Use `STAT MEM` to check the maximum loadable COM size and `RSX LIST` to identify services you can unload. Removing an unnecessary RSX can recover program memory; unloading a CPX does not increase the maximum COM size by its reclaimable command-code size.

This is a resource limitation rather than necessarily an incompatibility in the program's operating-system calls.

Programs That Modify System Memory

Some historical CP/M programs make assumptions about the location or contents of the command processor, BDOS, BIOS, or other system structures.

Such programs may rely on details of a particular CP/M implementation rather than on its documented programming interface.

BetterCP/M does not guarantee compatibility with every such assumption.

Programs that use documented operating-system entry points are generally more portable than programs that inspect or modify private system structures.

E.3 File and Disk Compatibility

File Naming

BetterCP/M retains CP/M's 8.3 filename convention.

Ordinary CP/M filenames and file types remain valid.

BetterCP/M also retains conventional wildcard characters and file attributes.

Its enhanced utilities provide additional file-selection capabilities, but ordinary programs are not required to use them.

User Areas

Digital Research CP/M 2.2 conventionally provides user areas 0 through 15.

BetterCP/M extends this range to 0 through 31.

The additional user areas are a BetterCP/M facility.

Programs that assume the conventional range may not recognize or manage files in the additional areas correctly.

For maximum compatibility with conventional utilities, use user areas 0 through 15 when working with software that makes assumptions about the available range.

Drive/User Specifications

BetterCP/M commands can accept explicit drive/user specifications such as:

```
B3:REPORT.TXT
```

Some enhanced utilities also support compound selectors:

```
B[-]:*.COM
[A0,B3]:*.DOC
```

These forms are BetterCP/M command extensions.

They are not part of the conventional CP/M 2.2 filename syntax understood by every application.

A third-party CP/M program may interpret such arguments differently or reject them.

Use conventional file specifications when invoking software that does not explicitly support BetterCP/M's extensions.

Disk Formats

CP/M was used on many computers with different disk formats.

BetterCP/M can support multiple formats through its drive-configuration facilities, subject to the capabilities of the computer and its disk controller.

However, a disk that uses a valid CP/M filesystem is not necessarily physically readable by every CP/M computer.

Differences in recording density, sector size, track organization, and controller capabilities can prevent direct interchange.

BetterCP/M's CONFIG and DUP utilities help manage supported formats, but they do not eliminate hardware limitations.

File Attributes

BetterCP/M retains conventional CP/M file attributes, including read-only and system attributes.

Its enhanced utilities also support archive-attribute selection and management.

Programs that use ordinary CP/M file operations can continue to use the conventional attributes.

However, software that maintains file attributes independently may not follow BetterCP/M's extended backup conventions.

In particular, the ARC attribute should not be assumed to identify every modified file unless the software involved maintains it appropriately.

E.4 Command Compatibility

BetterCP/M retains the familiar CP/M command environment while extending several commands.

Some extensions are available through resident commands, while others require transient utilities.

Resident and Transient Commands

Digital Research CP/M 2.2 includes a small set of resident commands and uses transient programs for other operations.

BetterCP/M follows the same general arrangement but supports additional resident commands through Command Processor Extensions.

The standard resident command package, `RCP.CPX`, provides commands such as:

```
DIR   ERA   TYPE   REN   USER
CLS   VER   COPY   MOVE
```

Some commands are also available as transient programs.

For example, DIR and `DIR.COM` provide related but different capabilities.

The transient utility supports additional sorting, display, and file-search options.

Explicit Transient Execution

BetterCP/M provides a way to execute a transient utility even when a resident command with the same name is available.

Prefix the command with a colon or period:

```
A0>:DIR
A0>.DIR
```

Both forms explicitly request `DIR.COM`.

This is a BetterCP/M extension to the conventional command environment.

The prefixes are interpreted by BetterCP/M's command processor and do not change the executable filename.

Differences in Command Syntax

Some BetterCP/M commands accept syntax that conventional CP/M utilities do not recognize.

For example:

```
A0>COPY B[-]:*.DOC C3:
```

uses BetterCP/M's extended drive/user selection.

Similarly:

```
A0>:DIR /S=Z-
```

uses a transient DIR option for sorting by allocated size in descending order.

These facilities are useful when working within BetterCP/M, but they should not be assumed to exist on other CP/M implementations.

Compatibility of Command Files

SUBMIT retains the familiar `.SUB` command-file convention and positional parameter substitution.

However, a SUBMIT file containing BetterCP/M-specific commands or options may not run unchanged under Digital Research CP/M 2.2.

For example, a batch file that invokes CONFIG or uses extended COPY options depends on BetterCP/M facilities.

When preparing command files intended for several CP/M systems, restrict them to commands and syntax supported by all the target environments.

Conventional Utility Behavior

BetterCP/M aims to preserve the familiar behavior of conventional CP/M utilities where compatibility matters.

However, some enhanced utilities deliberately provide additional safeguards, confirmation prompts, or error-handling facilities.

For example, transient ERA provides more extensive confirmation and file-protection controls than the traditional resident ERA command.

Likewise, transient COPY offers verification, destination-collision handling, and backup operations.

These differences are intentional and are documented in the corresponding command references.

E.5 Console and Terminal Compatibility

Conventional Console Services

BetterCP/M retains the conventional CP/M console services used by ordinary applications.

Programs that read and write characters through the supported BIOS and BDOS interfaces can continue to use those facilities.

The actual keyboard and display hardware depend on the computer.

Terminal-Specific Applications

Many historical CP/M applications were configured for a particular terminal.

For example, a full-screen editor might expect ADM-3A or VT52 control sequences.

BetterCP/M provides resident terminal personalities for:

Terminal	Extension
Montezuma Micro-style ADM-3A	`ADM3A.RSX`
VT52	`VT52.RSX`
ANSI/VT100 subset	`ANSI.RSX`

These extensions interpret supported terminal-control sequences and pass the corresponding display operations to a screen backend.

The application must still be configured for the appropriate terminal type.

Terminal Limitations

BetterCP/M's terminal personalities provide supported subsets of the corresponding terminal behavior.

They do not necessarily reproduce every feature of the original physical terminals.

For example, ANSI support does not constitute complete VT100 emulation.

Similarly, keyboard translation depends on the computer, BIOS, terminal emulator, and application.

A terminal personality that interprets display sequences does not automatically guarantee that every physical key generates the codes expected by the application.

Direct Hardware Access

Some applications bypass CP/M's console services and manipulate display hardware directly.

These programs may be tied to a particular computer.

BetterCP/M's terminal monitors do not automatically translate such hardware access.

For best portability, use applications that communicate through conventional CP/M console services or supported terminal-control sequences.

E.6 Date and Time Compatibility

Digital Research CP/M 2.2 did not define a general-purpose date-and-time service in its standard programming interface.

Consequently, historical CP/M systems adopted several different clock interfaces.

BetterCP/M provides a native clock service and an optional compatibility interface for programs using P2DOS-style clock functions.

Native Clock Support

BetterCP/M's TIME utility obtains the date and time through a loadable clock provider.

BetterCP/M 1.0 includes:

Provider	Environment
`FREHDCLK.RSX`	FreHD clock interface
`ZPRTC.RSX`	z80pack/cpmsim clock interface

Both providers are read-only.

They support obtaining the date and time but do not permit setting the underlying clock.

The TIME utility implements a SET request, but the supplied providers reject it as unsupported.

P2DOS Compatibility

The optional, separately buildable `P2DOS.RSX` supports applications using the P2DOS-compatible date-and-time interface. It is not selected System Disk content; install a compatible carrier in A0 before using the example below.

The extension supplies the corresponding BDOS functions through BetterCP/M's native clock service.

For example, under z80pack:

```
A0>RSX LOAD ZPRTC
A0>RSX LOAD P2DOS
```

This makes the P2DOS-compatible interface available to programs that use it.

The P2DOS extension does not provide an independent hardware clock.

Its capabilities depend on the installed native provider.

Consequently, setting the clock remains unsupported with the standard read-only providers.

Compatibility Limits

Programs using the conventional CP/M 2.2 programming interface do not require clock support.

Programs depending on a particular manufacturer's proprietary clock interface may require additional compatibility facilities.

BetterCP/M's native and P2DOS-compatible clock services should not be assumed to reproduce every historical CP/M clock interface.

For more information, see Chapter 11, *Date and Time*.

E.7 Resident Extensions and Compatibility

BetterCP/M provides two types of resident extensions:

- **CPXs**, which add commands to the command processor.
- **RSXs**, which provide services that remain available while programs execute.

These are BetterCP/M facilities and are not part of the standard Digital Research CP/M 2.2 operating environment.

Command Processor Extensions

CPXs allow additional commands to remain resident.

Ordinary CP/M programs do not need to know whether a command is provided by a CPX or loaded from disk.

However, programs that inspect the command processor's private memory structures may encounter differences from Digital Research CP/M.

Resident System Extensions

RSXs provide optional services such as terminal support, clock providers, and batch input.

They can remain installed across ordinary warm boots.

Because RSXs occupy memory, they can reduce the available TPA.

This may affect programs that require nearly all the memory available on a conventional CP/M system.

Unloading unnecessary extensions can restore memory for such programs.

Extension Dependencies

Some RSXs require supporting extensions.

For example, a terminal personality depends on a suitable screen backend.

Load the backend before the personality, and unload them in the reverse order.

For additional information, see Chapter 8, *Resident Extensions*.

E.8 Compatibility Considerations for Existing Software

Most conventional CP/M applications should require little or no special preparation beyond having the appropriate executable files and terminal configuration.

However, several factors are worth checking when a program does not behave as expected.

Operating-System Interface

Programs using documented CP/M 2.2 BDOS and BIOS services are generally more portable than programs relying on undocumented internal structures.

If an application behaves differently under BetterCP/M, determine whether it uses conventional interfaces or makes assumptions about a particular implementation.

Terminal Configuration

Full-screen applications must be configured for a supported terminal environment.

If the display is incorrect, check the installed terminal personality and the application's terminal settings.

Use the supplied terminal test utilities to verify the monitor independently of the application.

Memory Requirements

Resident extensions reduce available transient memory.

If a program reports insufficient memory, unload optional extensions that are not required for its operation.

File and Disk Assumptions

Some programs assume particular disk capacities, drive arrangements, or user-area ranges.

BetterCP/M's extended facilities do not automatically remove those assumptions.

Use compatible disk formats and conventional file locations when working with software that has limited configuration options.

Program-Specific Installation

Some historical CP/M applications require installation or configuration for a particular terminal, printer, or computer.

These requirements remain relevant under BetterCP/M.

The operating system cannot automatically determine every application-specific hardware assumption.

Consult the application's original documentation when necessary.

E.9 Reporting Compatibility Problems

If a conventional CP/M program behaves unexpectedly under BetterCP/M, the most useful first step is to determine whether the problem is reproducible.

Record the program's name and version, the BetterCP/M version, the computer or emulator, and the circumstances under which the problem occurs.

Where possible, determine whether the same program behaves differently under Digital Research CP/M 2.2 on an equivalent platform.

This can help distinguish an operating-system compatibility issue from an application configuration problem or a hardware limitation.

Information to Record

A useful compatibility report should identify:

Information	Example
Application	Program name and version
Operating system	BetterCP/M release and build

Information	Example
Platform	Computer or emulator
Terminal configuration	Installed screen backend and personality
Resident extensions	Output of `RSX LIST` and `CPX LIST`
Disk configuration	Relevant drive and format assignments
Problem	Expected behavior and observed behavior

Include the commands needed to reproduce the problem when possible.

If the application displays an error message, record its exact wording.

Comparing with Digital Research CP/M

When practical, test the same application under a known working Digital Research CP/M 2.2 installation.

Use equivalent hardware or emulator configurations where possible.

A difference in behavior may identify a compatibility issue, but it may also reflect different BIOS implementations, terminal settings, or disk formats.

A controlled comparison is more useful than simply observing that a program works on one computer and fails on another.

Further Technical Documentation

This appendix summarizes compatibility issues from the user's perspective.

More detailed compatibility information is maintained separately, including documentation of the supported CP/M interfaces and the results of compatibility testing.

Those documents are intended primarily for developers, system maintainers, and users investigating specific compatibility questions.

For the programming interfaces themselves, consult the BetterCP/M RSX and CPX Programmer's Guide and the original Digital Research CP/M 2.2 documentation.

Appendix F - System Messages and Error Reference

BetterCP/M reports errors and other operating conditions through messages displayed at the console.

Some messages originate in the command processor, while others are produced by individual utilities or resident extensions. A message may indicate a simple mistake, such as an incorrectly entered command, or a more serious problem involving disk access, system configuration, or program execution.

This appendix provides a reference to the principal messages encountered when using BetterCP/M 1.0. It explains their meanings, identifies likely causes, and suggests appropriate corrective actions.

The messages are grouped by subject rather than alphabetically. Related messages often have similar causes, and grouping them makes it easier to identify the underlying problem.

Messages shown in quotation marks or code blocks are intended to reproduce documented BetterCP/M wording. Where a utility may produce several related messages, the description explains the condition without implying that every variation has identical wording.

For general troubleshooting procedures, see Chapter 12, *Troubleshooting*. For command-specific restrictions and recovery options, consult Part II, *Command and Utility Reference*.

F.1 Command Processor Messages

`?`

Meaning: BetterCP/M could not locate or recognize the requested command.

For example:

```
A0>MYPROG
?
```

The command processor first checks its resident commands and installed CPXs. If no command handles the name, it searches for a corresponding transient `.COM` program.

For an unqualified command, BetterCP/M searches the current drive/user area and then `A0:`.

If no executable program is found, the command fails.

What to do: Check the spelling of the command and verify that the corresponding `.COM` file exists.

For example:

```
A0>DIR *.COM
A0>DIR A0:*.COM
```

If the program is in another location, specify its drive and user area explicitly.

A qualified command such as `B3:MYPROG` searches only the specified location; it does not fall back to `A0:`.

`CPX?`

Meaning: An optional Command Processor Extension could not be loaded or failed validation.

BetterCP/M continues to the command prompt with the remaining usable extensions.

Commands supplied by the rejected CPX are unavailable.

The extension may remain in the configured CPX list even though it did not load successfully.

What to do: Examine the configured extensions:

```
A0>CPX LIST
```

If the affected extension is optional, remove its configured entry:

```
A0>CPX UNLOAD BAD
```

Replace `BAD` with the affected extension's name.

Alternatively, replace the damaged extension file with a valid copy.

The warning may recur when the command environment is restored until the problem is corrected.

`DIR: Use :DIR.`

Meaning: The requested DIR operation requires the transient `DIR.COM` utility.

For example:

```
A0>DIR /WHEREIS ED.COM
DIR: Use :DIR.
```

The resident DIR command recognizes that the requested option belongs to the transient version.

It does not automatically execute that version.

What to do: Repeat the command with a colon or period prefix:

```
A0>:DIR /WHEREIS ED.COM
```

The same convention applies to other resident commands with transient-only options.

`VER: Use :VER.`

Meaning: The requested VER operation is available through the transient utility rather than the resident command.

For example:

```
A0>VER /BANNER
VER: Use :VER.
```

What to do: Invoke the transient version:

```
A0>:VER /BANNER
```

Invalid Command Syntax

A command may report an error when its arguments are missing, malformed, or supplied in an unsupported order.

The exact message depends on the command.

What to do: Check the syntax in Part II.

Pay particular attention to required operands, option placement, filename qualifications, and whether the command has resident and transient versions.

F.2 File and Directory Messages

`Invalid filespec.`

Meaning: A file specification contains invalid syntax.

Possible causes include an invalid filename, a malformed wildcard pattern, or an unsupported drive/user selector.

For example:

```
F*A.DAT
```

is invalid under BetterCP/M's bounded wildcard rules because characters follow the asterisk within the filename field.

What to do: Correct the specification.

A terminal asterisk must occupy the remainder of its filename or file-type field.

For example:

```
F*.DAT
```

is valid.

`NO FILE`

Meaning: No files matched the requested selection.

This may occur when the named file does not exist or when a wildcard or attribute expression matches no files.

What to do: Check the current drive and user area, the filename, and any attribute restrictions.

For example:

```
A0>DIR B3:*.COM
```

can help determine whether matching files exist in the expected location.

`File Not Found`

Meaning: A requested file or matching selection could not be found.

STAT may report this condition when examining an empty selected location.

What to do: Verify the drive/user specification and filename.

Remember that files in different user areas are distinct, even when they have identical names.

`FILE EXISTS`

Meaning: A destination file already exists and the requested operation does not authorize replacing it.

For example, COPY with `/B` but without `/O` or `/S` reports this condition for an existing writable destination.

What to do: Determine whether the destination should be preserved, skipped, renamed, or replaced.

For COPY, the appropriate options include:

```
/B /O
/B /S
```

The first permits replacement of existing writable files. The second skips them.

Read-only destination files remain protected.

`COPY MADE; SOURCE NOT ERASED`

Meaning: MOVE successfully created the destination file but could not remove the original.

Both files may now exist.

What to do: Examine the source and destination before taking further action.

If the destination is complete and correct, you may delete the source manually.

Do not assume that the original was removed.

`Invalid /VER usage.`

Meaning: The `/VER` option was combined with arguments or options that the utility does not permit.

Most BetterCP/M utilities require `/VER` to be used alone.

What to do: Enter the utility name followed only by `/VER`.

For example:

```
A0>COPY /VER
```

If the utility has a resident and transient version, use the appropriate invocation form.

F.3 Disk and Filesystem Messages

Disk-related errors may originate in BDOS, the BIOS, or a utility performing disk operations.

The exact wording depends on the operation and the platform.

Disk Read Error

Meaning: The system could not reliably read the requested disk data.

Possible causes include damaged media, an incorrect disk-format assignment, a drive problem, or an unavailable disk.

What to do: Check that the correct disk is inserted and that its assigned format matches its actual organization.

Use CONFIG to inspect the current drive settings:

```
A0>CONFIG /SHOW
```

If the disk remains readable, preserve important files before attempting further modifications.

Disk Write Error

Meaning: A requested disk write failed.

Possible causes include write protection, a full disk, damaged media, or a hardware problem.

What to do: Check the disk's write-protection setting and available space.

If the error occurs during a destructive disk operation, do not assume that the destination remains complete or usable.

Disk Full

Meaning: The disk cannot accommodate the requested file data or directory entries.

CP/M disks have limits on both data allocation and directory capacity.

A disk may run out of directory entries even when some data blocks remain free.

What to do: Remove unnecessary files, copy files to another disk, or use a suitable disk with greater capacity.

Read-Only Disk

Meaning: The operating system is refusing writes to the selected disk.

This may result from an explicit read-only condition or from disk-change protection.

What to do: Determine why the disk is read-only before attempting further writes.

If removable media were changed, use the appropriate disk-reset or warm-boot procedure.

Do not force writes to a disk whose directory state may be inconsistent with the operating system's cached information.

Unavailable Drive

Meaning: The requested logical drive is not configured or cannot be accessed.

What to do: Examine the drive assignments with CONFIG.

A logical drive may have been unbound, assigned to an unavailable physical drive, or configured with an unsuitable format.

F.4 DUP Messages

DUP reports errors encountered while formatting, copying, and checking disks.

```
Format complete.
```

Meaning: DUP completed the requested formatting operation and its verification.

This is a successful completion message.

```
Copy complete; destination verified.
```

Meaning: DUP copied the represented sector contents and verified the destination against the source data.

This is a successful completion message.

```
Check complete. Unreadable sectors: n
```

Meaning: DUP completed a physical readability check.

The number `n` identifies the unreadable physical sectors encountered.

For example:

```
Check complete. Unreadable sectors: 0
```

indicates that the examined sectors were readable.

It does not establish that the filesystem or individual files are logically correct.

```
Format aborted. Disk is only partly formatted.
```

Meaning: Formatting was interrupted after the operation began.

The destination disk may not be usable until it is formatted again.

What to do: Correct the cause of the interruption and repeat formatting when safe.

```
Operation stopped. Destination may be partly written.
```

Meaning: A disk-copy operation was interrupted.

The source remains unchanged, but the destination may contain only part of the intended copy.

What to do: Do not rely on the destination as a complete backup. Repeat the operation from a reliable source.

```
Check stopped. No disk contents changed.
```

Meaning: A disk check was interrupted.

CHECK is read-only, so the disk was not modified by the operation.

However, the check is incomplete.

```
Cannot restore destination configuration. Press a key to retry.
```

Meaning: DUP could not restore a temporary destination-drive assignment after a disk-copy operation.

DUP retains the information required for restoration and requests another attempt.

What to do: Follow the prompt and retry.

Avoid resetting the system merely to dismiss the message, since the temporary configuration may still be active.

Disk Error Status

DUP may report a numeric disk status together with the affected cylinder, side, sector, or record.

The following status information is useful when interpreting those reports.

Status	Meaning
1	Request failed or drive did not respond
2	Unsupported operation or disk address
FEh	Read-back data did not match
Bit 7	Drive not ready
Bit 6	Disk write-protected
Bit 4	Sector or track not found
Bit 3	CRC error
Bit 2	Data transfer failed
Other	Disk I/O failure

The bit descriptions identify conditions reported through the disk interface. They are not necessarily mutually exclusive.

For additional information, see Section 16.4, *DUP*.

F.5 SYSGEN Messages

SYSGEN reports conditions encountered while installing BetterCP/M or saving system-image packages.

```
Physical drive 0 is protected as the running system disk.
```

Meaning: The requested installation destination resolves to physical drive 0.

BetterCP/M protects this drive against SYSGEN destination writes.

What to do: Select another suitable physical drive.

Changing the logical drive letter does not bypass this protection.

```
Source and destination refer to the same physical drive.
```

Meaning: A disk-source installation would read and write through the same physical drive.

SYSGEN requires different physical drives for disk-to-disk installation.

What to do: Select a different destination.

```
The selected drive is undefined or unavailable.
```

Meaning: SYSGEN could not obtain a valid drive assignment for the requested source or destination.

What to do: Examine the drive configuration and correct the assignment before retrying.

```
Destination reserved system area is too small (160 records required).
```

Meaning: The destination does not provide enough reserved system space for BetterCP/M 1.0.

SYSGEN requires at least 160 reserved 128-byte logical records.

What to do: Use a disk format with a sufficiently large reserved system area.

SYSGEN cannot create additional reserved space by itself.

```
Destination bootstrap geometry is not supported by this system package.
```

Meaning: The destination's bootstrap geometry is incompatible with the system image being installed.

What to do: Use a compatible destination format or a system image appropriate to the destination.

SYSGEN does not perform disk-format conversion.

```
The system package is invalid or cannot represent the destination.
```

Meaning: The system-image package failed validation or cannot be installed safely using the destination's configuration.

What to do: Check the package and destination format.

If necessary, obtain a valid system package or recreate it from a known working installation.

```
The destination system area will be replaced.
```

Meaning: SYSGEN is requesting confirmation before installation.

The accompanying message explains that files outside the reserved system tracks will be preserved.

Only `Y` authorizes installation.

```
System installed and verified. The destination is bootable.
```

Meaning: SYSGEN completed installation and verified the written system records.

This is a successful completion message.

It does not mean that all ordinary utility files or startup-support files have been copied to the destination.

```
Installation stopped. The destination may not boot.
```

Meaning: Installation was interrupted after system-area writing began.

The destination may contain an incomplete operating system.

The filesystem area remains outside SYSGEN's write range.

What to do: Repeat installation from a valid source after correcting the cause of the interruption.

```
Package exported and verified.
```

Meaning: SYSGEN successfully saved and verified an installed operating-system image as a package file.

The package contains the operating system and its saved configuration, but not ordinary files from the source disk.

```
Replace existing package (Y/N)?
```

Meaning: The requested package filename already exists.

Only `Y` authorizes replacement.

Read-only and system files remain protected.

SYSGEN Recovery Files

During package export, SYSGEN may use:

```
SYSGEN.$$$
SYSGEN.BAK
```

If an export fails, one or both files may remain.

These are recovery files, not necessarily disposable temporary files.

What to do: Examine the reported failure and preserve the files until you've determined whether they contain a valid package or the previous version.

For additional information, see Section 16.5, *SYSGEN*.

F.6 Configuration Messages

CONFIG reports errors involving startup settings, drive definitions, disk formats, and saved configuration information.

Invalid Configuration Value

Meaning: A requested configuration value is outside its permitted range or is not supported by the selected field.

For example, a physical-drive profile may specify an unsupported number of tracks or sides.

What to do: Check the accepted values in the CONFIG reference.

Do not assume that a value accepted by one platform is suitable for another.

Incompatible Drive Assignment

Meaning: The requested disk-format definition is incompatible with the selected physical drive.

What to do: Verify the drive's physical characteristics and the disk-format definition.

A format that requires more tracks or sides than the drive supports cannot be assigned safely.

```
Drive profile changed; binding cleanup incomplete
```

Meaning: CONFIG changed a physical-drive profile but could not complete all associated logical-binding checks or adjustments.

The physical profile change has already occurred.

What to do: Examine the affected logical-drive assignments and correct any incompatible bindings.

Do not assume that the previous configuration remains completely intact.

Configuration Verification Failure

Meaning: CONFIG could not verify the installed system information needed for the requested configuration operation.

This can occur when the system image is damaged or CONFIG does not match the installed operating-system build.

What to do: Use the CONFIG utility supplied with the corresponding BetterCP/M system build.

If necessary, restore a known working system image.

Save Failure

Meaning: CONFIG could not successfully write or verify the requested saved configuration.

What to do: Check the disk and follow the reported recovery instructions.

A failed save should not be treated as evidence that the new configuration will be available after a cold boot.

For additional information, see Section 16.1, *CONFIG*.

F.7 Resident Extension Messages

Resident extensions may report errors when being loaded, unloaded, or restored.

CPX Load Failure

An optional CPX that fails validation may produce:

```
CPX?
```

The system continues with the remaining usable command extensions.

The affected commands are unavailable until the extension is repaired or removed.

RSX Load Failure

Meaning: The requested RSX could not be installed.

Possible causes include a missing or damaged extension file, an incompatible extension, insufficient memory, or an unavailable dependency.

What to do: Check that the extension file exists and that its required supporting components are installed.

For example, a terminal personality requires a compatible screen backend.

RSX Unload Failure

Meaning: The requested extension could not be removed.

One possible cause is that another resident extension still depends on it.

What to do: Examine the installed extensions:

```
A0>RSX LIST
```

Unload dependent extensions before removing their supporting components.

Insufficient Resident Memory

Meaning: The requested extension cannot be installed because insufficient suitable memory is available.

What to do: Unload unnecessary extensions and retry.

Resident extensions reduce the memory available to transient programs.

For additional information, see Chapter 8, *Resident Extensions*, and Chapter 17, *Batch and Extension Commands*.

F.8 Date and Time Messages

TIME reports errors when no clock provider is available, when a provider cannot obtain valid time information, or when an unsupported clock-setting operation is requested.

Clock Service Unavailable

Meaning: TIME cannot access an installed clock provider.

What to do: Load the appropriate provider for the platform.

For example:

```
A0>RSX LOAD FREHDCLK
```

or:

```
A0>RSX LOAD ZPRTC
```

Then retry:

```
A0>TIME
```

Clock Read Failure

Meaning: The installed provider could not obtain a valid date and time.

Possible causes include unavailable hardware, an emulator-interface problem, or invalid clock data.

What to do: Check the platform's clock configuration.

TIME cannot correct an inaccurate or unavailable underlying clock.

Clock Setting Unsupported

Meaning: TIME accepted a SET request, but the installed clock provider does not support changing the clock.

Both BetterCP/M 1.0 providers, `FREHDCLK.RSX` and `ZPRTC.RSX`, are read-only.

What to do: Adjust the clock through the hardware or host environment, if supported.

The TIME utility's SET syntax does not make a read-only provider writable.

Invalid SET Arguments

Meaning: The arguments supplied to TIME's SET operation are malformed or outside the accepted ranges.

The command requires five hexadecimal bytes:

```
TIME /SET LL HH HR MN SC
```

These represent the native clock record, not an ordinary calendar-date string.

What to do: Check the hexadecimal values and the required date-and-time representation.

For additional information, see Section 16.3, *TIME*.

F.9 Common Interactive Prompts

Some BetterCP/M utilities provide interactive prompts when an operation requires a decision.

The available responses depend on the utility and the operation.

Yes/No Confirmation

Many potentially destructive operations require explicit confirmation.

For example, SYSGEN asks:

```
Proceed (Y/N)?
```

Only `Y` authorizes the operation.

Other responses cancel it.

Do not assume that command-line invocation bypasses confirmation. DUP and SYSGEN retain their destructive-operation safeguards even when invoked with complete command-line arguments.

COPY File-Collision Prompt

When COPY encounters an existing writable destination file, the transient utility may offer:

```
Y/N/O/S/R/?
```

Response	Meaning
Y	Replace this file

Response	Meaning
N	Skip this file
O	Replace this and subsequent writable collisions
S	Skip this and subsequent writable collisions
R	Enter a new destination filename
?	Display help

The O and S policies apply only to the current COPY invocation.

REN Name-Collision Prompt

When transient REN encounters an existing destination filename, it may offer:

```
File exists. N/S/A/?
```

Response	Meaning
N	Enter a new filename
S	Skip the file
A	Abort the remaining operation
?	Display help

REN never overwrites an existing destination.

Retry, Skip, or Abort

Some utilities offer recovery choices when an individual file operation fails.

For example:

```
R/S/A/?
```

The responses are:

Response	Meaning
R	Retry the operation
S	Skip the affected file
A	Abort the remaining operation
?	Display help

The availability of these choices depends on the utility.

Noninteractive modes may follow a predetermined error policy instead of displaying a recovery prompt.

Help at Prompts

When `?` appears among the available responses, entering it displays an explanation of the choices.

Requesting help does not itself retry, skip, abort, or authorize an operation.

The utility returns to the same decision after displaying the help text.

F.10 Interpreting Error Messages

When an error occurs, the most useful first step is to determine which component reported it.

A command-processor error usually concerns command recognition or program execution.

A file-management error may involve filenames, attributes, destination conflicts, or disk space.

A disk utility error may involve physical media, drive configuration, or unsupported operations.

A resident-extension error may involve missing files, incompatible components, dependencies, or memory limitations.

The following table provides a quick guide.

Symptom	Likely area to investigate
Command not recognized	Command name, `.COM` file location, CPX availability
File not found	Filename, drive/user area, wildcard or attribute selection
Destination already exists	COPY, MOVE, or REN collision handling
File cannot be modified	Read-only or system attributes
Disk cannot be read	Media, drive configuration, disk format
Disk cannot be written	Write protection, capacity, media errors
Program has insufficient memory	Loaded resident extensions, available TPA
Full-screen display is incorrect	Terminal personality, screen backend, application configuration
TIME cannot display the clock	Clock provider or underlying clock source
System fails after configuration changes	Saved drive settings, startup command, system image
System disk will not boot	Bootstrap compatibility, incomplete installation, damaged media

When an Operation Fails Partway Through

Some BetterCP/M operations process several files or disk records in sequence.

If an error occurs, earlier operations may already have completed.

For example, a multi-file COPY may have created several destination files before encountering an error.

A multi-file REN may have renamed some files before encountering a name conflict.

An interrupted DUP copy may leave the destination partly written.

An interrupted SYSGEN installation may leave the destination's system area incomplete.

Do not assume that an interrupted operation has been automatically undone.

Examine the affected files or disks before retrying.

Preserving Evidence of a Problem

If an error appears to be reproducible, record its exact wording.

Also note the command entered, the current drive and user area, and any relevant configuration information.

For problems involving resident extensions, the output of `CPX LIST` or `RSX LIST` may be useful.

For disk-related problems, record the drive assignments and disk format.

When reporting a compatibility problem, identify the application, BetterCP/M version, and computer or emulator.

See Appendix E for additional guidance on compatibility reporting.

Appendix G - Glossary

This glossary defines the principal terms used throughout the BetterCP/M User's Guide.

It includes terminology inherited from Digital Research CP/M 2.2, terms associated with BetterCP/M's extensions, and expressions commonly encountered when working with CP/M computers and software.

The definitions are intended to help readers understand the manual rather than provide exhaustive technical descriptions. Where a subject requires more detailed treatment, references to the relevant chapters are provided.

A

Allocation Block
A unit of disk space allocated to a file by the CP/M filesystem. An allocation block consists of one or more 128-byte logical records. The allocation-block size depends on the disk format.

Archive Attribute (ARC)
A file attribute commonly used by backup utilities. BetterCP/M can select files according to their ARC status and provides facilities for clearing the attribute after successful backup operations. The attribute does not necessarily indicate whether a file has changed unless the software modifying it maintains the attribute appropriately. See Appendix D.

ASCII (American Standard Code for Information Interchange)
A character-encoding standard used by CP/M for text and control characters. Standard ASCII defines 128 character codes, including printable characters and control codes such as carriage return, line feed, and Escape.

Attribute
Information associated with a file that affects its treatment by the operating system or utilities. CP/M attributes include read-only, system, and archive. BetterCP/M provides extended facilities for selecting files according to their attributes.

B

Batch File
A text file containing commands to be executed in sequence. BetterCP/M uses `.SUB` files with the SUBMIT utility for batch processing. See Chapter 6.

BDOS (Basic Disk Operating System)
The portion of CP/M that provides standard operating-system services to programs, including file management, console input and output, and other system functions. Programs ordinarily access BDOS through the conventional CP/M calling interface.

BIOS (Basic Input/Output System)
The hardware-dependent portion of CP/M. The BIOS provides low-level services for disk drives, the console, and other devices. Different computers require BIOS implementations suited to their hardware.

Boot
The process of starting the operating system. A cold boot initializes a new operating session, while a warm boot returns control to the operating environment without performing a complete cold startup.

Boot Disk
A disk containing the operating-system image and bootstrap information required to start the computer. A boot disk may also contain ordinary programs and data files.

Bootstrap
The initial program or sequence of operations that loads the operating system into memory and transfers control

to it.

BS (Backspace)
ASCII control character `08H`, commonly generated by Ctrl-H or a Backspace key. Its treatment depends on the console or application.

C

CCP (Console Command Processor)
The portion of CP/M responsible for accepting and interpreting commands entered at the console. BetterCP/M extends the conventional command environment through resident commands, command history, and Command Processor Extensions.

Cold Boot
A complete operating-system startup that establishes a new session and initializes the configured environment. BetterCP/M executes its saved startup command during a cold boot. Command history begins empty, and the configured startup environment is established.

Command History
A collection of previously entered commands retained by BetterCP/M's command processor. Commands can be recalled, edited, and executed again. BetterCP/M 1.0 uses Ctrl-K and Ctrl-J for history navigation. See Appendix B.

Command Processor Extension (CPX)
A resident extension that adds commands to BetterCP/M's command processor. The standard resident command package is `RCP.CPX`. CPXs are managed using the CPX utility. See Chapter 8.

Command Tail
The text following a transient program's name on the command line. CP/M makes this text available to the program using its conventional command-tail representation.

Console
The keyboard and display, terminal, or equivalent device through which the user communicates with the operating system.

Control Character
An ASCII character traditionally used for a control operation rather than as a printable symbol. Examples include carriage return, line feed, backspace, and Escape. See Appendix C.

CP/M (Control Program for Microcomputers)
The operating system developed by Digital Research for microcomputers, particularly those using Intel 8080-compatible processors. CP/M 2.2 is the principal compatibility target for BetterCP/M.

CP/M 2.2
The widely used Digital Research CP/M release that provides the standard operating-system interface and behavior BetterCP/M seeks to reproduce.

CPX
See *Command Processor Extension*.

Current Configuration
The system settings presently in use. These may differ from the settings saved for subsequent cold boots. See Chapter 7.

D

DDT (Dynamic Debugging Tool)
Digital Research's machine-language debugging utility. DDT provides facilities for examining and modifying memory, disassembling instructions, and controlling program execution.

Directory
The portion of a CP/M filesystem containing information about files, including their names, user areas, attributes, and allocated storage. Unlike hierarchical filesystems, conventional CP/M directories do not contain subdirectories.

Directory Entry
A record in the CP/M directory describing a file or part of a file. A file may require several directory entries depending on its size and the disk format.

Disk Format
The organization of a disk, including its physical recording characteristics and filesystem parameters. CP/M computers used many different formats, not all of which are interchangeable.

Disk-Format Definition
A description of a disk's organization used by BetterCP/M to access it. Format definitions are selected and managed through CONFIG.

Drive
A logical or physical storage device. CP/M identifies logical drives by letters such as A and B.

Drive/User (DU)
A particular combination of logical drive and user area, such as `A0:` or `B3:`. BetterCP/M uses drive/user specifications extensively in its enhanced file-management commands.

DU Selector
A BetterCP/M file-selection expression identifying one or more drive/user areas. For example, `B[-]:` selects all user areas on drive B. See Appendix D.

E

ED
Digital Research's traditional CP/M context editor. ED is a line-oriented text editor operated through commands rather than a full-screen display.

End-of-Text Marker
The Ctrl-Z character (`1AH`), traditionally used to indicate the end of meaningful text in CP/M text files.

Escape Sequence
A sequence of character codes, commonly beginning with ASCII Escape (`1BH`), used to request terminal operations such as cursor movement or screen clearing.

Extent
A unit of file organization represented by CP/M directory information. Large files may occupy multiple extents and directory entries.

F

FCB (File Control Block)
A data structure used by CP/M programs to identify files and perform file operations through BDOS. Conventional CP/M programs use FCBs for opening, reading, writing, and managing files.

File Attribute
See *Attribute*.

File Specification (Filespec)
A filename, optionally accompanied by a drive/user qualification, wildcard characters, or supported attribute expressions. For example, `B3:*.COM[$ARC]` is an extended BetterCP/M file specification.

File Type
The optional portion of a CP/M filename following the period. A file type contains up to three characters.

Common examples include `.COM`, `.ASM`, `.TXT`, and `.SUB`.

FreHD
A storage peripheral for the TRS-80 Model 4 and related systems. BetterCP/M provides a FreHD clock provider through `FREHDCLK.RSX`.

H

Hexadecimal
A base-16 number system using digits 0-9 and letters A-F. Hexadecimal is commonly used when displaying memory addresses, machine instructions, and byte values. In this manual, hexadecimal values may be identified by an `H` suffix.

Host Computer
The computer running an emulator or providing services to an emulated CP/M system. For example, z80pack may obtain date and time information from its host environment.

I

IOBYTE
A conventional CP/M system value used to select logical character-device assignments. BetterCP/M retains the IOBYTE mechanism. STAT manages the live value, while CONFIG can save a value for subsequent disk cold boots.

L

Logical Drive
A drive identified by a CP/M drive letter, such as A or B. BetterCP/M associates logical drives with physical devices and disk-format definitions.

Logical Record
The standard 128-byte unit used by CP/M file and disk services. Physical disk sectors may contain one or more logical records.

M

Montezuma Micro
A company known for its CP/M implementation for the TRS-80 Model 4. BetterCP/M retains compatibility with selected Montezuma Micro conventions, including its ADM-3A terminal behavior and keyboard arrangements.

P

P2DOS
A CP/M-compatible operating-system implementation that introduced additional services, including date-and-time facilities. BetterCP/M provides optional P2DOS-compatible clock services through `P2DOS.RSX`.

Physical Drive
The actual disk drive or storage device associated with a logical drive. BetterCP/M distinguishes physical-drive characteristics from logical-drive assignments.

PIP (Peripheral Interchange Program)
The traditional CP/M utility for copying and transferring data between files and devices. PIP supports a variety of text and data-transfer options.

Program Image
The machine-language instructions and associated data of a program as stored in a file or loaded into memory.

R

RCP (Resident Command Package)
The standard BetterCP/M Command Processor Extension, supplied as `RCP.CPX`. It provides frequently used commands such as DIR, ERA, REN, TYPE, COPY, MOVE, CLS, and VER.

Read-Only Attribute (RO)
A file attribute that protects a file against ordinary modification or deletion. BetterCP/M utilities may provide explicit facilities for handling protected files.

Resident Command
A command available directly through the command processor or an installed CPX, without loading a separate transient `.COM` program.

Resident System Extension (RSX)
A loadable component that provides services while other programs execute. BetterCP/M uses RSXs for terminal support, clock providers, batch input, and other facilities. See Chapter 8.

RSX
See *Resident System Extension*.

S

Saved Configuration
System settings stored for use during subsequent cold boots. Saved settings may differ from those currently active.

Sector
A physical unit of disk storage. Sector sizes and organization depend on the disk format. A physical sector may contain several CP/M logical records.

SUBMIT
The CP/M batch-processing utility that executes commands stored in `.SUB` files. BetterCP/M retains the familiar SUBMIT conventions. See Chapter 6.

System Area
The reserved portion of a bootable CP/M disk containing operating-system and bootstrap information. SYSGEN installs BetterCP/M into this area.

System Attribute (SYS)
A file attribute that ordinarily suppresses a file from conventional directory listings. The attribute does not necessarily mean that the file is part of the operating system.

System Image
The operating-system code and associated configuration information installed in a disk's reserved system area. BetterCP/M system images can be saved and installed using SYSGEN.

SYSTEM.SYS
The conventional filename for a BetterCP/M system-image package. Such packages contain the operating-system image and associated installation information.

T

Terminal
A device or emulated environment providing keyboard input and character-oriented display output.

Terminal Monitor
A resident facility that interprets supported terminal-control sequences and performs the corresponding display operations through a screen backend.

Terminal Personality
An RSX that interprets the control sequences associated with a particular terminal type. BetterCP/M provides ADM-3A, VT52, and ANSI/VT100-subset personalities.

TPA (Transient Program Area)
The region of memory available for ordinary CP/M programs. Resident extensions reduce the amount of TPA available to transient programs.

Transient Command
A command implemented by a program loaded from disk, normally as a `.COM` file. BetterCP/M can explicitly invoke a transient command using a colon or period prefix.

U

User Area
A numbered file-organization area within a CP/M drive. Conventional CP/M 2.2 supports user areas 0-15; BetterCP/M extends the range to 0-31.

V

VT52
A DEC video terminal whose control sequences were supported by many CP/M applications. BetterCP/M provides a compatible terminal personality through `VT52.RSX`.

VT100
A DEC video terminal associated with ANSI-style control sequences. BetterCP/M provides a supported subset of ANSI/VT100 behavior through `ANSI.RSX`.

W

Warm Boot
A return to the CP/M command environment without performing a complete cold startup. BetterCP/M preserves supported resident extensions and command history across ordinary warm boots.

Wildcard
A character used in a file specification to select several files. CP/M uses `?` and `*` as wildcard characters. BetterCP/M provides consistent bounded wildcard rules in its enhanced file-selection facilities.

WordStar
A widely used CP/M word processor whose control-key conventions influenced many other programs. BetterCP/M uses some WordStar-style editing conventions, although its command-history assignments differ.

X

XSUB
A CP/M batch-processing extension that allows compatible programs to receive submitted input through buffered console-input services. BetterCP/M provides this facility using resident batch-input support.

Z

Z80
An 8-bit microprocessor developed by Zilog and widely used in CP/M computers. It is compatible with the Intel 8080 instruction set while providing additional instructions and facilities.

z80pack
A collection of Z80 emulation tools, including cpmsim. BetterCP/M supports z80pack environments and provides a clock provider through `ZPRTC.RSX`.

Appendix H - Further Reading and Technical Documentation

BetterCP/M is accompanied by documentation covering its operation, programming interfaces, internal organization, and compatibility with Digital Research CP/M 2.2.

This User's Guide is intended primarily for people using BetterCP/M. It explains how to operate the system, manage files and disks, configure the environment, and use the supplied commands and utilities.

Readers who want to write programs, adapt BetterCP/M to another computer, investigate compatibility questions, or study the operating system's implementation will find additional information in the technical documentation.

This appendix describes the principal documentation resources and explains where to look for further information.

H.1 BetterCP/M Documentation

The BetterCP/M documentation is organized around the needs of different readers.

Some documents introduce the operating system and its facilities. Others serve as programming references or provide detailed information for developers and system maintainers.

The principal documentation categories are described below.

User's Guide

Audience: BetterCP/M users.

The *BetterCP/M User's Guide* is the document you are reading.

It covers the operating environment, everyday commands, file management, disk operations, configuration, resident extensions, and other facilities needed for normal use.

Part I introduces the system and explains common tasks.

Part II provides detailed command and utility references.

The appendices summarize commands, keyboard conventions, file specifications, compatibility considerations, and system messages.

The User's Guide is not intended to describe the internal implementation of every facility.

Technical Manuals and Architecture References

The README documentation plan groups the developing manuals into User Documentation, Programmer Documentation, Technical Manuals, and a Build & Development Guide. These families are still being developed; they are not four completed books.

There is no separate completed manual titled *Architecture Overview* in the current repository. Start with Overall System Architecture and the architecture specifications for the current system contracts.

Programmer Documentation

Use the actual working reference, BetterCP/M RSX and CPX Programmer's Guide, for extension interfaces. It records its own status and includes later-version proposals, which are not promises of installed 1.0 features. Developing monitor RSXs covers monitor development; Runtime disk configuration, ABI version 5 covers the disk configuration interface.

This manual uses *Programmer Documentation* for the planned family and the exact titles above for available references. Earlier references to a generic *Developer's Guide* do not identify an additional completed manual.

For ordinary CP/M applications, also consult the original Digital Research interface documentation.

Build, Platform Preparation and Porting

The consolidated *Build & Development Guide* is being developed; there is no separate completed *Porting Guide* at present. Current practical entry points are:

- Building the current BetterCP/M System Disks - host-side distribution assembly and its current status.
- BetterCP/M build disk - self-hosted native build, source volumes and installation.
- Default high-capacity test disks - Model 4 test-disk preparation and launcher, with historical qualification notes.
- z80pack / cpmsim adapter - that target's builder, launcher, media and platform requirements.
- Hardware Abstraction and platform specifications - current architecture and platform material for maintainers.

Engineering Specifications and History

The engineering records contain implementation specifications, decisions and dated qualification results. They are supporting technical records, not a substitute for a finished user or porting manual. Check each document's status and date against the current source, architecture specifications, and 1.0 implementation contracts; an older milestone does not override later contracts or establish current release qualification.

Compatibility Documentation

Audience: Developers, system maintainers, and users investigating differences between BetterCP/M and historical CP/M implementations.

BetterCP/M's compatibility documentation examines the behavior of conventional CP/M interfaces and utilities.

It records compatibility requirements, implementation differences, and results of compatibility testing.

This documentation is useful when investigating a program that behaves differently under BetterCP/M than under Digital Research CP/M 2.2.

Appendix E provides a user-level introduction to these issues.

For detailed compatibility analysis, see the compatibility documentation and the README compatibility and testing section.

H.2 Utility Documentation

BetterCP/M includes utilities ranging from simple commands to substantial programs with their own command languages and operating conventions.

The frequently used commands are documented in Part II of this manual.

More substantial utilities may have separate documentation.

This follows a practice familiar from Digital Research CP/M, where programs such as ED, ASM, and DDT were documented individually.

Standard Utilities

The User's Guide provides references for the principal BetterCP/M commands and system utilities.

These include:

Utility	Documentation
DIR, ERA, REN, TYPE	Chapter 14
COPY, MOVE	Chapter 14

Utility	Documentation
USER, CLS, VER	Chapter 15
GET, GO, JUMP, PEEK, POKE, SAVE	Chapter 15
CONFIG, STAT, TIME	Chapter 16
DUP, SYSGEN	Chapter 16
SUBMIT, XSUB	Chapter 17
CPX, RSX	Chapter 17

These references describe the commands available to users, their syntax, options, and important restrictions.

Development Utilities

The System Disk supplies ED. The separate Utilities Disk selection includes ZSM4 and LINK. Traditional tools such as ASM, LOAD and DDT must be obtained separately if needed; they are not selected BetterCP/M distribution contents.

These familiar tools have distinct purposes:

ASM - An assembler for Intel 8080 assembly-language programs.

LOAD - A utility for converting Intel hexadecimal object files into executable CP/M programs.

DDT - A machine-language debugger.

ED - A line-oriented context editor.

These programs have their own operating conventions and may require more extensive documentation than is appropriate for a single User's Guide entry.

Where separate manuals are provided, they should be consulted for complete instructions.

Other Utilities

Additional programs may be supplied for system construction, diagnostics, terminal testing, or specialized file operations.

For example, SYSBUILD is a system-construction tool, not a selected System Disk utility. See Section 3.2, *What Is on Each Disk*, for the distribution inventory.

Terminal test utilities such as ADMTST, VT52TST, and ANSITST help verify supported terminal environments.

The documentation accompanying each utility describes its intended use and any special requirements.

Not every utility necessarily belongs in the User's Guide. Some are primarily intended for developers or system maintainers.

H.3 Digital Research CP/M Documentation

Digital Research's original CP/M documentation remains an important reference for BetterCP/M users and programmers.

BetterCP/M preserves the principal CP/M 2.2 programming conventions, so the original manuals provide useful background and detailed descriptions of the historical operating system.

The following publications are particularly relevant.

CP/M Operating System Manual

The *CP/M Operating System Manual* provides an introduction to CP/M and its operation.

It covers the command environment, file organization, system utilities, and other facilities of the original operating system.

Readers familiar with BetterCP/M may find it useful for understanding the historical conventions retained by the newer system.

CP/M 2.2 Interface Guide

The *CP/M 2.2 Interface Guide* describes the programming interfaces provided by Digital Research CP/M.

It is particularly important for programmers using BDOS and BIOS services.

The guide documents the conventional calling interfaces and system behavior expected by CP/M applications.

A documentation caution: Surviving editions and reprints of the CP/M Interface Guide are not necessarily identical. In particular, some editions omit the addendum describing BDOS functions 37 and 40.

Programmers should ensure that their reference includes the CP/M 2.2 additions rather than relying solely on the main function summary, which may end at function 36.

CP/M 2.2 Alteration Guide

The *CP/M 2.2 Alteration Guide* describes the procedures and interfaces involved in adapting CP/M to different computer systems.

It is primarily intended for system programmers and hardware developers.

The document is useful for understanding the historical division between the hardware-independent operating system and the machine-specific BIOS.

BetterCP/M's own porting and engineering documentation should be used when adapting BetterCP/M itself.

ED, ASM, and DDT Documentation

Digital Research also published documentation for its principal development utilities.

These include:

ED - The CP/M context editor.

ASM - The CP/M assembler.

DDT - The Dynamic Debugging Tool.

These manuals remain useful for understanding the conventions and command languages of the traditional CP/M development environment.

BetterCP/M's corresponding utility documentation should be consulted for any implementation-specific differences.

H.4 Other Historical CP/M Documentation

Digital Research was not the only source of CP/M documentation.

Computer manufacturers, software publishers, and independent authors produced manuals describing their own CP/M implementations and programming environments.

These publications can be particularly useful when working with historical hardware or software.

Manufacturer Documentation

Manufacturers frequently supplied CP/M manuals adapted to their computers.

These might describe the machine's BIOS, disk formats, terminal characteristics, peripheral devices, and installation procedures.

Examples include documentation published for systems from Morrow Designs, North Star Computers, Heath, and other CP/M manufacturers.

Such manuals can help explain why software behaves differently on particular computers.

However, manufacturer-specific conventions should not automatically be treated as standard CP/M behavior.

Independent Programming References

Independent books and programming guides also provide valuable information about CP/M.

For example, Sol Libes's *Programmer's Guide to CP/M* discusses programming techniques and the operating-system environment from the perspective of contemporary CP/M development.

These publications may contain practical examples and explanations not found in Digital Research's official manuals.

As with manufacturer documentation, readers should distinguish standard CP/M interfaces from implementation-specific techniques.

Historical Documentation and BetterCP/M

Historical manuals are especially useful when investigating compatibility questions.

For example, a program may rely on a particular terminal convention, disk layout, or undocumented operating-system behavior.

Contemporary documentation can help identify the assumptions under which the program was originally written.

BetterCP/M's compatibility documentation records the results of investigations into such behavior.

H.5 Online Resources

The BetterCP/M source repository is the principal online resource for current project documentation.

BetterCP/M Repository

The project is maintained at:

https://github.com/CPMArchives/BetterCPM

The repository contains source code, documentation, engineering specifications, and other project materials.

The main documentation directory is:

```
docs/
```

Engineering specifications are maintained under:

```
docs/engineering/
```

Readers should consult the repository for the latest documentation corresponding to the current BetterCP/M development state.

Digital Research Documentation Archives

Historical CP/M documentation is available through several online archives.

One particularly useful collection is the Digital Research library maintained at:

http://www.cpm.z80.de/drilib.html

This collection includes scans of original Digital Research manuals and other CP/M-related publications.

It is a valuable resource for comparing different editions of the documentation and examining the original programming interfaces.

Additional CP/M Resources

Other online archives preserve CP/M software, disk images, manuals, source code, and historical discussions.

These resources are useful when locating original application documentation, studying machine-specific implementations, or investigating compatibility problems.

When using archived software or documentation, pay attention to the version and target computer.

A manual describing one manufacturer's CP/M implementation may not accurately describe another.

H.6 Choosing the Right Documentation

The following table summarizes where to begin when looking for additional information.

If you want to...	Consult...
Learn to use BetterCP/M	User's Guide, Part I
Look up a command or utility	User's Guide, Part II
Find command syntax quickly	Appendix A
Learn command-line editing keys	Appendix B
Understand control characters	Appendix C
Use extended file specifications	Appendix D
Understand CP/M compatibility differences	Appendix E
Interpret system messages	Appendix F
Look up terminology	Appendix G
Understand BetterCP/M's internal organization	Overall System Architecture
Write applications or extensions	RSX and CPX Programmer's Guide; original CP/M interfaces for ordinary applications
Adapt BetterCP/M to another computer	Hardware Abstraction, platform specifications and current source; consolidated porting guidance is planned
Modify or maintain an operating-system component	Engineering records and current architecture contracts
Investigate compatibility behavior	Compatibility documentation
Study original CP/M programming interfaces	Digital Research CP/M 2.2 Interface Guide
Study historical CP/M system adaptation	Digital Research CP/M 2.2 Alteration Guide
Use a substantial development utility	Its individual utility manual

The User's Guide should ordinarily provide enough information for everyday operation.

When a subject requires deeper technical treatment, the appropriate programming, architecture, or engineering document provides the next level of detail.